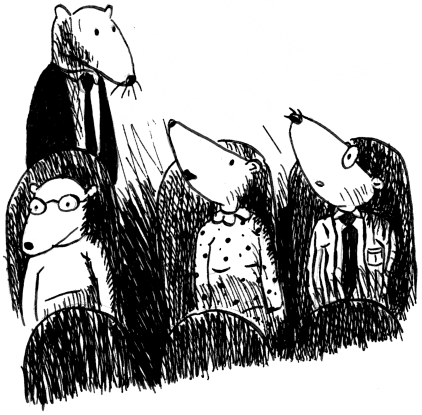
INSTITUTO FEDERAL
Brasília
Campus São Sebastião



drawn by adit.io

Sistemas Computacionais

Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

TADS

Eduardo Camargo de Siqueira

22 de maio de 2026

Apresentação

Esta apostila foi elaborada como material de apoio à disciplina *Sistemas Computacionais*, ofertada no curso de Tecnologia em Análise e Desenvolvimento de Sistemas.

Seu objetivo é apresentar, de forma introdutória e aplicada, os fundamentos necessários para que o estudante compreenda o funcionamento dos computadores e sua relação com o desenvolvimento de software. Ao longo do material, são discutidos temas como organização básica de um sistema computacional, representação de dados, sistemas operacionais, arquivos, diretórios, processos, virtualização, contêineres, redes, endereçamento IP e ferramentas de acesso remoto.

A proposta da apostila é combinar explicações conceituais, exemplos didáticos, atividades práticas e exercícios reflexivos. Sempre que possível, os conteúdos são relacionados a situações reais da área de tecnologia, aproximando a teoria da prática profissional de quem desenvolve, configura, executa e mantém sistemas computacionais.

Este material foi produzido por **Eduardo Camargo de Siqueira** e está disponível livremente para uso não comercial, especialmente em contextos pessoais, educacionais e didáticos. É permitida a utilização, cópia, adaptação, modificação, mesclagem, publicação e distribuição gratuita deste conteúdo, desde que preservada a finalidade não comercial.

A venda total ou parcial deste material é proibida.

Caso utilize, adapte ou reproduza qualquer parte deste material, inclua a seguinte indicação de autoria:

© duca.pro

Sugestões, correções, dúvidas e contribuições são bem-vindas. Caso encontre algum erro ou deseje entrar em contato, envie uma mensagem para:

eduardo.siqueira@ifb.edu.br

Espero que este material contribua para sua aprendizagem e ajude a construir uma compreensão mais sólida, crítica e prática sobre os sistemas computacionais que sustentam o mundo digital contemporâneo.

Como usar esta apostila

Cada capítulo está organizado em cinco partes principais:

- objetivos de aprendizagem;
- exposição teórica dos conceitos;
- exemplos didáticos;
- atividades práticas;
- exercícios de fixação e reflexão.

Recomenda-se que você realize as atividades práticas em um computador com sistema operacional Linux, Windows ou macOS. Sempre que possível, serão apresentados exemplos utilizando terminal, máquinas virtuais, contêineres e ferramentas comuns ao desenvolvimento de software.



drawn by adit.io

Sumário

1 Introdução **1**

1.1 O que são sistemas computacionais 1

1.2 Por que estudar sistemas computacionais em ADS? 1

1.3 Breve história dos computadores 2

1.3.1 Das máquinas imaginadas aos primeiros autômatos 2

1.3.2 Máquinas de calcular 3

1.3.3 Babbage, Ada Lovelace e a ideia de computador programável 3

1.3.4 Alan Turing e a base teórica da computação 4

1.3.5 Computadores eletrônicos e a arquitetura de von Neumann 4

1.3.6 Da computação corporativa ao computador pessoal 5

1.3.7 Internet, Web e computação em rede 5

1.3.8 Computação móvel, nuvem e inteligência artificial 5

1.3.9 Linha do tempo resumida 6

1.4 Componentes básicos de um computador 7

1.4.1 Processador 7

1.4.2 Memória principal 8

1.4.3 Dispositivos de armazenamento 9

1.4.4 Placa-mãe 9

1.4.5 Dispositivos de entrada 10

1.4.6 Dispositivos de saída 11

1.4.7 Placa de vídeo 11

1.4.8 Fonte de alimentação 12

1.4.9 Gabinete e refrigeração 12

1.4.10 Interfaces de rede 13

1.4.11 Como os componentes trabalham juntos 13

1.4.12 Relação com o desenvolvimento de software 14

1.5 O papel do terminal 15

1.5.1 Interface gráfica versus interface de linha de comando 15

1.5.2 Shell, terminal e prompt 16

1.5.3 Por que o terminal é importante para desenvolvedores 17

1.5.4 Comandos como pequenas ferramentas . . . 18
1.5.5 Navegação em diretórios . . . 18
1.5.6 Manipulação de arquivos e diretórios . . . 19
1.5.7 Execução de programas . . . 19
1.5.8 Terminal e processos . . . 20
1.5.9 Terminal e redes . . . 20
1.5.10 Automação com scripts . . . 21
1.5.11 Terminal, Git e desenvolvimento colaborativo . . . 22
1.5.12 Cuidados ao usar o terminal . . . 22
1.5.13 O terminal como ferramenta de pensamento . . . 23

2 Sistemas Numéricos 26
2.1 Por que computadores usam números binários? . . . 26
2.2 Sistema decimal . . . 26
2.2.1 Valor posicional . . . 27
2.2.2 O decimal e a computação . . . 28
2.3 Sistema binário . . . 28
2.3.1 Base 2 e valor posicional . . . 28
2.3.2 Primeiros números em binário . . . 29
2.3.3 Bits e combinações . . . 30
2.3.4 O binário e os computadores . . . 30
2.4 Sistema octal . . . 31
2.4.1 Base 8 e valor posicional . . . 31
2.4.2 Relação entre octal e binário . . . 32
2.4.3 Uso do sistema octal em permissões . . . 32
2.5 Sistema hexadecimal . . . 33
2.5.1 Base 16 e valor posicional . . . 34
2.5.2 Relação entre hexadecimal e binário . . . 35
2.5.3 Hexadecimal em cores digitais . . . 36
2.5.4 Hexadecimal em memória e programação . . . 36
2.5.5 Por que hexadecimal é tão usado em computação? . . . 37
2.6 Conversão entre bases . . . 37
2.6.1 Conversão de decimal para binário . . . 38
2.6.2 Conversão de binário para decimal . . . 39
2.6.3 Conversão de decimal para octal . . . 39
2.6.4 Conversão de octal para decimal . . . 40
2.6.5 Conversão de decimal para hexadecimal . . . 41
2.6.6 Conversão de hexadecimal para decimal . . . 41
2.6.7 Conversão direta entre binário e octal . . . 42

2.6.8 Conversão direta entre binário e hexadecimal . . . 44
2.6.9 Conversão entre octal e hexadecimal . . . 45
2.6.10 Resumo dos métodos . . . 47
2.6.11 Erros comuns em conversões . . . 47
2.6.12 Aplicação prática com Python . . . 48
2.6.13 Atividade prática . . . 49
2.7 Bits, bytes e unidades de medida . . . 51
2.8 Unidades decimais e binárias . . . 51
2.9 Representação de caracteres . . . 51
2.9.1 O que é um caractere? . . . 52
2.9.2 Da letra ao número . . . 52
2.9.3 ASCII . . . 53
2.9.4 Limitações do ASCII . . . 54
2.9.5 Unicode . . . 55
2.9.6 UTF-8 . . . 55
2.9.7 Caracteres e tamanho de arquivos . . . 56
2.9.8 Problemas de codificação . . . 56
2.9.9 Representação de caracteres em HTML . . . 57
2.9.10 Representação de caracteres em Python . . . 58
2.9.11 Caracteres de controle . . . 58
2.9.12 Por que isso importa para o desenvolvimento de sistemas? . . . 59
2.9.13 Atividade prática . . . 60
2.10 Representação de números inteiros . . . 60
2.10.1 Inteiros sem sinal . . . 61
2.10.2 Limite de representação . . . 62
2.10.3 O problema dos números negativos . . . 63
2.10.4 Representação com sinal . . . 63
2.10.5 O bit mais significativo . . . 64
2.10.6 Complemento de dois . . . 65
2.10.7 Interpretando números negativos em complemento de dois . . . 67
2.10.8 Soma com complemento de dois . . . 67
2.10.9 Overflow . . . 68
2.10.10 Inteiros em linguagens de programação . . . 70
2.10.11 Exemplo prático com Python . . . 70
2.10.12 Aplicações práticas . . . 71
2.10.13 Atividade prática . . . 72
2.11 Representação de números decimais . . . 72
2.11.1 Parte inteira e parte fracionária . . . 73
2.11.2 Frações em binário . . . 74

2.11.3 Conversão da parte fracionária para binário . . . 75
2.11.4 Conversão com parte inteira e parte fracionária . . . 76
2.11.5 Nem toda fração decimal tem representação binária finita . . . 77
2.11.6 Ponto fixo . . . 78
2.11.7 Ponto flutuante . . . 79
2.11.8 Precisão e arredondamento . . . 79
2.11.9 Ponto flutuante em linguagens de programação . . . 80
2.11.10 Números decimais em aplicações financeiras . . . 81
2.11.11 Usando `Decimal` em Python . . . 81
2.11.12 Erro absoluto e erro relativo . . . 82
2.11.13 Comparação entre números de ponto flutuante . . . 82
2.11.14 Aplicações práticas . . . 83
2.11.15 Atividade prática . . . 84
2.11.16 Outra atividade: dinheiro como inteiro . . . 84

3 Princípios de Funcionamento de Sistemas Operacionais 86
3.1 O que é um sistema operacional . . . 86
3.2 Funções principais de um sistema operacional . . . 86
3.2.1 Gerenciamento de processos . . . 87
3.2.2 Escalonamento do processador . . . 87
3.2.3 Gerenciamento de memória . . . 88
3.2.4 Gerenciamento de arquivos . . . 89
3.2.5 Gerenciamento de diretórios . . . 89
3.2.6 Controle de permissões e usuários . . . 90
3.2.7 Gerenciamento de dispositivos . . . 91
3.2.8 Gerenciamento de entrada e saída . . . 91
3.2.9 Gerenciamento de rede . . . 92
3.2.10 Interface com o usuário . . . 92
3.2.11 Segurança e proteção do sistema . . . 93
3.2.12 Execução de serviços em segundo plano . . . 93
3.2.13 Abstração dos recursos do hardware . . . 94
3.2.14 Resumo das principais funções . . . 94
3.2.15 Atividade prática . . . 96
3.3 Processos . . . 96
3.3.1 O que é um processo . . . 96
3.3.2 Programa versus processo . . . 97
3.3.3 Processos de usuário . . . 97
3.3.4 Processos do sistema operacional . . . 100
3.3.5 Estados de um processo . . . 102

3.4 Divisão de tempo do processador . . . 106
3.4.1 A ilusão de simultaneidade . . . 107
3.4.2 Tempo de CPU . . . 107
3.4.3 Escalonamento de processos . . . 107
3.4.4 Quantum de tempo . . . 108
3.4.5 Troca de contexto . . . 108
3.4.6 Processos interativos e processos em lote . . . 109
3.4.7 Prioridades de processos . . . 110
3.4.8 Concorrência e paralelismo . . . 111
3.4.9 Processos limitados por CPU e por entrada/saída . . . 111
3.4.10 Divisão de tempo em sistemas com múltiplos núcleos . . . 112
3.4.11 Threads e divisão de tempo . . . 113
3.4.12 Responsividade do sistema . . . 113
3.4.13 Observando o uso da CPU . . . 113
3.4.14 Exemplo prático com Python . . . 114
3.4.15 Exemplo prático: processo limitado por entrada . . . 115
3.4.16 Atividade prática . . . 116
3.5 Arquivos e diretórios . . . 116
3.5.1 O que é um arquivo? . . . 117
3.5.2 Nome e extensão de arquivos . . . 117
3.5.3 Tipo de arquivo e formato . . . 119
3.5.4 Tamanho de arquivos . . . 119
3.5.5 O que é um diretório? . . . 120
3.5.6 Organização hierárquica . . . 120
3.5.7 Caminhos absolutos e relativos . . . 121
3.5.8 Recursão de diretórios . . . 122
3.5.9 Arquivos ocultos . . . 123
3.5.10 Permissões de arquivos e diretórios . . . 124
3.5.11 Permissão de execução . . . 125
3.5.12 Arquivos em projetos de software . . . 126
3.5.13 Arquivos temporários e arquivos de configuração . . . 126
3.5.14 Operações comuns com arquivos e diretórios . . . 127
3.5.15 Exemplo prático: criando uma estrutura de projeto . . . 128
3.5.16 Exemplo prático: caminhos relativos . . . 129
3.5.17 Relação com o desenvolvimento de software . . . 129
3.6 Comandos úteis para observar o sistema . . . 130
3.6.1 Linux . . . 130
3.6.2 Windows . . . 130

4 Máquinas Virtuais e Contêineres **133**
4.1 O que é virtualização . . . 133
4.2 Hipervisores . . . 133
4.2.1 O que o hipervisor faz? . . . 134
4.2.2 Sistema hospedeiro e sistema convidado . . . 134
4.2.3 Isolamento entre máquinas virtuais . . . 135
4.2.4 Virtualização de recursos . . . 135
4.2.5 Tipos de hipervisores . . . 136
4.2.6 Hipervisor tipo 1 . . . 136
4.2.7 Hipervisor tipo 2 . . . 138
4.2.8 Comparação entre hipervisor tipo 1 e tipo 2 . . . 141
4.3 Máquinas virtuais . . . 141
4.3.1 Conceito . . . 142
4.3.2 Vantagens . . . 143
4.3.3 Limitações . . . 145
4.3.4 Exemplos de uso . . . 147
4.4 Principais ferramentas de virtualização . . . 149
4.5 Contêineres . . . 150
4.5.1 O que são contêineres . . . 150
4.5.2 Diferenças entre máquinas virtuais e contêineres . . . 154
4.6 Docker . . . 154
4.6.1 Por que usar Docker? . . . 155
4.6.2 Arquitetura básica do Docker . . . 155
4.6.3 Imagem . . . 156
4.6.4 Contêiner . . . 159
4.6.5 Dockerfile . . . 162
4.6.6 Docker Compose . . . 167
4.6.7 Comandos Docker e Docker Compose . . . 171
4.6.8 Boas práticas iniciais . . . 171
4.7 Comandos básicos de Docker . . . 172

5 Aplicações para Desenvolvimento de Software **174**
5.1 O computador como ambiente de desenvolvimento . . . 174
5.1.1 Sistema operacional . . . 175
5.1.2 Editor de código . . . 175
5.1.3 Terminal . . . 176
5.1.4 Interpretadores, compiladores e runtimes . . . 177
5.1.5 Gerenciadores de pacotes . . . 178
5.2 Interpretadores e compiladores . . . 178

5.2.1 Linguagens compiladas . . . 179
5.2.2 Linguagens interpretadas . . . 180
5.2.3 Máquinas virtuais de linguagem . . . 180
5.3 Ambientes locais de desenvolvimento . . . 181
5.3.1 Instalação de linguagens . . . 182
5.3.2 Variáveis de ambiente . . . 182
5.3.3 PATH . . . 183
5.3.4 Ambientes virtuais . . . 184
5.4 Controle de versão com Git . . . 185
5.4.1 Repositórios . . . 185
5.4.2 Commits . . . 185
5.4.3 Branches . . . 186
5.4.4 Repositórios remotos . . . 186
5.5 Servidores locais . . . 187
5.5.1 Servidor HTTP simples . . . 187
5.6 Da máquina local para a produção . . . 188
5.6.1 Desenvolvimento local . . . 188
5.6.2 Ambiente de testes . . . 188
5.6.3 Homologação . . . 189
5.6.4 Produção . . . 189
5.6.5 Deploy . . . 189
5.7 Relação com DevOps . . . 190
5.7.1 Automação . . . 190
5.7.2 Integração contínua . . . 191
5.7.3 Entrega contínua . . . 191
5.7.4 Monitoramento . . . 192
5.7.5 Atividade prática . . . 193
5.7.6 Perguntas para reflexão . . . 195

A Lista de Comandos Úteis **196**
A.1 Comandos de arquivos e diretórios . . . 196
A.2 Comandos de processos . . . 196
A.3 Comandos de rede . . . 196
A.4 Comandos de Git . . . 197
A.5 Comandos de Docker . . . 197

B Glossário **198**

Capítulo 1

Introdução

Objetivos de aprendizagem

Ao final deste capítulo, você deverá ser capaz de:

- compreender o papel da disciplina de Sistemas Computacionais;
- diferenciar hardware, software, sistema operacional e aplicações;
- identificar a relação entre sistemas computacionais e desenvolvimento de software;
- reconhecer a importância do terminal, dos arquivos, dos processos e das redes na rotina profissional.

1.1 O que são sistemas computacionais

Um sistema computacional pode ser entendido como o conjunto formado por hardware, software, dados, usuários e procedimentos que possibilitam o processamento de informações.

1.2 Por que estudar sistemas computacionais em ADS?

O profissional de Análise e Desenvolvimento de Sistemas não utiliza o computador apenas como usuário final. Ele precisa compreender como os programas são executados, como os dados são armazenados, como os processos utilizam memória e processador, como os sistemas operacionais organizam recursos e como aplicações se comunicam em rede.

1.3 Breve história dos computadores

A história dos computadores não começa com os notebooks, celulares ou servidores modernos. Ela é resultado de uma longa trajetória humana de tentativas de automatizar cálculos, organizar informações, controlar máquinas e resolver problemas por meio de instruções precisas.

Antes mesmo da existência dos computadores eletrônicos, diferentes civilizações já utilizavam instrumentos de cálculo, como o ábaco. Mais do que uma simples ferramenta aritmética, o ábaco representa uma ideia fundamental para a computação: a possibilidade de representar quantidades por meio de símbolos e posições.

1.3.1 Das máquinas imaginadas aos primeiros autômatos

Muito antes da computação moderna, a humanidade já imaginava máquinas capazes de agir segundo regras. Um exemplo simbólico é Talos, o gigante de bronze da mitologia grega, descrito como um guardião mecânico da ilha de Creta. Embora Talos pertença ao campo do mito, ele ajuda a introduzir uma ideia importante: uma máquina pode ser pensada como algo que segue regras predefinidas diante de determinadas situações.



Figura 1.1: Representação da morte de Talos em uma ânfora apúlia do século IV a.C.

Outro personagem importante nessa história é Herão de Alexandria, que viveu no século I d.C. e projetou dispositivos mecânicos baseados em ar, água, vapor, pesos e contrapesos. Entre suas invenções mais conhecidas está a aeolípila, um dispositivo que girava com a força do vapor. Embora não fosse um computador, esse tipo de invenção revela uma preocupação antiga com automação, controle e conversão de energia em movimento.

A aeolípila é frequentemente lembrada como uma das primeiras demonstrações de conversão de vapor em movimento. Ainda que não tenha sido utilizada como motor industrial, ela mostra que ideias relacionadas à automação e ao controle mecânico já estavam presentes na Antiguidade.

1.3.2 Máquinas de calcular

Com o desenvolvimento da matemática, do comércio, da navegação e da administração pública, cresceu também a necessidade de realizar cálculos com mais rapidez e precisão. No século XVII, Blaise Pascal construiu a Pascaline, uma calculadora mecânica criada para realizar operações de adição e subtração. A máquina foi desenvolvida entre 1642 e 1644 e é considerada uma das primeiras calculadoras mecânicas produzidas para uso prático.



Figura 1.2: A Pascaline, calculadora mecânica associada a Blaise Pascal.

A Pascaline ainda estava distante dos computadores atuais, mas já trazia um princípio essencial: uma máquina poderia manipular números de maneira automática, reduzindo o esforço humano em tarefas repetitivas.

1.3.3 Babbage, Ada Lovelace e a ideia de computador programável

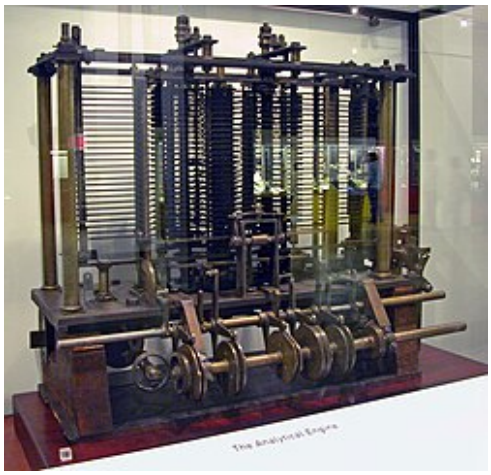


Figura 1.3: Modelo ou parte da Máquina Analítica de Charles Babbage.

No século XIX, Charles Babbage propôs máquinas muito mais ambiciosas. Sua Máquina Diferencial tinha como objetivo calcular tabelas matemáticas. Posteriormente, Babbage concebeu a Máquina Analítica, geralmente apontada como uma das primeiras ideias de computador de uso geral. A Máquina Analítica não foi concluída em sua época, mas seu projeto antecipava elementos presentes nos computadores modernos, como unidade de processamento, memória, entrada e saída de dados.

Ada Lovelace, ao estudar a Máquina Analítica, percebeu que aquele tipo de máquina poderia ir além do cálculo numérico. Suas anotações sobre o projeto de Babbage incluíam uma sequência de instruções para cálculo dos números de Bernoulli, frequentemente lembrada como um dos primeiros programas de computador da história.

A contribuição de Babbage e Lovelace é especialmente importante para a disciplina de Sistemas Computacionais porque mostra que o computador não é apenas uma calculadora mais rápida. Ele é uma máquina programável: recebe dados, segue instruções, armazena informações intermediárias e produz resultados.

1.3.4 Alan Turing e a base teórica da computação

No século XX, Alan Turing propôs um modelo teórico conhecido como Máquina de Turing. Esse modelo não era uma máquina física específica, mas uma forma abstrata de descrever o que significa computar. A ideia de Turing ajudou a estabelecer os fundamentos da ciência da computação e influenciou profundamente o desenvolvimento dos computadores modernos.

A partir desse momento, tornou-se possível discutir de maneira mais precisa quais problemas podem ou não ser resolvidos por algoritmos. Esse ponto é essencial para estudantes de desenvolvimento de sistemas, pois todo software é, em última instância, uma descrição formal de passos que uma máquina deve executar.

1.3.5 Computadores eletrônicos e a arquitetura de von Neumann

Durante a primeira metade do século XX, especialmente no contexto da Segunda Guerra Mundial e do período imediatamente posterior, surgiram computadores eletrônicos digitais de grande porte. Entre os exemplos históricos mais conhecidos estão o Colossus, o Harvard Mark I e o ENIAC.

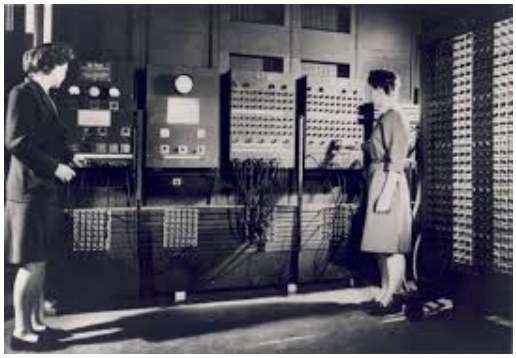


Figura 1.4: ENIAC, um dos primeiros grandes computadores eletrônicos digitais de propósito geral.

O ENIAC, apresentado em 1946, foi um dos primeiros grandes computadores eletrônicos digitais de propósito geral. Ele ocupava uma grande sala, consumia muita energia e era programado de maneira muito diferente dos computadores atuais. Ainda assim, representou um marco importante na transição das máquinas mecânicas e eletromecânicas para os computadores eletrônicos.

Outro conceito decisivo foi a chamada arquitetura de von Neumann. Nessa organização, dados e instruções ficam armazenados na memória, e o processador executa instruções em sequência, buscando, decodificando e executando comandos. Essa ideia está na base de muitos computadores modernos e ajuda a compreender a relação entre processador, memória, programas e sistema operacional.

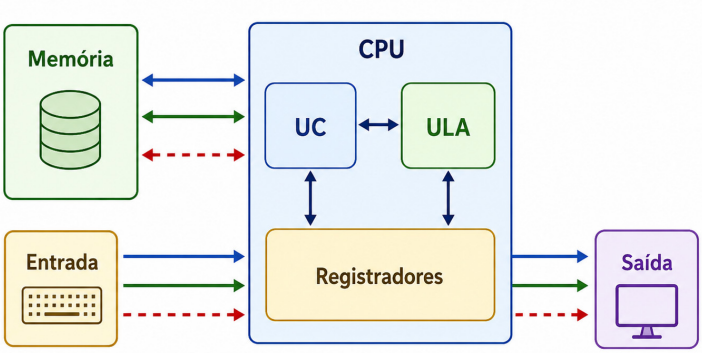


Figura 1.5: A arquitetura de von Neumann.

1.3.6 Da computação corporativa ao computador pessoal

Nas décadas seguintes, os computadores passaram por um processo de miniaturização, barateamento e popularização. A invenção do transistor, dos circuitos integrados e dos microprocessadores permitiu a criação de máquinas menores, mais confiáveis e acessíveis.

A partir das décadas de 1970 e 1980, os computadores pessoais começaram a chegar a empresas, escolas e residências. Interfaces gráficas, sistemas operacionais comerciais e ferramentas de produtividade aproximaram o computador do usuário comum.

Esse processo modificou profundamente a relação das pessoas com a tecnologia. O computador deixou de ser apenas uma máquina institucional, restrita a governos, universidades e grandes empresas, e passou a ser também uma ferramenta cotidiana de trabalho, estudo, comunicação e entretenimento.

1.3.7 Internet, Web e computação em rede

Outro marco fundamental foi o desenvolvimento das redes de computadores. A ARPANET, criada no final da década de 1960, é frequentemente lembrada como uma das precursoras da internet. Posteriormente, a consolidação dos protocolos TCP/IP permitiu a comunicação entre diferentes redes.

Na década de 1990, a World Wide Web tornou a internet mais acessível para usuários comuns, por meio de páginas, navegadores, links e servidores web. Esse momento é particularmente importante para o curso de Análise e Desenvolvimento de Sistemas, pois grande parte das aplicações atuais depende de redes, servidores, bancos de dados, APIs, navegadores e serviços em nuvem.

1.3.8 Computação móvel, nuvem e inteligência artificial

No século XXI, os computadores deixaram de estar restritos ao formato tradicional de desktop ou notebook. Smartphones, tablets, relógios inteligentes, dispositivos embarcados, sensores, servidores em nuvem e sistemas distribuídos passaram a fazer parte do cotidiano.

Ao mesmo tempo, a inteligência artificial ganhou novo impulso com o avanço do aprendizado de máquina, das redes neurais profundas, dos transformers e dos grandes modelos de linguagem. Isso mostra que a história dos computadores continua em movimento: novas arquiteturas, novos dispositivos e novos modos de interação continuam transformando a forma como desenvolvemos e utilizamos sistemas computacionais.

1.3.9 Linha do tempo resumida

Período	**Marco histórico**
Antiguidade	Uso de instrumentos de cálculo, como o ábaco, e imaginação de autômatos em mitos e projetos mecânicos.
Século I d.C.	Herão de Alexandria projeta dispositivos automáticos e mecanismos movidos por ar, água e vapor.
1642	Blaise Pascal desenvolve a Pascaline, uma calculadora mecânica.
Século XIX	Charles Babbage concebe a Máquina Diferencial e a Máquina Analítica.
1843	Ada Lovelace publica anotações sobre a Máquina Analítica, incluindo uma sequência de instruções considerada precursora da programação.
1936	Alan Turing propõe um modelo teórico de computação conhecido como Máquina de Turing.
Década de 1940	Surgem grandes computadores eletromecânicos e eletrônicos, como Colossus, Harvard Mark I e ENIAC.
1945	A arquitetura de von Neumann consolida a ideia de programa armazenado na memória.
Décadas de 1970 e 1980	Popularização dos microprocessadores e dos computadores pessoais.
Década de 1990	Expansão da Web e popularização da internet comercial.
Século XXI	Expansão da computação móvel, computação em nuvem, sistemas distribuídos e inteligência artificial.

Observação

A história dos computadores não deve ser vista apenas como uma sequência de máquinas mais potentes. Ela também é a história da evolução de ideias: representação de dados, automação, algoritmos, programação, memória, processamento, redes e interação entre humanos e máquinas.

Atividade prática

Pesquise uma máquina, pessoa ou tecnologia mencionada nesta seção e prepare uma apresentação curta, de três a cinco minutos, respondendo:

- Qual problema essa invenção ou pessoa ajudou a resolver?
- Que relação ela tem com os computadores atuais?
- Que conceito da disciplina aparece nesse marco histórico?

1.4 Componentes básicos de um computador

Um computador pode parecer, à primeira vista, apenas uma tela, um teclado e alguns programas em execução. No entanto, por trás dessa aparência simples existe um conjunto de componentes físicos trabalhando de forma coordenada. Cada componente possui uma função específica, mas nenhum deles funciona de maneira isolada: o computador só realiza tarefas porque processador, memória, armazenamento, dispositivos de entrada e saída, placa-mãe e sistema operacional atuam em conjunto.

Para você de Análise e Desenvolvimento de Sistemas, conhecer esses componentes é importante porque todo software depende de uma base física para ser executado. Um programa ocupa espaço em disco, é carregado para a memória, utiliza o processador, acessa arquivos, envia dados pela rede e interage com dispositivos de entrada e saída.

1.4.1 Processador

O processador, também chamado de CPU (*Central Processing Unit*), é o componente responsável por executar instruções. Sempre que um programa está em funcionamento, suas instruções precisam ser interpretadas e processadas pela CPU.

De forma simplificada, o processador realiza três etapas continuamente:

- busca uma instrução na memória;
- interpreta o que deve ser feito;
- executa a operação necessária.

Esse ciclo é conhecido como ciclo de busca, decodificação e execução (*fetch-decode-execute*). Ele ocorre milhões ou bilhões de vezes por segundo, dependendo da capacidade do processador.

Exemplo didático

Quando um usuário abre um navegador, o sistema operacional cria um processo para esse programa. A CPU passa a executar instruções relacionadas à abertura da janela, carregamento da interface, comunicação com a rede, renderização das páginas e resposta aos comandos do usuário.

Algumas características comuns dos processadores são:

- **frequência**: indica quantos ciclos por segundo o processador consegue realizar, geralmente medida em GHz;
- **núcleos**: representam unidades internas capazes de executar tarefas. Processadores com vários núcleos podem lidar melhor com múltiplas atividades simultâneas;
- **cache**: pequena memória interna muito rápida, usada para armazenar dados e instruções acessados com frequência;
- **arquitetura**: define o conjunto de instruções e a forma como o processador organiza suas operações internas.

Observação

Um processador mais rápido nem sempre significa um computador melhor em todas as situações. O desempenho depende também da quantidade de memória, da velocidade do armazenamento, do sistema operacional, dos programas em execução e até da temperatura do equipamento.

1.4.2 Memória principal

A memória principal, geralmente chamada de memória RAM (*Random Access Memory*), é o local onde ficam temporariamente os dados e instruções dos programas em execução.

Quando um programa é aberto, ele não é executado diretamente do disco. Primeiro, partes do programa e dos dados necessários são carregadas para a memória RAM. A partir daí, o processador consegue acessar essas informações com muito mais rapidez.

Exemplo didático

Ao abrir um editor de texto, o programa é carregado do armazenamento para a memória RAM. O texto que está sendo editado também permanece na memória enquanto o usuário trabalha. Se o computador for desligado sem salvar o arquivo, as alterações podem ser perdidas, pois a RAM é uma memória volátil.

A memória RAM é chamada de volátil porque perde seu conteúdo quando o computador é desligado. Por isso, os arquivos precisam ser salvos em dispositivos de armazenamento permanente, como SSDs, HDs ou unidades externas.

1.4.3 Dispositivos de armazenamento

Os dispositivos de armazenamento são responsáveis por guardar dados de forma persistente. Isso significa que os dados continuam armazenados mesmo quando o computador é desligado.

Entre os principais dispositivos de armazenamento, podemos citar:

- **HD** (*Hard Disk*): utiliza discos magnéticos e partes mecânicas. Costuma oferecer grande capacidade por menor custo, mas é mais lento que um SSD;
- **SSD** (*Solid State Drive*): utiliza memória flash, sem partes mecânicas. É mais rápido, silencioso e resistente a impactos que um HD tradicional;
- **pendrive**: dispositivo portátil de armazenamento, geralmente conectado por USB;
- **cartões de memória**: usados em câmeras, celulares, tablets e outros dispositivos;
- **armazenamento em nuvem**: dados guardados em servidores acessados pela internet.

Atividade prática

Compare o tempo de abertura de um programa em dois computadores diferentes, se possível um com HD e outro com SSD. Observe também o tempo necessário para copiar um arquivo grande. Depois, reflita: por que o tipo de armazenamento influencia tanto a sensação de velocidade do computador?

Do ponto de vista do sistema operacional, os dispositivos de armazenamento são organizados em arquivos e diretórios. Essa organização permite que programas e usuários encontrem, salvem, modifiquem e removam informações.

1.4.4 Placa-mãe

A placa-mãe é a principal placa de circuito do computador. Ela conecta e permite a comunicação entre processador, memória RAM, armazenamento, placa de vídeo, dispositivos USB, interfaces de rede e demais componentes.

Podemos pensar na placa-mãe como uma grande estrutura de integração. Ela não executa sozinha os programas do usuário, mas fornece os caminhos físicos para que os componentes troquem informações.

Entre os elementos encontrados em uma placa-mãe, estão:

- soquete do processador;
- slots de memória RAM;
- conectores para SSDs e HDs;
- portas USB;
- interface de rede;
- chipset;
- barramentos de comunicação;
- conectores de energia.

> **Observação**
>
> O conceito de barramento é importante em sistemas computacionais. Um barramento é um caminho de comunicação usado para transportar dados, endereços ou sinais de controle entre componentes.

1.4.5 Dispositivos de entrada

Dispositivos de entrada são aqueles que permitem ao usuário ou a outro sistema enviar dados para o computador. Eles transformam ações, sinais ou informações externas em dados que podem ser processados.

Exemplos de dispositivos de entrada:

- teclado;
- mouse;
- touchpad;
- tela sensível ao toque;
- microfone;
- scanner;
- câmera;
- leitor de código de barras;
- sensores.

Exemplo didático

Quando um usuário digita uma senha, o teclado envia sinais ao computador. Esses sinais são interpretados pelo sistema operacional e entregues ao programa que está solicitando a entrada, como um navegador ou aplicativo.

Em sistemas modernos, dispositivos de entrada não se limitam ao uso humano direto. Sensores de temperatura, GPS, leitores biométricos, câmeras de segurança e dispositivos de internet das coisas também enviam dados para processamento.

1.4.6 Dispositivos de saída

Dispositivos de saída apresentam ao usuário ou a outro sistema o resultado do processamento realizado pelo computador.

Exemplos de dispositivos de saída:

- monitor;
- impressora;
- caixas de som;
- fones de ouvido;
- projetor;
- painel de LED;
- atuadores em sistemas embarcados.

Exemplo didático

Ao assistir a um vídeo, o computador lê dados armazenados localmente ou recebidos pela internet, processa essas informações e envia sinais de imagem ao monitor e sinais de áudio às caixas de som ou fones de ouvido.

Alguns dispositivos podem atuar tanto como entrada quanto como saída. Uma tela sensível ao toque, por exemplo, exibe informações ao usuário e também recebe comandos por toque.

1.4.7 Placa de vídeo

A placa de vídeo, ou GPU (*Graphics Processing Unit*), é responsável pelo processamento gráfico. Em computadores simples, essa função pode estar integrada ao próprio processador. Em computadores mais potentes, pode haver uma placa de vídeo dedicada.

A GPU é especialmente importante em tarefas que envolvem muitos cálculos paralelos, como:

- jogos digitais;
- edição de vídeo;
- modelagem 3D;
- simulações;
- aprendizado de máquina;
- processamento de imagens.

Observação

Embora a GPU tenha se popularizado pelo uso em gráficos e jogos, ela se tornou também uma peça importante em áreas como inteligência artificial, ciência de dados e computação científica.

1.4.8 Fonte de alimentação

A fonte de alimentação converte a energia elétrica recebida da tomada em níveis adequados para os componentes internos do computador. Cada peça precisa receber energia com tensão e estabilidade apropriadas.

Uma fonte inadequada pode causar instabilidade, desligamentos inesperados ou até danificar componentes. Por isso, em computadores de mesa, a escolha da fonte deve considerar o consumo do processador, da placa de vídeo, dos discos e dos demais periféricos.

1.4.9 Gabinete e refrigeração

O gabinete organiza fisicamente os componentes e ajuda na circulação de ar. A refrigeração é necessária porque processador, placa de vídeo, fonte e outros componentes geram calor durante o funcionamento.

Sistemas de refrigeração podem usar:

- dissipadores de calor;
- ventoinhas;
- pasta térmica;
- refrigeração líquida em computadores de alto desempenho.

Observação

Quando a temperatura interna fica muito alta, o computador pode reduzir automaticamente o desempenho para evitar danos. Esse fenômeno é conhecido como *thermal throttling*.

1.4.10 Interfaces de rede

As interfaces de rede permitem que o computador se comunique com outros dispositivos. Essa comunicação pode ocorrer por cabo, como em conexões Ethernet, ou sem fio, como em redes Wi-Fi e Bluetooth.

Por meio da rede, o computador pode:

- acessar páginas web;
- enviar e receber arquivos;
- conectar-se a servidores;
- acessar bancos de dados remotos;
- utilizar serviços em nuvem;
- realizar chamadas de vídeo;
- executar aplicações distribuídas.

Exemplo didático

Quando uma aplicação web é acessada pelo navegador, o computador do usuário envia requisições pela rede até um servidor. O servidor processa essas requisições e devolve respostas, que são exibidas no navegador.

1.4.11 Como os componentes trabalham juntos

Para compreender a interação entre os componentes, podemos analisar uma situação simples: abrir e usar um navegador.

1. O usuário clica no ícone do navegador usando um dispositivo de entrada, como mouse ou touchpad.
2. O sistema operacional localiza o programa no dispositivo de armazenamento.
3. As instruções e dados necessários são carregados para a memória RAM.
4. O processador executa as instruções do programa.

5. A placa de vídeo ajuda a exibir a interface gráfica na tela.
6. A interface de rede permite acessar páginas, servidores e serviços na internet.
7. O monitor apresenta a saída visual ao usuário.
8. Novas ações do usuário, como digitar um endereço ou clicar em um link, reiniciam esse ciclo de entrada, processamento e saída.

Exemplo didático

A execução de um programa pode ser entendida como uma conversa contínua entre componentes. O armazenamento guarda o programa, a memória mantém os dados temporários, a CPU executa instruções, os dispositivos de entrada recebem comandos, os dispositivos de saída apresentam resultados e a rede permite comunicação com outros sistemas.

1.4.12 Relação com o desenvolvimento de software

Ao desenvolver software, muitas decisões dependem da compreensão dos componentes do computador. Um programa mal projetado pode consumir muita memória, usar excessivamente o processador, gerar muitos acessos ao disco, depender de rede instável ou não lidar corretamente com diferentes dispositivos.

Por isso, o desenvolvedor precisa considerar questões como:

- quanto de memória a aplicação utiliza;
- quanto processamento ela exige;
- se os dados são lidos do disco ou da rede;
- se a aplicação precisa funcionar offline;
- como arquivos são salvos e organizados;
- como o programa reage a falhas de rede;
- se o sistema será executado em computador local, servidor, máquina virtual, contêiner ou nuvem.

Atividade prática

Escolha um programa que você utiliza com frequência, como navegador, editor de texto, aplicativo de mensagens ou ambiente de desenvolvimento. Descreva quais componentes do computador são utilizados enquanto esse programa está em execução. Procure identificar entrada, processamento, memória, armazenamento, rede e saída.

Exemplo didático

Quando um usuário abre um navegador, várias camadas entram em ação: o hardware fornece processamento, memória e rede; o sistema operacional gerencia os recursos; o navegador executa como processo; e os dados são transmitidos pela rede usando protocolos como TCP/IP.

1.5 O papel do terminal

O terminal é uma interface textual que permite ao usuário interagir com o sistema operacional por meio de comandos. Em vez de clicar em menus, janelas e botões, o usuário escreve instruções que são interpretadas por um programa chamado *shell*.

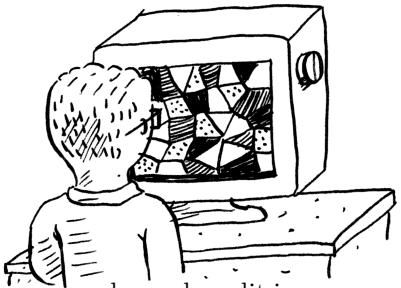
Embora muitas pessoas associem o terminal a uma forma antiga de usar o computador, ele continua sendo uma ferramenta essencial na área de tecnologia. Desenvolvedores, administradores de sistemas, profissionais de redes, cientistas de dados e especialistas em segurança utilizam o terminal diariamente para executar tarefas com rapidez, precisão e automação.

1.5.1 Interface gráfica versus interface de linha de comando

Os sistemas operacionais modernos geralmente oferecem dois tipos principais de interação: a interface gráfica e a interface de linha de comando.

A interface gráfica, conhecida como GUI (*Graphical User Interface*), permite interagir com o computador por meio de elementos visuais, como janelas, ícones, menus e botões. Ela é intuitiva e facilita o uso cotidiano do computador.

A interface de linha de comando, conhecida como CLI (*Command Line Interface*), permite interagir com o sistema digitando comandos. Ela pode parecer menos amigável no início, mas oferece grande poder de controle, repetição e automação.



drawn by adit.io

Observação

A interface gráfica e o terminal não são concorrentes. Na prática, profissionais de tecnologia usam as duas formas de interação. A escolha depende da tarefa, do contexto e do nível de controle necessário.

Aspecto	Interface gráfica	Terminal
Forma de interação	Cliques, menus e janelas	Comandos digitados
Facilidade inicial	Geralmente mais intuitiva	Exige aprendizagem dos comandos
Velocidade em tarefas repetitivas	Pode ser mais lenta	Pode ser muito rápida
Automação	Mais limitada	Muito forte, por meio de scripts
Precisão	Depende da navegação visual	Comandos podem ser repetidos exatamente
Uso profissional	Muito comum	Essencial em desenvolvimento e administração de sistemas

1.5.2 Shell, terminal e prompt

É comum usar a palavra terminal de forma genérica, mas existem alguns conceitos que merecem distinção.

O **terminal** é o programa ou ambiente em que os comandos são digitados. O **shell** é o interpretador que recebe os comandos, processa o que foi solicitado e chama os programas correspondentes. O **prompt** é o sinal exibido na tela indicando que o sistema está pronto para receber um novo comando.

Em sistemas Linux, alguns shells comuns são:

- `bash`;
- `zsh`;
- `fish`.

No Windows, são comuns:

- Prompt de Comando;
- PowerShell;
- Windows Terminal;
- ambientes Linux via WSL.

Exemplo didático

Um prompt simples em Linux pode aparecer assim:

```
usuario@computador:~$
```

Nesse exemplo, `usuario` indica o nome do usuário, `computador` indica o nome da máquina, `~` representa o diretório pessoal do usuário e o símbolo `$` indica que o terminal está aguardando um comando.

1.5.3 Por que o terminal é importante para desenvolvedores

O terminal é importante porque muitas ferramentas de desenvolvimento são criadas, configuradas ou executadas por linha de comando. Mesmo quando há interfaces gráficas disponíveis, a linha de comando costuma oferecer mais controle e clareza sobre o que está acontecendo.

Entre as tarefas comuns realizadas pelo terminal, estão:

- navegar por diretórios;
- criar, mover, copiar e remover arquivos;
- instalar pacotes e bibliotecas;
- executar programas e scripts;
- iniciar servidores locais;
- testar APIs;
- usar Git;
- acessar máquinas remotas;
- consultar processos em execução;
- verificar uso de disco, memória e rede;
- automatizar tarefas repetitivas.

Exemplo didático

Em um projeto Python, um desenvolvedor pode usar o terminal para criar um ambiente virtual, instalar dependências e executar uma aplicação:

```
python3 -m venv venv
source venv/bin/activate
pip install flask
python app.py
```

1.5.4 Comandos como pequenas ferramentas

Uma forma útil de entender o terminal é imaginar cada comando como uma pequena ferramenta. Alguns comandos mostram informações, outros criam arquivos, outros filtram textos, outros executam programas.

Em sistemas do tipo Unix, como Linux e macOS, existe uma filosofia importante: criar programas pequenos, especializados e capazes de trabalhar juntos. Por isso, muitos comandos podem ser combinados para realizar tarefas mais complexas.

Exemplo didático

O comando abaixo lista arquivos de um diretório:

```
ls
```

O comando abaixo mostra o diretório atual:

```
pwd
```

O comando abaixo cria uma nova pasta:

```
mkdir projetos
```

1.5.5 Navegação em diretórios

Uma das primeiras habilidades no terminal é saber se localizar no sistema de arquivos. Para isso, usamos comandos que mostram onde estamos e permitem mudar de diretório.

```
pwd
ls
cd Documentos
cd ..
cd ~
```

Esses comandos permitem responder a perguntas simples, mas fundamentais:

- em qual diretório estou?
- quais arquivos existem aqui?
- como entrar em outra pasta?
- como voltar ao diretório anterior?
- como retornar à minha pasta pessoal?

Observação

No terminal, é importante prestar atenção ao diretório atual. Muitos erros acontecem porque o usuário executa um comando no lugar errado, criando, apagando ou modificando arquivos em uma pasta diferente da pretendida.

1.5.6 Manipulação de arquivos e diretórios

Além de navegar, o terminal permite criar, copiar, mover, renomear e remover arquivos e diretórios.

```
mkdir aula01
cd aula01
touch anotacoes.txt
cp anotacoes.txt copia-anotacoes.txt
mv copia-anotacoes.txt resumo.txt
rm resumo.txt
```

Esses comandos são simples, mas mostram uma ideia importante: o terminal permite manipular diretamente a estrutura de arquivos do sistema.

Observação

Comandos de remoção devem ser usados com cuidado. Em muitos terminais, o comando `rm` remove arquivos sem enviá-los para a lixeira.

1.5.7 Execução de programas

O terminal também permite executar programas. Em desenvolvimento de software, isso é especialmente importante, pois muitas aplicações são testadas inicialmente por linha de comando.

```
python programa.py
node servidor.js
java Main
./script.sh
```

Quando um programa é executado, o sistema operacional cria um processo. Esse processo passa a consumir recursos como CPU, memória, arquivos e, em alguns casos, rede.

Exemplo didático

Ao executar:

```
python app.py
```

o sistema operacional inicia um processo do interpretador Python, que carrega e executa as instruções presentes no arquivo `app.py`.

1.5.8 Terminal e processos

O terminal permite observar e controlar processos em execução. Isso é útil quando um programa trava, quando um servidor local precisa ser interrompido ou quando queremos entender quais programas estão consumindo mais recursos.

Alguns comandos úteis em Linux são:

```
ps aux
top
htop
kill PID
```

No Windows, comandos semelhantes podem ser executados no Prompt de Comando ou PowerShell:

```
tasklist
taskkill /PID numero_do_processo /F
```

Atividade prática

Abra um terminal e execute um programa simples, como um servidor local Python:

```
python3 -m http.server 8000
```

Depois, em outro terminal, verifique os processos em execução. Identifique o processo relacionado ao servidor e encerre-o de forma controlada.

1.5.9 Terminal e redes

Muitas tarefas de rede também são realizadas pelo terminal. Isso inclui testar conexão, verificar endereços IP, consultar servidores DNS, realizar requisições HTTP e acessar máquinas remotamente.

```
ping exemplo.com
ip addr
nslookup exemplo.com
curl https://exemplo.com
ssh usuario@endereco_ip
```

Esses comandos são especialmente úteis para desenvolvedores web, pois permitem verificar se um servidor está respondendo, se uma API está funcionando ou se há problemas de conexão.

Exemplo didático

O comando `curl` pode ser usado para testar uma API local:

```
curl http://localhost:5000
```

Se a aplicação estiver em execução, o terminal exibirá a resposta retornada pelo servidor.

1.5.10 Automação com scripts

Uma das maiores vantagens do terminal é a possibilidade de automatizar tarefas. Em vez de repetir vários comandos manualmente, podemos criar um script contendo uma sequência de instruções.

Exemplo didático

Um script simples para criar uma estrutura de diretórios:

```
#!/bin/bash

mkdir -p projeto/src
mkdir -p projeto/docs
mkdir -p projeto/tests
touch projeto/README.md
```

Depois de salvo, o script pode receber permissão de execução e ser executado:

```
chmod +x criar-projeto.sh
./criar-projeto.sh
```

Esse tipo de automação é muito usado em instalação de ambientes, execução de testes, configuração de servidores, backups e processos de deploy.

1.5.11 Terminal, Git e desenvolvimento colaborativo

O Git é uma das ferramentas mais importantes no desenvolvimento de software. Embora existam interfaces gráficas para Git, muitos comandos são frequentemente executados pelo terminal.

```
git init
git status
git add .
git commit -m "primeiro commit"
git log
git push origin main
git pull
```

Ao usar Git pelo terminal, você compreende melhor o que está acontecendo no controle de versão: quais arquivos foram modificados, quais mudanças serão registradas e como o projeto se comunica com um repositório remoto.

1.5.12 Cuidados ao usar o terminal

O terminal oferece poder e rapidez, mas também exige atenção. Alguns comandos podem modificar o sistema, apagar arquivos ou alterar permissões.

Boas práticas incluem:

- ler o comando antes de executá-lo;
- evitar copiar comandos da internet sem entender sua função;
- conferir o diretório atual antes de remover arquivos;
- usar comandos administrativos apenas quando necessário;
- fazer backup de arquivos importantes;
- prestar atenção a mensagens de erro.

Observação

Um erro no terminal pode ser uma oportunidade de aprendizagem. Mensagens de erro geralmente indicam o que deu errado: arquivo inexistente, permissão negada, comando não encontrado, porta ocupada, dependência ausente ou caminho incorreto.

1.5.13 O terminal como ferramenta de pensamento

Mais do que uma lista de comandos, o terminal ajuda você a pensar computacionalmente. Ao usá-lo, é necessário compreender caminhos, entradas, saídas, permissões, processos, argumentos, arquivos e estados do sistema.

Essa forma de interação aproxima você do funcionamento interno do computador. Enquanto a interface gráfica muitas vezes esconde os detalhes, o terminal revela etapas importantes da execução de programas e da organização do sistema operacional.

Atividade prática

Crie uma pasta chamada `sistemas-computacionais`. Dentro dela, crie três subpastas: `aulas`, `exercicios` e `projetos`. Em seguida, crie um arquivo chamado `anotacoes.txt` dentro da pasta `aulas`. Faça tudo pelo terminal.

Depois, escreva no caderno ou em um arquivo de texto quais comandos você usou e o que cada um deles fez.

Exercícios

1. A história da computação costuma ser associada aos computadores eletrônicos do século XX, mas muitos conceitos surgiram antes disso. Que ideias presentes em instrumentos como o ábaco, autômatos antigos e calculadoras mecânicas ainda aparecem nos computadores atuais?

2. A Máquina Analítica de Charles Babbage nunca foi concluída como um computador funcional em sua época. Mesmo assim, ela é considerada um marco importante. Como uma máquina que não chegou a ser plenamente construída pode ter influenciado tanto a história da computação?

3. Ada Lovelace percebeu que uma máquina poderia manipular não apenas números, mas também símbolos e instruções. Por que essa percepção é tão importante para entendermos a programação como algo maior do que simples cálculo matemático?

4. A arquitetura de von Neumann organiza o computador em memória, unidade de processamento, entrada e saída. Como essa organização ajuda a explicar o funcionamento de um programa quando ele é aberto, carregado na memória e executado pelo processador?

5. A internet e a Web transformaram o desenvolvimento de sistemas. Como a lógica de criar programas mudou quando as aplicações deixaram de funcionar apenas em uma máquina local e passaram a depender de servidores, navegadores, redes e bancos de dados remotos?

6. Um desenvolvedor poderia escrever código sem conhecer profundamente sistemas operacionais. Porém, esse desconhecimento pode gerar limitações. Em quais situações práticas o conhecimento sobre processos, arquivos, permissões, memória ou rede pode ajudar a resolver problemas de desenvolvimento?

7. Ao longo da história da computação, muitas invenções surgiram para resolver problemas específicos, como calcular tabelas, automatizar tarefas ou comunicar computadores. Como necessidades sociais, científicas, militares, comerciais ou educacionais influenciaram o desenvolvimento dos computadores?

8. A computação moderna envolve máquinas físicas, sistemas operacionais, redes, servidores, aplicações e usuários. Por que é importante que um estudante de Análise e Desenvolvimento de Sistemas consiga enxergar essas camadas em conjunto, e não apenas o código-fonte de uma aplicação?

Conteúdos extras recomendados

Filmes e documentários

- *Pirates of Silicon Valley* — sobre os primeiros anos da Apple e da Microsoft.
- *The Internet's Own Boy* — documentário sobre Aaron Swartz e cultura digital.
- *The Imitation Game* — útil para introduzir Alan Turing e a relação entre guerra, criptografia e computação.
- *Pirates of Silicon Valley* — aborda a popularização da computação pessoal e a disputa entre empresas de tecnologia.
- *Halt and Catch Fire* — série sobre computadores pessoais, redes e cultura tecnológica.
- *Revolution OS* — documentário sobre software livre, GNU/Linux e cultura hacker.
- *The Internet's Own Boy* — documentário útil para discutir internet, cultura digital e acesso ao conhecimento.

Capítulo 2

Sistemas Numéricos

Objetivos de aprendizagem

Ao final deste capítulo, você deverá ser capaz de:

- compreender os sistemas decimal, binário, octal e hexadecimal;
- converter números entre diferentes bases;
- relacionar números binários com armazenamento e processamento de dados;
- compreender unidades de medida de informação, como bit, byte, kilobyte, megabyte e gigabyte.

2.1 Por que computadores usam números binários?

Computadores digitais representam informações utilizando dois estados básicos, normalmente associados aos valores 0 e 1. Essa representação está relacionada à forma como os circuitos eletrônicos distinguem estados de tensão elétrica.

2.2 Sistema decimal

O sistema decimal é o sistema de numeração mais utilizado em nosso cotidiano. Ele recebe esse nome porque é baseado em dez símbolos numéricos, chamados algarismos:

$$0,\ 1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7,\ 8,\ 9$$

Por utilizar dez símbolos, dizemos que o sistema decimal é um sistema de base 10. Isso significa que cada posição de um número representa uma potência de 10.

Por exemplo, o número 347 pode ser interpretado da seguinte forma:

$$347 = 3 \times 10^2 + 4 \times 10^1 + 7 \times 10^0$$

Como:

$$10^2 = 100, \quad 10^1 = 10, \quad 10^0 = 1$$

temos:

$$347 = 3 \times 100 + 4 \times 10 + 7 \times 1$$

$$347 = 300 + 40 + 7$$

Esse tipo de organização é chamado de sistema posicional. O valor de cada algarismo depende da posição que ele ocupa no número.

Exemplo didático

No número 555, o algarismo 5 aparece três vezes, mas possui valores diferentes em cada posição:

$$555 = 5 \times 10^2 + 5 \times 10^1 + 5 \times 10^0$$

$$555 = 500 + 50 + 5$$

Portanto, o primeiro 5 representa cinco centenas, o segundo representa cinco dezenas e o terceiro representa cinco unidades.

2.2.1 Valor posicional

No sistema decimal, as posições são organizadas da direita para a esquerda. A posição mais à direita representa as unidades. A próxima representa as dezenas. Depois vêm as centenas, milhares e assim por diante.

Potência	10^4	10^3	10^2	10^1	10^0
Valor	10000	1000	100	10	1
Nome	dezena de milhar	milhar	centena	dezena	unidade

Observação

Todo número elevado a zero é igual a 1, desde que esse número seja diferente de zero. Por isso, a posição das unidades é representada por 10^0.

2.2.2 O decimal e a computação

Embora os computadores trabalhem internamente com sinais binários, o sistema decimal continua sendo importante na computação. Ele é usado na interface com usuários, em relatórios, sistemas financeiros, formulários, planilhas, bancos de dados e aplicações comerciais.

Quando um usuário digita o número 2026 em um sistema, ele está usando a representação decimal. Internamente, porém, esse valor precisa ser convertido para uma representação que o computador consiga manipular em nível eletrônico.

Exemplo didático

Quando um sistema bancário mostra o saldo de R$ 1.250,00, a informação é apresentada ao usuário em decimal. No entanto, internamente, os dados são codificados em bits, armazenados em memória e processados por circuitos digitais.

2.3 Sistema binário

O sistema binário é o sistema de numeração de base 2. Ele utiliza apenas dois símbolos:

$$0,\ 1$$

Esse sistema é fundamental para a computação porque os computadores digitais trabalham com circuitos eletrônicos capazes de representar dois estados principais, como ligado e desligado, presença ou ausência de tensão, verdadeiro ou falso.

Esses dois estados são representados simbolicamente pelos valores 0 e 1. Cada dígito binário é chamado de bit, palavra derivada da expressão *binary digit*.

2.3.1 Base 2 e valor posicional

Assim como no sistema decimal cada posição representa uma potência de 10, no sistema binário cada posição representa uma potência de 2.

Por exemplo, o número binário 1011_2 pode ser interpretado assim:

$$1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

Como:

$$2^3 = 8, \quad 2^2 = 4, \quad 2^1 = 2, \quad 2^0 = 1$$

temos:

$$1011_2 = 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1$$

$$1011_2 = 8 + 0 + 2 + 1 = 11_{10}$$

Portanto:

$$1011_2 = 11_{10}$$

Observação

Para evitar confusão entre sistemas numéricos diferentes, é comum usar um índice indicando a base. Por exemplo, 1011_2 indica que o número está em base 2, enquanto 1011_{10} indica que o número está em base 10.

2.3.2 Primeiros números em binário

Decimal	Binário	Decimal	Binário
0	0	8	1000
1	1	9	1001
2	10	10	1010
3	11	11	1011
4	100	12	1100
5	101	13	1101
6	110	14	1110
7	111	15	1111

2.3.3 Bits e combinações

Um único bit pode representar apenas dois valores possíveis: 0 ou 1.
Com dois bits, temos quatro combinações possíveis:

$$00,\ 01,\ 10,\ 11$$

Com três bits, temos oito combinações possíveis:

$$000,\ 001,\ 010,\ 011,\ 100,\ 101,\ 110,\ 111$$

De modo geral, com n bits, é possível representar:

$$2^n$$

combinações diferentes.

Exemplo didático

Com 8 bits, temos:

$$2^8 = 256$$

Isso significa que um conjunto de 8 bits pode representar 256 valores diferentes. Por isso, um byte, formado por 8 bits, é uma unidade tão importante na computação.

2.3.4 O binário e os computadores

O sistema binário é a base da representação digital. Textos, imagens, sons, vídeos, programas e instruções são codificados internamente como sequências de bits.

Um caractere, por exemplo, pode ser representado por um código binário. Uma imagem pode ser representada por pixels, e cada pixel pode ser descrito por valores numéricos. Um som pode ser transformado em amostras numéricas. Em todos esses casos, a informação precisa ser convertida em uma forma que o computador consiga armazenar e processar.

Exemplo didático

A letra `A`, em algumas codificações, pode ser associada ao valor decimal 65. Esse valor, por sua vez, pode ser representado em binário como:

$$65_{10} = 01000001_2$$

Assim, mesmo um texto aparentemente simples é armazenado como uma sequência de bits.

2.4 Sistema octal

O sistema octal é um sistema de numeração de base 8. Ele utiliza oito símbolos:

$$0,\ 1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7$$

Como sua base é 8, cada posição de um número octal representa uma potência de 8.

Embora o sistema octal seja menos usado atualmente do que o hexadecimal, ele tem importância histórica e ainda aparece em alguns contextos da computação, especialmente em permissões de arquivos em sistemas Unix e Linux.

2.4.1 Base 8 e valor posicional

O número octal 157_8 pode ser interpretado da seguinte forma:

$$157_8 = 1 \times 8^2 + 5 \times 8^1 + 7 \times 8^0$$

Como:

$$8^2 = 64, \quad 8^1 = 8, \quad 8^0 = 1$$

temos:

$$157_8 = 1 \times 64 + 5 \times 8 + 7 \times 1$$

$$157_8 = 64 + 40 + 7 = 111_{10}$$

Portanto:

$$157_8 = 111_{10}$$



drawn by adit.io

2.4.2 Relação entre octal e binário

A relação entre octal e binário é simples porque 8 é uma potência de 2:

$$8 = 2^3$$

Isso significa que cada dígito octal pode ser representado por exatamente 3 bits.

Octal	Binário
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Exemplo didático

Para converter o número octal 725_8 para binário, basta substituir cada dígito octal por seu equivalente de 3 bits:

$$7_8 = 111_2$$

$$2_8 = 010_2$$

$$5_8 = 101_2$$

Logo:

$$725_8 = 111010101_2$$

2.4.3 Uso do sistema octal em permissões

Um uso muito comum do sistema octal aparece nas permissões de arquivos em sistemas Unix e Linux. Permissões de leitura, escrita e execução podem ser representadas por valores numéricos.

Permissão	Símbolo	Valor
Leitura	r	4
Escrita	w	2
Execução	x	1

A soma desses valores indica as permissões de um usuário, grupo ou outros usuários.

Exemplo didático

A permissão 7 representa:

$$7 = 4 + 2 + 1$$

Ou seja:

$$7 = r + w + x$$

Portanto, o valor 7 indica permissão de leitura, escrita e execução.
Já o comando:

```
chmod 755 script.sh
```

indica que:

- o dono do arquivo tem permissão 7: leitura, escrita e execução;
- o grupo tem permissão 5: leitura e execução;
- os demais usuários têm permissão 5: leitura e execução.

2.5 Sistema hexadecimal

O sistema hexadecimal é um sistema de numeração de base 16. Ele utiliza dez algarismos numéricos e seis letras:

$$0,\ 1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7,\ 8,\ 9,\ A,\ B,\ C,\ D,\ E,\ F$$

As letras representam os valores decimais de 10 a 15.

Hexadecimal	Decimal
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
A	10
B	11
C	12
D	13
E	14
F	15

O sistema hexadecimal é muito usado em computação porque permite representar números binários longos de maneira mais curta e legível.

2.5.1 Base 16 e valor posicional

Como o sistema hexadecimal é de base 16, cada posição representa uma potência de 16.

Por exemplo, o número hexadecimal $2F_{16}$ pode ser convertido para decimal assim:

$$2F_{16} = 2 \times 16^1 + F \times 16^0$$

Como:

$$F = 15, \quad 16^1 = 16, \quad 16^0 = 1$$

temos:

$$2F_{16} = 2 \times 16 + 15 \times 1$$

$$2F_{16} = 32 + 15 = 47_{10}$$

Portanto:

$$2F_{16} = 47_{10}$$

2.5.2 Relação entre hexadecimal e binário

A relação entre hexadecimal e binário é direta porque 16 é uma potência de 2:

$$16 = 2^4$$

Isso significa que cada dígito hexadecimal corresponde exatamente a 4 bits.

Hexadecimal	**Binário**
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

Exemplo didático

Para converter o valor hexadecimal $A3_{16}$ para binário, substituímos cada dígito por seu equivalente de 4 bits:

$$A_{16} = 1010_2$$

$$3_{16} = 0011_2$$

Logo:

$$A3_{16} = 10100011_2$$

2.5.3 Hexadecimal em cores digitais

Um dos usos mais conhecidos do sistema hexadecimal aparece na representação de cores em páginas web, imagens digitais e interfaces gráficas.

Em HTML e CSS, uma cor pode ser representada por três pares de dígitos hexadecimais, no formato:

$$\#RRGGBB$$

Nesse formato:

- `RR` representa a intensidade de vermelho;
- `GG` representa a intensidade de verde;
- `BB` representa a intensidade de azul.

Cada par hexadecimal varia de `00` a `FF`. Em decimal, isso corresponde a valores de 0 a 255.

Exemplo didático

A cor branca pode ser representada como:

```
#FFFFFF
```

Isso significa intensidade máxima de vermelho, verde e azul.

A cor preta pode ser representada como:

```
#000000
```

Isso significa ausência de vermelho, verde e azul.

2.5.4 Hexadecimal em memória e programação

O sistema hexadecimal também aparece em endereços de memória, identificadores, depuração de programas, códigos de caracteres, representação de bytes e análise de arquivos binários.

Em muitas linguagens de programação, valores hexadecimais são indicados com o prefixo `0x`.

Exemplo didático

Em Python, podemos escrever:

```
numero = 0x2F
print(numero)
```

A saída será:

```
47
```

Isso acontece porque `0x2F` representa o valor hexadecimal $2F_{16}$, que corresponde ao decimal 47.

2.5.5 Por que hexadecimal é tão usado em computação?

Números binários podem ficar longos e difíceis de ler. O hexadecimal resolve esse problema ao representar grupos de 4 bits com apenas um símbolo.

Exemplo didático

O número binário:

$$11111111_2$$

pode ser agrupado em dois blocos de 4 bits:

$$1111\ 1111$$

Cada bloco `1111` corresponde a `F` em hexadecimal. Portanto:

$$11111111_2 = FF_{16}$$

Essa forma é mais compacta e mais fácil de ler.

Observação

O valor hexadecimal `FF` é muito comum em computação. Ele representa o maior valor possível em 8 bits:

$$FF_{16} = 255_{10} = 11111111_2$$

2.6 Conversão entre bases

Converter números entre bases significa representar a mesma quantidade utilizando sistemas de numeração diferentes. Por exemplo, o número decimal 10_{10} pode ser escrito

como 1010_2 em binário, 12_8 em octal e A_{16} em hexadecimal.

A quantidade representada é a mesma. O que muda é a forma de escrita.

Observação

Quando trabalhamos com mais de uma base numérica, é importante indicar a base usada. Para isso, podemos escrever a base como índice:

$$1010_2, \quad 12_8, \quad 10_{10}, \quad A_{16}$$

Assim, evitamos confundir, por exemplo, 10_2 com 10_{10}. O primeiro representa o valor decimal 2, enquanto o segundo representa o valor decimal 10.

2.6.1 Conversão de decimal para binário

Para converter um número decimal para binário, podemos usar o método das divisões sucessivas por 2. O procedimento consiste em dividir o número por 2, anotar o resto da divisão e repetir o processo com o quociente até chegar a zero.

O número binário é formado pelos restos lidos de baixo para cima.

Exemplo didático

Vamos converter o número decimal 25_{10} para binário.

$$25 \div 2 = 12 \quad \text{resto } 1$$

$$12 \div 2 = 6 \quad \text{resto } 0$$

$$6 \div 2 = 3 \quad \text{resto } 0$$

$$3 \div 2 = 1 \quad \text{resto } 1$$

$$1 \div 2 = 0 \quad \text{resto } 1$$

Lendo os restos de baixo para cima:

$$25_{10} = 11001_2$$

Divisão	Quociente	Resto
$25 \div 2$	12	1
$12 \div 2$	6	0
$6 \div 2$	3	0
$3 \div 2$	1	1
$1 \div 2$	0	1

2.6.2 Conversão de binário para decimal

Para converter um número binário para decimal, multiplicamos cada dígito pela potência de 2 correspondente à sua posição. A contagem das posições começa da direita para a esquerda, iniciando em zero.

Exemplo didático

Vamos converter 11001_2 para decimal.

$$11001_2 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$11001_2 = 1 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 1 \times 1$$

$$11001_2 = 16 + 8 + 0 + 0 + 1 = 25_{10}$$

Portanto:

$$11001_2 = 25_{10}$$

2.6.3 Conversão de decimal para octal

A conversão de decimal para octal segue a mesma lógica da conversão para binário, mas usando divisões sucessivas por 8.

Exemplo didático

Vamos converter 95_{10} para octal.

$$95 \div 8 = 11 \quad \text{resto } 7$$

$$11 \div 8 = 1 \quad \text{resto } 3$$

$$1 \div 8 = 0 \quad \text{resto } 1$$

Lendo os restos de baixo para cima:

$$95_{10} = 137_8$$

Divisão	**Quociente**	**Resto**
$95 \div 8$	11	7
$11 \div 8$	1	3
$1 \div 8$	0	1

2.6.4 Conversão de octal para decimal

Para converter de octal para decimal, multiplicamos cada dígito pela potência de 8 correspondente à sua posição.

Exemplo didático

Vamos converter 137_8 para decimal.

$$137_8 = 1 \times 8^2 + 3 \times 8^1 + 7 \times 8^0$$

$$137_8 = 1 \times 64 + 3 \times 8 + 7 \times 1$$

$$137_8 = 64 + 24 + 7 = 95_{10}$$

Portanto:

$$137_8 = 95_{10}$$

2.6.5 Conversão de decimal para hexadecimal

Para converter um número decimal para hexadecimal, usamos divisões sucessivas por 16. Os restos podem variar de 0 a 15. Quando o resto for maior que 9, usamos letras:

$$10 = A, \quad 11 = B, \quad 12 = C, \quad 13 = D, \quad 14 = E, \quad 15 = F$$

Exemplo didático

Vamos converter 254_{10} para hexadecimal.

$$254 \div 16 = 15 \quad \text{resto } 14$$

$$15 \div 16 = 0 \quad \text{resto } 15$$

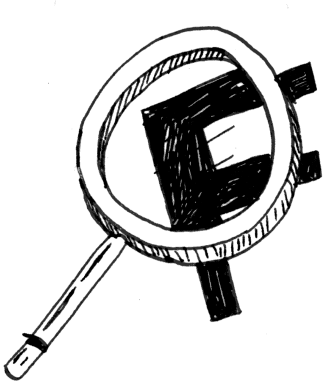
O resto 14 corresponde à letra E, e o resto 15 corresponde à letra F. Lendo os restos de baixo para cima:

$$254_{10} = FE_{16}$$

Divisão	Quociente	Resto decimal	Resto hexadecimal
$254 \div 16$	15	14	E
$15 \div 16$	0	15	F

2.6.6 Conversão de hexadecimal para decimal

Para converter de hexadecimal para decimal, multiplicamos cada símbolo pela potência de 16 correspondente à sua posição.



drawn by adit.io

Exemplo didático

Vamos converter FE_{16} para decimal.

$$FE_{16} = F \times 16^1 + E \times 16^0$$

Como:

$$F = 15$$

$$E = 14$$

temos:

$$FE_{16} = 15 \times 16 + 14 \times 1$$

$$FE_{16} = 240 + 14 = 254_{10}$$

Portanto:

$$FE_{16} = 254_{10}$$

2.6.7 Conversão direta entre binário e octal

A conversão entre binário e octal é simples porque cada dígito octal corresponde a um grupo de 3 bits:

$$8 = 2^3$$

Para converter de binário para octal, agrupamos os bits de três em três, da direita para a esquerda. Se necessário, completamos o grupo da esquerda com zeros.

Exemplo didático

Vamos converter 1101011_2 para octal.

Primeiro, agrupamos os bits em blocos de três, da direita para a esquerda:

$$1\ 101\ 011$$

Como o primeiro grupo ficou incompleto, acrescentamos zeros à esquerda:

$$001\ 101\ 011$$

Agora convertemos cada grupo:

$$001_2 = 1_8$$

$$101_2 = 5_8$$

$$011_2 = 3_8$$

Logo:

$$1101011_2 = 153_8$$

Para converter de octal para binário, fazemos o processo inverso: substituímos cada dígito octal pelo grupo de 3 bits correspondente.

Exemplo didático

Vamos converter 724_8 para binário.

$$7_8 = 111_2$$

$$2_8 = 010_2$$

$$4_8 = 100_2$$

Logo:

$$724_8 = 111010100_2$$

Octal	Binário
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

2.6.8 Conversão direta entre binário e hexadecimal

A conversão entre binário e hexadecimal também é direta porque cada dígito hexadecimal corresponde a um grupo de 4 bits:

$$16 = 2^4$$

Para converter de binário para hexadecimal, agrupamos os bits de quatro em quatro, da direita para a esquerda. Se necessário, completamos o grupo da esquerda com zeros.

Exemplo didático

Vamos converter 11110110_2 para hexadecimal.
Agrupando os bits de quatro em quatro:

$$1111\ 0110$$

Agora convertemos cada grupo:

$$1111_2 = F_{16}$$

$$0110_2 = 6_{16}$$

Logo:

$$11110110_2 = F6_{16}$$

Para converter de hexadecimal para binário, substituímos cada símbolo hexadecimal pelo grupo de 4 bits correspondente.

Exemplo didático

Vamos converter $3A9_{16}$ para binário.

$$3_{16} = 0011_2$$

$$A_{16} = 1010_2$$

$$9_{16} = 1001_2$$

Logo:

$$3A9_{16} = 001110101001_2$$

Em muitos casos, podemos remover zeros à esquerda:

$$3A9_{16} = 1110101001_2$$

Hexadecimal	**Binário**
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

2.6.9 Conversão entre octal e hexadecimal

Para converter diretamente entre octal e hexadecimal, uma estratégia simples é usar o binário como base intermediária.

Exemplo didático

Vamos converter 745_8 para hexadecimal.

Primeiro, convertemos cada dígito octal em 3 bits:

$$7_8 = 111_2$$

$$4_8 = 100_2$$

$$5_8 = 101_2$$

Logo:

$$745_8 = 111100101_2$$

Agora agrupamos em blocos de 4 bits, da direita para a esquerda:

$$1\ 1110\ 0101$$

Completando o grupo da esquerda com zeros:

$$0001\ 1110\ 0101$$

Convertendo cada grupo para hexadecimal:

$$0001_2 = 1_{16}$$

$$1110_2 = E_{16}$$

$$0101_2 = 5_{16}$$

Portanto:

$$745_8 = 1E5_{16}$$

2.6.10 Resumo dos métodos

Conversão	Método recomendado
Decimal para binário	Divisões sucessivas por 2 e leitura dos restos de baixo para cima.
Binário para decimal	Soma dos dígitos multiplicados por potências de 2.
Decimal para octal	Divisões sucessivas por 8 e leitura dos restos de baixo para cima.
Octal para decimal	Soma dos dígitos multiplicados por potências de 8.
Decimal para hexadecimal	Divisões sucessivas por 16 e leitura dos restos de baixo para cima.
Hexadecimal para decimal	Soma dos símbolos multiplicados por potências de 16.
Binário para octal	Agrupamento dos bits em blocos de 3, da direita para a esquerda.
Octal para binário	Substituição de cada dígito octal por 3 bits.
Binário para hexadecimal	Agrupamento dos bits em blocos de 4, da direita para a esquerda.
Hexadecimal para binário	Substituição de cada dígito hexadecimal por 4 bits.
Octal para hexadecimal	Conversão para binário e depois para hexadecimal.
Hexadecimal para octal	Conversão para binário e depois para octal.

2.6.11 Erros comuns em conversões

Durante as conversões entre bases, alguns erros são frequentes:

- esquecer de ler os restos de baixo para cima nas conversões a partir do decimal;
- confundir o valor de um número com sua representação. Por exemplo, 10_2 não é igual a 10_{10};
- esquecer que hexadecimal usa letras de A a F;
- agrupar bits pelo lado errado. Em conversões de binário para octal ou hexadecimal, os agrupamentos devem ser feitos da direita para a esquerda;
- remover zeros à esquerda quando eles fazem parte de uma representação padronizada, como bytes, códigos de cor ou endereços.

Observação

Zeros à esquerda normalmente não alteram o valor numérico. Por exemplo:

$$00101_2 = 101_2$$

No entanto, em computação, zeros à esquerda podem ser importantes para manter um tamanho fixo de representação, como em um byte de 8 bits:

$$00000101_2$$

2.6.12 Aplicação prática com Python

As linguagens de programação geralmente possuem funções para converter números entre bases. Em Python, podemos usar as funções `bin`, `oct` e `hex`.

Exemplo didático

```
numero = 254

print(bin(numero))  # bin rio
print(oct(numero))  # octal
print(hex(numero))  # hexadecimal
```

A saída será:

```
0b11111110
0o376
0xfe
```

Os prefixos indicam a base:

- `0b`: binário;
- `0o`: octal;
- `0x`: hexadecimal.

Também é possível converter strings em diferentes bases para números decimais usando a função `int`.

Exemplo didático

```
print(int("11111110", 2))
print(int("376", 8))
print(int("fe", 16))
```

A saída será:

```
254
254
254
```

2.6.13 Atividade prática

Atividade prática

Utilize papel, calculadora ou Python para converter os seguintes números entre bases:

1. 37_{10} para binário, octal e hexadecimal.
2. 101101_2 para decimal, octal e hexadecimal.
3. 72_8 para decimal, binário e hexadecimal.
4. $2A_{16}$ para decimal, binário e octal.

Depois, responda: em quais conversões o binário funcionou como uma ponte entre as bases?

Exercícios

1. Converter um número para outra base muda o valor do número ou apenas sua forma de representação? Explique com um exemplo.

2. Por que o método das divisões sucessivas funciona para converter números decimais para outras bases?

3. Por que a conversão entre binário, octal e hexadecimal é mais direta do que a conversão entre decimal e binário?

4. Em quais situações práticas um desenvolvedor pode encontrar números em hexadecimal, octal ou binário?

5. Por que é importante indicar a base de um número quando ele pode gerar ambiguidade?

6. Explique por que 100_2, 4_{10} e 4_8 não representam necessariamente a mesma escrita, embora alguns deles possam representar a mesma quantidade.

7. Ao trabalhar com permissões de arquivos, cores digitais, endereços de memória ou bytes, por que compreender bases numéricas ajuda a evitar erros?

8. O que os prefixos `0b`, `0o` e `0x` indicam em linguagens de programação como Python?

9. Por que zeros à esquerda podem ser irrelevantes em matemática, mas importantes em computação?

10. Escolha um número decimal pequeno e represente-o em binário, octal e hexadecimal. Depois, explique qual representação parece mais fácil de ler e por quê.

2.7 Bits, bytes e unidades de medida

- 1 bit: menor unidade de informação;
- 1 byte: conjunto de 8 bits;
- 1 KB: aproximadamente mil bytes;
- 1 MB: aproximadamente um milhão de bytes;
- 1 GB: aproximadamente um bilhão de bytes;
- 1 TB: aproximadamente um trilhão de bytes.

2.8 Unidades decimais e binárias

Observação

Em muitos contextos, os termos KB, MB e GB são usados de maneira aproximada. Tecnicamente, existe diferença entre unidades decimais e binárias:

- 1 KB = 1.000 bytes;
- 1 KiB = 1.024 bytes;
- 1 MB = 1.000.000 bytes;
- 1 MiB = 1.048.576 bytes.

2.9 Representação de caracteres

Computadores não armazenam apenas números. Eles também precisam representar letras, acentos, símbolos, sinais de pontuação, emojis e caracteres usados em diferentes idiomas.

Para que isso seja possível, cada caractere precisa ser associado a um valor numérico. Esse valor, por sua vez, é armazenado internamente como bits.

Observação

O computador não "entende" a letra `A` como uma letra da mesma forma que uma pessoa entende. Internamente, essa letra é representada por um código numérico, que é armazenado em formato binário.

2.9.1 O que é um caractere?

Um caractere é uma unidade textual que pode ser representada em um sistema computacional. Ele pode ser uma letra, um número, um sinal de pontuação, um espaço em branco ou um símbolo especial.

Exemplos de caracteres:

$$A, \quad b, \quad 7, \quad ?, \quad @, \quad \%, \quad \text{ç}, \quad \text{ã}, \quad \Omega, \quad \heartsuit$$

Em um texto simples, cada elemento visível geralmente corresponde a um caractere. O espaço entre palavras também é um caractere, mesmo que não tenha uma forma visível.

Exemplo didático

Na palavra:

`Casa`

temos quatro caracteres:

$$C, \quad a, \quad s, \quad a$$

Na frase:

`Olá mundo`

temos nove caracteres visíveis se contarmos as letras e o espaço entre as palavras.

2.9.2 Da letra ao número

Para armazenar caracteres, o computador utiliza uma tabela de codificação. Essa tabela associa cada caractere a um número.

Por exemplo, em uma codificação bastante conhecida, a letra `A` está associada ao valor decimal 65. Esse número pode ser convertido para binário:

$$65_{10} = 01000001_2$$

Assim, quando um arquivo de texto contém a letra `A`, o computador pode armazenar internamente o valor binário correspondente.

Exemplo didático

A palavra `ADS`, usando uma codificação compatível com ASCII, pode ser representada pelos seguintes valores decimais:

$$A = 65$$

$$D = 68$$

$$S = 83$$

Em binário, esses valores podem ser representados como:

$$A = 01000001_2$$

$$D = 01000100_2$$

$$S = 01010011_2$$

2.9.3 ASCII

ASCII é a sigla para *American Standard Code for Information Interchange*, ou Código Americano Padrão para Intercâmbio de Informação.

A tabela ASCII foi criada para padronizar a representação de caracteres em computadores e sistemas de comunicação. Ela utiliza originalmente 7 bits, permitindo representar 128 valores diferentes:

$$2^7 = 128$$

Esses valores incluem letras maiúsculas e minúsculas do alfabeto inglês, números, sinais de pontuação e alguns caracteres de controle.

Caractere	Decimal	Binário
A	65	01000001
B	66	01000010
C	67	01000011
a	97	01100001
b	98	01100010
c	99	01100011
0	48	00110000
1	49	00110001
Espaço	32	00100000

Observação

Embora a tabela ASCII original utilize 7 bits, é comum representar seus valores em 8 bits, completando com um zero à esquerda. Isso facilita o armazenamento em bytes.

2.9.4 Limitações do ASCII

O ASCII foi muito importante historicamente, mas possui uma limitação clara: ele foi pensado principalmente para o alfabeto inglês.

Por isso, a tabela ASCII original não representa adequadamente caracteres como:

á, é, í, ó, ú, ç, ã, õ

Também não representa alfabetos como grego, cirílico, árabe, hebraico, chinês, japonês, coreano e muitos outros sistemas de escrita.

Essa limitação criou a necessidade de codificações mais amplas, capazes de representar caracteres usados em diferentes idiomas e contextos.

Exemplo didático

Um texto em português, como:

`ação, coração, ciência, programação`

possui caracteres acentuados e cedilha. Esses caracteres não fazem parte da tabela ASCII original, o que exige codificações mais completas.

2.9.5 Unicode

Unicode é um padrão criado para representar caracteres de praticamente todos os sistemas de escrita usados no mundo. Seu objetivo é permitir que textos em diferentes idiomas possam ser armazenados, transmitidos e exibidos corretamente.

Em vez de depender de várias tabelas incompatíveis, o Unicode define um código único para cada caractere. Esses códigos são chamados de pontos de código, ou *code points*.

Um ponto de código Unicode costuma ser escrito no formato:

`U+XXXX`

Exemplo didático

Alguns exemplos de pontos de código Unicode:

Caractere	Ponto de código Unicode
A	`U+0041`
ç	`U+00E7`
á	`U+00E1`
	`U+03A9`
€	`U+20AC`

Com o Unicode, tornou-se possível representar, em um mesmo documento, caracteres de diferentes idiomas, símbolos matemáticos, sinais monetários e emojis.

2.9.6 UTF-8

Unicode define quais caracteres existem e quais códigos estão associados a eles. Mas ainda é necessário definir como esses códigos serão armazenados em bytes. É nesse ponto que entram as formas de codificação, como UTF-8, UTF-16 e UTF-32.

A codificação UTF-8 é uma das mais usadas atualmente, especialmente na Web, em sistemas Linux, bancos de dados, linguagens de programação e arquivos de texto.

UTF-8 possui uma característica importante: ela usa quantidade variável de bytes para representar caracteres.

- caracteres mais simples, como letras do inglês, usam 1 byte;
- caracteres acentuados geralmente usam 2 bytes;
- alguns símbolos usam 3 bytes;
- muitos emojis usam 4 bytes.

Exemplo didático

Em UTF-8, a letra `A` ocupa 1 byte:

$$A \rightarrow 01000001$$

Já o caractere `ç` ocupa mais de 1 byte, pois não pertence ao conjunto ASCII original. Por isso, duas palavras com a mesma quantidade de caracteres visíveis podem ocupar tamanhos diferentes em bytes.

2.9.7 Caracteres e tamanho de arquivos

Quando salvamos um arquivo de texto, seu tamanho depende da quantidade de caracteres e da codificação utilizada.

Um arquivo contendo apenas caracteres ASCII simples tende a ocupar aproximadamente 1 byte por caractere em UTF-8. Já um arquivo com muitos acentos, símbolos ou emojis pode ocupar mais bytes.

Exemplo didático

Considere as duas palavras:

`casa`

`ação`

A palavra `casa` contém apenas caracteres simples do conjunto ASCII e tende a ocupar 4 bytes em UTF-8.
A palavra `ação` contém caracteres como `ç` e `ã`, que exigem mais de 1 byte cada em UTF-8. Portanto, embora também tenha quatro caracteres visíveis, pode ocupar mais de 4 bytes no arquivo.

Observação

Quantidade de caracteres e tamanho em bytes não são sempre a mesma coisa. Isso é especialmente importante quando trabalhamos com textos em português, bancos de dados, APIs, arquivos CSV, páginas web e sistemas multilíngues.

2.9.8 Problemas de codificação

Problemas de codificação acontecem quando um texto é gravado usando uma codificação e lido usando outra. Nesses casos, caracteres acentuados podem aparecer de forma incorreta.

Um exemplo comum é ver textos assim:

programação

ou com símbolos estranhos no lugar de acentos.

Esses problemas indicam que o sistema não interpretou corretamente os bytes do arquivo.

Exemplo didático

Um arquivo salvo em UTF-8 pode ser aberto por um programa que espera outra codificação. Nesse caso, a palavra:

programação

pode aparecer com caracteres incorretos.

O problema não está necessariamente na palavra original, mas na forma como os bytes foram interpretados.

2.9.9 Representação de caracteres em HTML

Em páginas web, é importante indicar a codificação usada pelo documento. Em HTML, isso normalmente é feito com a tag:

```
<meta charset="UTF-8">
```

Essa linha informa ao navegador que o documento utiliza a codificação UTF-8. Sem essa informação, caracteres acentuados podem ser interpretados incorretamente, dependendo do ambiente.

Exemplo didático

Uma página HTML básica com codificação UTF-8:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>Exemplo</title>
</head>
<body>
    <p>Programao, cincia e corao.</p>
</body>
</html>
```

2.9.10 Representação de caracteres em Python

Em Python, podemos usar funções para descobrir o código associado a um caractere e também para converter códigos em caracteres.

A função `ord` retorna o código numérico de um caractere. A função `chr` faz o caminho inverso.

Exemplo didático

```
print(ord("A"))
print(ord("  "))
print(chr(65))
print(chr(231))
```

Uma saída possível é:

```
65
231
A

```

Também podemos verificar como uma string é codificada em bytes usando o método `encode`.

Exemplo didático

```
texto = " a    o "

print(texto)
print(len(texto))
print(texto.encode("utf-8"))
print(len(texto.encode("utf-8")))
```

Esse exemplo permite comparar a quantidade de caracteres visíveis com a quantidade de bytes usados na codificação UTF-8.

2.9.11 Caracteres de controle

Nem todos os caracteres são visíveis na tela. Alguns são chamados de caracteres de controle, pois indicam ações ou separações no texto.

Exemplos comuns:

- quebra de linha;
- tabulação;

- retorno de carro;
- fim de arquivo em alguns contextos;
- separadores usados em protocolos e arquivos.

Em muitas linguagens de programação, esses caracteres são representados por sequências especiais.

Representação	Significado
`\n`	Quebra de linha
`\t`	Tabulação
`\r`	Retorno de carro
`\\`	Barra invertida
`\"`	Aspas duplas em uma string

Exemplo didático

Em Python:

```
print("Linha 1\nLinha 2")
print("Nome:\tAna")
```

A sequência `\n` cria uma nova linha, enquanto `\t` insere uma tabulação.

2.9.12 Por que isso importa para o desenvolvimento de sistemas?

A representação de caracteres aparece em diversas situações práticas do desenvolvimento de software:

- criação de páginas HTML;
- leitura e escrita de arquivos de texto;
- importação de arquivos CSV;
- armazenamento de dados em bancos de dados;
- envio de dados por APIs;
- tratamento de nomes de pessoas, cidades e instituições;
- internacionalização de sistemas;
- busca e comparação de textos;
- processamento de linguagem natural.

Um erro de codificação pode afetar relatórios, formulários, cadastros, mensagens, arquivos importados e respostas de APIs.

Exemplo didático

Imagine um sistema acadêmico que armazena nomes de estudantes. Se a codificação não for tratada corretamente, nomes como:

`João`, `Vitória`, `Conceição`, `Luísa`

podem ser gravados ou exibidos de forma incorreta. Isso compromete a qualidade dos dados e a experiência do usuário.

2.9.13 Atividade prática

Atividade prática

Crie um arquivo de texto chamado `caracteres.txt` contendo as seguintes linhas:

```
ADS
Programacao
Programao
ao
corao
Ol mundo
```

Depois, responda:

1. Qual é o tamanho do arquivo em bytes?
2. Todas as linhas possuem a mesma quantidade de caracteres?
3. As linhas com acentos ocupam o mesmo espaço que as linhas sem acentos?
4. O emoji ocupa o mesmo espaço que uma letra comum?
5. O que isso revela sobre a diferença entre caracteres e bytes?

Se possível, use Python para comparar `len(texto)` com `len(texto.encode("utf-8"))`.

2.10 Representação de números inteiros

Além de representar caracteres, imagens, sons e instruções, os computadores precisam representar números. Entre os tipos numéricos mais básicos estão os números inteiros.

Números inteiros são valores sem parte decimal, como:

$$-3, \quad -2, \quad -1, \quad 0, \quad 1, \quad 2, \quad 3$$

Em programação, números inteiros aparecem em contadores, identificadores, idades, quantidades, índices de vetores, códigos de status, tamanhos de arquivos, permissões, endereços e muitas outras situações.

Observação

Embora os números inteiros pareçam simples para nós, o computador precisa representá-los usando apenas bits. Por isso, é necessário definir regras para armazenar valores positivos, negativos e o zero.

2.10.1 Inteiros sem sinal

A forma mais simples de representar números inteiros é considerar apenas valores não negativos. Nesse caso, dizemos que o número é um inteiro sem sinal, ou *unsigned integer*.

Com n bits, é possível representar:

$$2^n$$

combinações diferentes.

Como os inteiros sem sinal começam em zero, o intervalo de valores representáveis é:

$$0 \text{ até } 2^n - 1$$

Exemplo didático

Com 4 bits, temos:

$$2^4 = 16$$

Portanto, é possível representar 16 valores diferentes, de 0 até 15.

Binário	Decimal sem sinal
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

2.10.2 Limite de representação

Um ponto fundamental na computação é que a quantidade de bits define o intervalo de valores possíveis. Diferentemente da matemática abstrata, em um computador real os números ocupam espaço finito.

Quantidade de bits	Combinações	Intervalo sem sinal
4 bits	$2^4 = 16$	0 a 15
8 bits	$2^8 = 256$	0 a 255
16 bits	$2^{16} = 65.536$	0 a 65.535
32 bits	2^{32}	0 a 4.294.967.295
64 bits	2^{64}	0 a 18.446.744.073.709.551.615

Observação

Quanto mais bits são usados, maior é a quantidade de valores que podem ser representados. Porém, também aumenta o espaço ocupado na memória.

2.10.3 O problema dos números negativos

A representação sem sinal é suficiente para valores como idade, quantidade de itens, tamanho de arquivos ou número de acessos. No entanto, muitos problemas exigem números negativos.

Exemplos:

- temperaturas abaixo de zero;
- saldos negativos;
- diferenças entre valores;
- deslocamentos em coordenadas;
- resultados de subtrações;
- variações positivas e negativas.

Para representar números negativos, é necessário reservar parte das combinações de bits para valores menores que zero.

Uma primeira ideia seria usar um bit apenas para indicar o sinal. Porém, os computadores modernos usam principalmente uma técnica chamada **complemento de dois**.

2.10.4 Representação com sinal

Um inteiro com sinal, ou *signed integer*, pode representar tanto valores positivos quanto negativos.

Com n bits, continuam existindo 2^n combinações possíveis. A diferença é que essas combinações são divididas entre números negativos, zero e números positivos.

Na representação em complemento de dois, o intervalo de um inteiro com sinal usando n bits é:

$$-2^{n-1} \text{ até } 2^{n-1} - 1$$

Exemplo didático

Com 4 bits, o intervalo com sinal é:

$$-2^{4-1} \text{ até } 2^{4-1} - 1$$

$$-2^{3} \text{ até } 2^{3} - 1$$

$$-8 \text{ até } 7$$

Portanto, com 4 bits em complemento de dois, podemos representar os valores de -8 até 7.

Binário	**Decimal com sinal**
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	-8
1001	-7
1010	-6
1011	-5
1100	-4
1101	-3
1110	-2
1111	-1

2.10.5 O bit mais significativo

Na representação com sinal em complemento de dois, o bit mais à esquerda é chamado de bit mais significativo. Ele ajuda a identificar se o número é positivo ou negativo.

Em números com sinal:

- se o bit mais à esquerda é 0, o número é não negativo;
- se o bit mais à esquerda é 1, o número é negativo.

Exemplo didático

Usando 4 bits:

$$0110_2$$

começa com 0, portanto representa um número não negativo. Seu valor é:

$$0110_2 = 6_{10}$$

Já:

$$1110_2$$

começa com 1, portanto representa um número negativo em complemento de dois. Nesse caso, representa:

$$1110_2 = -2_{10}$$

Observação

O bit mais à esquerda não deve ser interpretado isoladamente como "apenas um sinal". No complemento de dois, ele participa do valor do número de uma forma específica. Por isso, não basta remover esse bit para descobrir o valor absoluto.

2.10.6 Complemento de dois

O complemento de dois é a forma mais comum de representar inteiros com sinal em computadores modernos.

Uma vantagem importante dessa representação é que operações de soma e subtração podem ser realizadas de maneira eficiente pelos circuitos do processador, usando regras semelhantes às dos números binários sem sinal.

Como obter o negativo de um número

Para encontrar a representação negativa de um número em complemento de dois, podemos seguir dois passos:

1. inverter todos os bits;
2. somar 1 ao resultado.

Exemplo didático

Vamos representar -5 usando 4 bits.

Primeiro, escrevemos 5 em binário com 4 bits:

$$5_{10} = 0101_2$$

Agora, invertemos todos os bits:

$$0101 \rightarrow 1010$$

Depois, somamos 1:

$$1010 + 1 = 1011$$

Portanto:

$$-5_{10} = 1011_2$$

em complemento de dois com 4 bits.

Outro exemplo

Exemplo didático

Vamos representar -3 usando 8 bits.

Primeiro, escrevemos 3 em binário com 8 bits:

$$3_{10} = 00000011_2$$

Invertemos os bits:

$$00000011 \rightarrow 11111100$$

Somamos 1:

$$11111100 + 1 = 11111101$$

Logo:

$$-3_{10} = 11111101_2$$

em complemento de dois com 8 bits.

2.10.7 Interpretando números negativos em complemento de dois

Para descobrir o valor decimal de um número negativo em complemento de dois, podemos aplicar o processo inverso:

1. observar se o bit mais à esquerda é 1;
2. inverter todos os bits;
3. somar 1;
4. converter o resultado para decimal;
5. colocar o sinal negativo.

Exemplo didático

Vamos interpretar o número binário com 8 bits:

$$11111010_2$$

Como o bit mais à esquerda é 1, trata-se de um número negativo.
Invertemos os bits:

$$11111010 \rightarrow 00000101$$

Somamos 1:

$$00000101 + 1 = 00000110$$

Convertendo para decimal:

$$00000110_2 = 6_{10}$$

Portanto:

$$11111010_2 = -6_{10}$$

em complemento de dois com 8 bits.

2.10.8 Soma com complemento de dois

Uma das razões para o uso do complemento de dois é que ele permite somar números positivos e negativos usando a mesma lógica de soma binária.

Exemplo didático

Vamos calcular:

$$5 + (-3)$$

usando 4 bits.
Primeiro, representamos os números:

$$5_{10} = 0101_2$$

$$-3_{10} = 1101_2$$

Agora somamos:

$$\begin{array}{r} 0101 \\ +1101 \\ \hline 10010 \end{array}$$

Como estamos usando apenas 4 bits, descartamos o bit excedente à esquerda:

$$0010$$

Logo:

$$5 + (-3) = 2$$

$$0010_2 = 2_{10}$$

Observação

O descarte do bit excedente faz sentido porque a operação está limitada ao tamanho definido da representação. Essa é uma das propriedades que torna o complemento de dois prático para implementação em hardware.

2.10.9 Overflow

Overflow ocorre quando o resultado de uma operação não cabe na quantidade de bits disponível.

Exemplo didático

Com 4 bits sem sinal, o maior valor possível é:

$$1111_2 = 15_{10}$$

Se somarmos:

$$15 + 1$$

teríamos:

$$1111 + 0001 = 10000$$

Mas o resultado precisa de 5 bits. Se o sistema só armazena 4 bits, o valor excede a capacidade de representação.

No caso de inteiros com sinal em 4 bits, o intervalo é de -8 até 7. Assim, a soma:

$$7 + 1$$

não pode ser representada corretamente com 4 bits em complemento de dois.

Exemplo didático

Usando 4 bits:

$$7_{10} = 0111_2$$

$$1_{10} = 0001_2$$

Somando:

$$\begin{array}{r} 0111 \\ +0001 \\ \hline 1000 \end{array}$$

Em complemento de dois com 4 bits, 1000_2 representa -8, não 8. Esse resultado incorreto ocorre por overflow.

Observação

Overflow é um problema importante em programação. Em algumas linguagens, ele pode gerar erro. Em outras, pode fazer o valor "dar a volta" e produzir resultados inesperados.

2.10.10 Inteiros em linguagens de programação

Linguagens de programação costumam oferecer tipos inteiros com tamanhos diferentes. Esses tamanhos determinam os intervalos possíveis.

Em muitas linguagens, encontramos tipos como:

- `byte`;
- `short`;
- `int`;
- `long`;
- `unsigned int`;
- `long long`.

Exemplo didático

Em uma linguagem que usa `int` com 32 bits com sinal, o intervalo normalmente é:

$$-2^{31} \text{ até } 2^{31} - 1$$

ou seja:

$$-2.147.483.648 \text{ até } 2.147.483.647$$

Observação

Em Python, os inteiros podem crescer além de 32 ou 64 bits, limitados principalmente pela memória disponível. Ainda assim, compreender a representação em bits continua importante, especialmente ao trabalhar com arquivos binários, redes, bancos de dados, linguagens de baixo nível e sistemas embarcados.

2.10.11 Exemplo prático com Python

Podemos usar Python para observar representações binárias de números inteiros.

Exemplo didático

```
numero = 13

print(bin(numero))
print(format(numero, "08b"))
```

A saída será:

```
0b1101
00001101
```

A função `bin` mostra a representação binária com o prefixo `0b`. Já a função `format` permite exibir o número com 8 bits, preenchendo com zeros à esquerda.

Para representar números negativos em complemento de dois com uma quantidade fixa de bits, podemos usar uma máscara.

Exemplo didático

```
numero = -3
bits = 8

representacao = numero & ((1 << bits) - 1)

print(format(representacao, "08b"))
```

A saída será:

```
11111101
```

Esse é o valor de -3 em complemento de dois usando 8 bits.

2.10.12 Aplicações práticas

A representação de números inteiros aparece em várias áreas da computação:

- contadores e laços de repetição;
- índices de listas, vetores e matrizes;
- identificadores de registros em bancos de dados;
- códigos de status em protocolos;
- permissões de arquivos;
- endereços e deslocamentos de memória;

- processamento de imagens;
- sistemas embarcados;
- comunicação entre dispositivos;
- arquivos binários.

Exemplo didático

Em uma imagem digital, cada componente de cor pode ser representado por um inteiro de 8 bits sem sinal. Assim, os valores possíveis vão de 0 até 255.
No modelo RGB, uma cor pode ser representada por três inteiros:

$$R, \quad G, \quad B$$

Cada um indicando a intensidade de vermelho, verde e azul.

2.10.13 Atividade prática

Atividade prática

Utilize papel, calculadora ou Python para responder:

1. Qual é o maior valor sem sinal que pode ser representado com 8 bits?
2. Qual é o intervalo de valores com sinal que pode ser representado com 8 bits em complemento de dois?
3. Represente 12_{10} em binário usando 8 bits.
4. Represente -12_{10} em complemento de dois usando 8 bits.
5. Interprete o valor 11110100_2 como inteiro com sinal usando complemento de dois com 8 bits.
6. Explique o que acontece quando somamos $127 + 1$ usando 8 bits com sinal.

2.11 Representação de números decimais

Além dos números inteiros, os computadores também precisam representar números com parte fracionária, como:

$$3{,}14, \quad -0{,}5, \quad 2{,}718, \quad 0{,}001, \quad 1500{,}75$$

Esses números aparecem em diversas situações: cálculos científicos, valores monetários, medições, coordenadas, porcentagens, notas, estatísticas, sensores, gráficos, jogos digitais e aplicações de inteligência artificial.

Na matemática, estamos acostumados a pensar nesses números como valores contínuos. No computador, porém, toda informação precisa ser representada por uma quantidade finita de bits. Isso cria um desafio: como armazenar números com casas decimais usando apenas zeros e uns?

Observação

Em computação, o termo "número decimal" pode gerar ambiguidade. Às vezes, ele significa um número escrito na base 10. Em outros contextos, é usado informalmente para indicar números com parte fracionária. Nesta seção, usaremos o termo para tratar de números que possuem casas decimais, também chamados de números reais ou números de ponto flutuante em programação.

2.11.1 Parte inteira e parte fracionária

Um número com casas decimais pode ser dividido em duas partes:

$$25{,}75$$

Nesse exemplo:

- a parte inteira é 25;
- a parte fracionária é 0,75.

No sistema decimal, as casas à direita da vírgula representam potências negativas de 10:

$$25{,}75 = 2 \times 10^1 + 5 \times 10^0 + 7 \times 10^{-1} + 5 \times 10^{-2}$$

Como:

$$10^1 = 10, \quad 10^0 = 1, \quad 10^{-1} = \frac{1}{10}, \quad 10^{-2} = \frac{1}{100}$$

temos:

$$25{,}75 = 20 + 5 + 0{,}7 + 0{,}05$$

$$25{,}75 = 25{,}75$$

A mesma ideia pode ser aplicada ao sistema binário. A diferença é que, em vez de potências de 10, usamos potências de 2.

2.11.2 Frações em binário

No sistema binário, as posições à esquerda da vírgula representam potências positivas de 2. As posições à direita representam potências negativas de 2.

Por exemplo:

$$101{,}101_2$$

pode ser interpretado como:

$$1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$$

Como:

$$2^2 = 4, \quad 2^1 = 2, \quad 2^0 = 1$$

$$2^{-1} = \frac{1}{2}, \quad 2^{-2} = \frac{1}{4}, \quad 2^{-3} = \frac{1}{8}$$

temos:

$$101{,}101_2 = 4 + 0 + 1 + \frac{1}{2} + 0 + \frac{1}{8}$$

$$101{,}101_2 = 5 + 0{,}5 + 0{,}125$$

$$101{,}101_2 = 5{,}625_{10}$$

Exemplo didático

O número binário:

$$10{,}01_2$$

representa:

$$1 \times 2^1 + 0 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2}$$

$$10{,}01_2 = 2 + 0 + 0 + \frac{1}{4}$$

$$10{,}01_2 = 2{,}25_{10}$$

2.11.3 Conversão da parte fracionária para binário

Para converter a parte fracionária de um número decimal para binário, podemos usar multiplicações sucessivas por 2.

O procedimento é:

1. multiplicar a parte fracionária por 2;
2. anotar a parte inteira do resultado;
3. repetir o processo com a nova parte fracionária;
4. os bits são lidos na ordem em que foram obtidos.

Exemplo didático

Vamos converter $0{,}625_{10}$ para binário.

$$0{,}625 \times 2 = 1{,}25$$

A parte inteira é 1.
Agora usamos a parte fracionária 0,25:

$$0{,}25 \times 2 = 0{,}5$$

A parte inteira é 0.
Agora usamos a parte fracionária 0,5:

$$0{,}5 \times 2 = 1{,}0$$

A parte inteira é 1.
Como a parte fracionária chegou a zero, paramos. Logo:

$$0{,}625_{10} = 0{,}101_2$$

Multiplicação	**Parte inteira**	**Nova fração**
$0{,}625 \times 2 = 1{,}25$	1	0,25
$0{,}25 \times 2 = 0{,}5$	0	0,5
$0{,}5 \times 2 = 1{,}0$	1	0,0

2.11.4 Conversão com parte inteira e parte fracionária

Quando o número possui parte inteira e parte fracionária, podemos converter cada parte separadamente.

Exemplo didático

Vamos converter $13{,}625_{10}$ para binário.
Primeiro, convertemos a parte inteira:

$$13_{10} = 1101_2$$

Depois, convertemos a parte fracionária:

$$0{,}625_{10} = 0{,}101_2$$

Juntando as duas partes:

$$13{,}625_{10} = 1101{,}101_2$$

2.11.5 Nem toda fração decimal tem representação binária finita

Um ponto muito importante é que nem todo número decimal com representação finita possui uma representação binária finita.

Por exemplo, o número:

$$0{,}5_{10}$$

tem representação binária simples:

$$0{,}5_{10} = 0{,}1_2$$

pois:

$$0{,}1_2 = 1 \times 2^{-1} = \frac{1}{2}$$

Já o número:

$$0{,}1_{10}$$

não possui representação binária finita. Sua representação em binário é periódica, isto é, continua indefinidamente.

Observação

Esse é um dos motivos pelos quais computadores podem apresentar pequenos erros em cálculos com números fracionários. O problema não é exatamente um defeito do computador, mas uma consequência da tentativa de representar certos valores decimais usando uma quantidade finita de bits.

2.11.6 Ponto fixo

Uma forma simples de representar números com casas decimais é usar **ponto fixo**. Nessa abordagem, define-se previamente quantas casas serão usadas para a parte inteira e quantas serão usadas para a parte fracionária.

Por exemplo, poderíamos definir uma representação com 8 bits, sendo:

- 4 bits para a parte inteira;
- 4 bits para a parte fracionária.

Nesse caso, o ponto binário fica sempre na mesma posição:

$$IIII{,}FFFF$$

onde I representa bits da parte inteira e F representa bits da parte fracionária.

Exemplo didático

Considere a representação:

$$0011{,}1000_2$$

A parte inteira é:

$$0011_2 = 3_{10}$$

A parte fracionária é:

$$1000_2 = 1 \times 2^{-1} + 0 \times 2^{-2} + 0 \times 2^{-3} + 0 \times 2^{-4}$$

$$1000_2 = 0{,}5_{10}$$

Portanto:

$$0011{,}1000_2 = 3{,}5_{10}$$

Observação

O ponto fixo é simples e previsível, mas possui limitações. Se reservarmos muitos bits para a parte fracionária, teremos menos espaço para números grandes. Se reservarmos muitos bits para a parte inteira, teremos menos precisão nas frações.

2.11.7 Ponto flutuante

A forma mais comum de representar números reais em computadores modernos é chamada de **ponto flutuante**. Essa representação permite armazenar números muito pequenos e muito grandes usando uma estrutura semelhante à notação científica.

Na notação científica decimal, podemos escrever:

$$123000 = 1{,}23 \times 10^{5}$$

$$0{,}00123 = 1{,}23 \times 10^{-3}$$

A ideia do ponto flutuante é parecida, mas usando base 2:

$$\text{número} = \text{sinal} \times \text{mantissa} \times 2^{\text{expoente}}$$

De forma simplificada, um número em ponto flutuante possui três partes:

- **sinal**: indica se o número é positivo ou negativo;
- **expoente**: indica a escala do número;
- **mantissa**: guarda os dígitos significativos do número.

Exemplo didático

A ideia geral pode ser comparada com escrever:

$$602000000000000000000000 = 6{,}02 \times 10^{23}$$

Em vez de armazenar todos os zeros, armazenamos os dígitos mais importantes e o expoente. Em ponto flutuante binário, o raciocínio é semelhante, mas a base usada é 2.

2.11.8 Precisão e arredondamento

Como a quantidade de bits é limitada, nem todos os números podem ser representados exatamente em ponto flutuante. Muitos valores são aproximados.

Isso significa que uma operação aparentemente simples pode produzir um resultado com pequena diferença.

Exemplo didático

Em várias linguagens de programação, uma expressão como:

```
print(0.1 + 0.2)
```

pode produzir uma saída parecida com:

```
0.30000000000000004
```

O resultado esperado matematicamente seria 0,3. A pequena diferença aparece porque valores como 0,1 e 0,2 não são representados de forma exata em binário com uma quantidade finita de bits.

Observação

Esse comportamento não é exclusivo de Python. Ele aparece em muitas linguagens e plataformas que usam representação binária de ponto flutuante.

2.11.9 Ponto flutuante em linguagens de programação

Em muitas linguagens, números com casas decimais são representados por tipos como:

- `float`;
- `double`;
- `real`;
- `decimal`, em alguns contextos específicos.

O tipo `float` normalmente usa menos bits e oferece menor precisão. O tipo `double` usa mais bits e oferece maior precisão. Ainda assim, ambos podem apresentar aproximações.

Exemplo didático

Em Python, o tipo `float` usa ponto flutuante de dupla precisão:

```
x = 0.1
y = 0.2

print(x + y)
print(type(x))
```

2.11.10 Números decimais em aplicações financeiras

Em aplicações financeiras, pequenos erros de arredondamento podem gerar problemas. Imagine um sistema bancário, uma folha de pagamento ou uma loja virtual acumulando diferenças de centavos em milhares de operações.

Por isso, em sistemas financeiros, nem sempre é adequado usar ponto flutuante binário comum para representar dinheiro.

Duas estratégias frequentes são:

- armazenar valores monetários como inteiros, usando centavos;
- usar tipos decimais próprios, como `Decimal`, que evitam parte dos problemas de aproximação binária.

Exemplo didático

Em vez de armazenar R$ 19,99 como número de ponto flutuante, um sistema pode armazenar o valor como 1999 centavos.
Assim, o valor é tratado como inteiro:

$$R\$\,19{,}99 \rightarrow 1999$$

Na exibição ao usuário, o sistema formata o número novamente como valor monetário.

2.11.11 Usando `Decimal` em Python

Python possui o módulo `decimal`, que permite trabalhar com aritmética decimal de forma mais adequada para certos tipos de aplicação.

Exemplo didático

```
from decimal import Decimal

a = Decimal("0.1")
b = Decimal("0.2")

print(a + b)
```

A saída será:

```
0.3
```

Observe que os valores foram escritos como strings, usando aspas. Isso evita que eles sejam primeiro convertidos para `float`.

Observação

Tipos decimais podem ser mais adequados para valores financeiros, mas costumam ser mais lentos do que números de ponto flutuante tradicionais. A escolha do tipo numérico depende do problema.

2.11.12 Erro absoluto e erro relativo

Quando um número é representado por aproximação, podemos analisar a diferença entre o valor real e o valor armazenado.

O **erro absoluto** é a diferença direta entre o valor esperado e o valor obtido:

$$\text{erro absoluto} = |\text{valor real} - \text{valor aproximado}|$$

O **erro relativo** compara o erro absoluto com o tamanho do valor real:

$$\text{erro relativo} = \frac{|\text{valor real} - \text{valor aproximado}|}{|\text{valor real}|}$$

Exemplo didático

Se o valor real é 100,0 e o valor aproximado é 99,9, então:

$$\text{erro absoluto} = |100{,}0 - 99{,}9| = 0{,}1$$

O erro relativo é:

$$\frac{0{,}1}{100{,}0} = 0{,}001$$

ou seja, aproximadamente 0,1%.

2.11.13 Comparação entre números de ponto flutuante

Como números de ponto flutuante podem ter pequenas diferenças de aproximação, comparar dois valores diretamente pode gerar resultados inesperados.

Exemplo didático

Em vez de escrever:

```
if resultado == 0.3:
    print("Igual")
```

pode ser melhor verificar se a diferença entre os valores é pequena:

```
resultado = 0.1 + 0.2

if abs(resultado - 0.3) < 0.000001:
    print("Praticamente igual")
```

Essa pequena margem de tolerância é frequentemente chamada de *epsilon*.

2.11.14 Aplicações práticas

A representação de números decimais aparece em muitas áreas da computação:

- sistemas financeiros;
- medições científicas;
- sensores de temperatura, pressão e distância;
- gráficos e visualizações;
- jogos digitais e simulações físicas;
- processamento de imagens;
- inteligência artificial e aprendizado de máquina;
- estatística e análise de dados;
- cálculos de porcentagens e médias;
- sistemas de geolocalização.

Exemplo didático

Em um sistema de geolocalização, coordenadas como latitude e longitude são representadas por números com casas decimais. Pequenos erros podem ser aceitáveis em alguns contextos, mas podem ser críticos em sistemas de navegação, agricultura de precisão ou aplicações de segurança.

2.11.15 Atividade prática

Atividade prática

Execute os comandos abaixo em Python e observe os resultados:

```
print(0.1 + 0.2)
print(0.3)

print(0.1 + 0.2 == 0.3)

resultado = 0.1 + 0.2
print(abs(resultado - 0.3) < 0.000001)
```

Depois, responda:

1. O resultado de `0.1 + 0.2` foi exatamente `0.3`?
2. Por que a comparação direta com `==` pode falhar?
3. Como a comparação com tolerância ajuda a lidar com aproximações?
4. Em quais tipos de sistemas esse comportamento poderia causar problemas?

2.11.16 Outra atividade: dinheiro como inteiro

Atividade prática

Considere os valores abaixo:

$$R\$\,10{,}50, \quad R\$\,3{,}25, \quad R\$\,7{,}99$$

Represente cada valor em centavos, usando números inteiros.
Depois, calcule a soma total em centavos e converta o resultado de volta para reais.
Responda:

1. Por que essa estratégia pode ser útil em sistemas financeiros?
2. Que cuidado deve ser tomado na hora de exibir o valor ao usuário?
3. Essa estratégia resolve todos os problemas financeiros possíveis ou apenas evita alguns erros de representação?

Conteúdos extras recomendados

Filmes e documentários

- *Hidden Figures* — mostra a atuação de matemáticas e cientistas em projetos aeroespaciais.
- *Her* — filme útil para discutir interfaces conversacionais, inteligência artificial, sistemas personalizados, interação humano-máquina e os impactos emocionais e sociais da tecnologia..
- *Matrix* — fiscute simulação, realidade virtual, sistemas computacionais complexos, controle e dependência tecnológica.
- *The Internet's Own Boy* — documentário sobre Aaron Swartz.
- *AlphaGo* — documentário sobre o sistema de inteligência artificial desenvolvido para jogar Go.

Capítulo 3

Princípios de Funcionamento de Sistemas Operacionais

Objetivos de aprendizagem

Ao final deste capítulo, você deverá ser capaz de:

- compreender o papel do sistema operacional;
- identificar processos de usuário e processos do sistema;
- explicar a divisão de tempo do processador;
- compreender conceitos de arquivos, diretórios e permissões;
- reconhecer tipos de arquivos que podem se tornar processos.

3.1 O que é um sistema operacional

O sistema operacional é o software responsável por intermediar a relação entre usuários, aplicações e hardware. Ele gerencia recursos como processador, memória, dispositivos de entrada e saída, arquivos e rede.

3.2 Funções principais de um sistema operacional

O sistema operacional é responsável por coordenar o uso dos recursos do computador. Ele atua como uma camada intermediária entre o hardware, os programas e os usuários. Sem essa camada de organização, cada aplicação precisaria lidar diretamente com detalhes complexos do processador, da memória, dos dispositivos, dos arquivos e da rede.

De forma simplificada, podemos dizer que o sistema operacional transforma o hardware em um ambiente utilizável. Ele oferece serviços, regras e interfaces para que programas

possam ser executados de maneira controlada, segura e eficiente.

Observação

Um sistema operacional não é apenas a interface gráfica que aparece na tela. Ele inclui também o núcleo do sistema, serviços em segundo plano, drivers, gerenciadores de arquivos, mecanismos de segurança, controle de processos, gerenciamento de memória e várias outras camadas internas.

3.2.1 Gerenciamento de processos

Uma das funções mais importantes do sistema operacional é o gerenciamento de processos. Um processo pode ser entendido como um programa em execução.

Quando abrimos um navegador, um editor de texto, um terminal ou uma aplicação qualquer, o sistema operacional cria um ou mais processos para representar essa execução. Cada processo precisa de recursos como tempo de processador, memória, arquivos, dispositivos e, em alguns casos, acesso à rede.

O sistema operacional é responsável por:

- criar processos;
- encerrar processos;
- alternar a execução entre processos;
- controlar prioridades;
- permitir comunicação entre processos;
- impedir que um processo interfira indevidamente em outro.

Exemplo didático

Ao abrir um navegador, um editor de código e um tocador de música ao mesmo tempo, parece que todos estão funcionando simultaneamente. Na prática, o sistema operacional organiza a divisão do processador entre esses programas, alternando rapidamente entre eles ou distribuindo tarefas entre diferentes núcleos da CPU.

3.2.2 Escalonamento do processador

O processador é um recurso disputado por vários processos. O sistema operacional precisa decidir qual processo será executado, por quanto tempo e em qual momento. Essa tarefa é chamada de escalonamento de processos.

O escalonador é o componente responsável por organizar essa divisão de tempo. Ele considera fatores como prioridade, estado do processo, tempo de espera, tipo de tarefa e disponibilidade da CPU.

Exemplo didático

Enquanto um vídeo é reproduzido, o navegador pode estar carregando uma página, o sistema pode estar atualizando arquivos em segundo plano e o usuário pode estar digitando em um editor de texto. O sistema operacional precisa distribuir o uso do processador para manter a sensação de resposta rápida.

Observação

A divisão de tempo do processador é uma das razões pelas quais temos a impressão de que muitos programas funcionam ao mesmo tempo, mesmo quando há menos núcleos de processamento do que processos em execução.

3.2.3 Gerenciamento de memória

A memória principal, geralmente chamada de memória RAM, é usada para armazenar temporariamente dados e instruções dos programas em execução. Como a memória é limitada, o sistema operacional precisa decidir como distribuí-la entre os processos.

Entre as funções relacionadas ao gerenciamento de memória, estão:

- reservar espaço de memória para processos;
- liberar memória quando processos terminam;
- evitar que um processo acesse a memória de outro indevidamente;
- organizar memória virtual;
- mover dados entre memória principal e armazenamento quando necessário.

Exemplo didático

Se um estudante abre um editor de código, um navegador com várias abas e uma máquina virtual ao mesmo tempo, cada aplicação consome parte da memória RAM. Caso a memória física não seja suficiente, o sistema operacional pode usar uma área do armazenamento como extensão temporária, conhecida como memória virtual ou área de troca.

Observação

Quando o computador começa a usar memória virtual de forma intensa, ele pode ficar mais lento. Isso acontece porque o armazenamento, mesmo em SSD, costuma ser mais lento do que a memória RAM.

3.2.4 Gerenciamento de arquivos

O sistema operacional também organiza a forma como os dados são armazenados em dispositivos como HDs, SSDs, pendrives e cartões de memória. Para isso, ele utiliza sistemas de arquivos.

Um sistema de arquivos define como os dados serão nomeados, armazenados, localizados, organizados e protegidos. Ele permite que usuários e programas trabalhem com arquivos e diretórios sem precisar conhecer todos os detalhes físicos do dispositivo de armazenamento.

Entre as funções do gerenciamento de arquivos, estão:

- criar arquivos;
- apagar arquivos;
- mover e renomear arquivos;
- organizar diretórios;
- controlar permissões;
- registrar datas de criação, modificação e acesso;
- mapear arquivos para regiões físicas do armazenamento.

Exemplo didático

Quando um usuário salva um documento chamado `relatorio.pdf`, o sistema operacional registra esse arquivo em um diretório, armazena seus dados no dispositivo de armazenamento e mantém informações como tamanho, tipo, permissões e data de modificação.

3.2.5 Gerenciamento de diretórios

Diretórios, também chamados de pastas, permitem organizar arquivos em uma estrutura hierárquica. Essa organização é fundamental para manter dados, programas, bibliotecas, configurações e documentos em locais adequados.

A estrutura de diretórios funciona em cascata: um diretório pode conter arquivos e também outros diretórios. Essa organização permite construir caminhos até os arquivos.

Exemplo didático

Um projeto de software pode ter a seguinte estrutura:

```
projeto/
 app.py
 README.md
 requirements.txt
 static/
    style.css
 templates/
     index.html
```

Nesse exemplo, o sistema operacional organiza arquivos e subdiretórios para que o desenvolvedor consiga localizar cada parte do projeto.

3.2.6 Controle de permissões e usuários

Sistemas operacionais modernos geralmente permitem que diferentes usuários utilizem o mesmo computador ou servidor. Para manter segurança e organização, o sistema precisa controlar quem pode acessar, modificar ou executar determinados arquivos e programas.

As permissões ajudam a responder perguntas como:

- quem pode ler este arquivo?
- quem pode modificar este arquivo?
- quem pode executar este programa?
- qual usuário é dono deste processo?
- quais ações exigem privilégios administrativos?

Exemplo didático

Em sistemas Linux, é comum encontrar permissões como:

```
-rwxr-xr--
```

Essa sequência indica permissões de leitura, escrita e execução para o dono do arquivo, o grupo e os demais usuários.

Observação

O controle de permissões é essencial em servidores, laboratórios, ambientes compartilhados e sistemas multiusuário. Ele evita que arquivos importantes sejam alterados indevidamente e limita danos causados por erros ou programas mal-intencionados.

3.2.7 Gerenciamento de dispositivos

O computador possui vários dispositivos físicos, como teclado, mouse, monitor, placa de rede, impressora, câmera, microfone, disco, placa de vídeo e interfaces USB. O sistema operacional precisa permitir que os programas usem esses dispositivos sem lidar diretamente com seus detalhes eletrônicos.

Para isso, o sistema operacional utiliza drivers. Um driver é um software que permite a comunicação entre o sistema operacional e um dispositivo específico.

Exemplo didático

Quando uma impressora é instalada, o sistema operacional precisa de um driver capaz de traduzir comandos de impressão para instruções que a impressora compreenda. Sem o driver adequado, a impressora pode não funcionar corretamente.

O gerenciamento de dispositivos envolve:

- detectar dispositivos conectados;
- carregar drivers;
- controlar entrada e saída de dados;
- gerenciar filas de impressão;
- permitir acesso a discos e unidades externas;
- lidar com erros de comunicação;
- controlar dispositivos proprietários quando necessário.

3.2.8 Gerenciamento de entrada e saída

Entrada e saída, frequentemente abreviadas como E/S ou I/O (*Input/Output*), representam a comunicação entre o computador e o mundo externo. Digitar no teclado, mover o mouse, ler um arquivo, gravar dados em disco, receber pacotes pela rede e enviar uma imagem ao monitor são exemplos de operações de entrada e saída.

Essas operações costumam ser mais lentas do que o processamento interno da CPU. Por isso, o sistema operacional precisa coordená-las de forma eficiente.

Exemplo didático

Quando um programa solicita a leitura de um arquivo grande, o sistema operacional pode continuar executando outros processos enquanto aguarda a conclusão da leitura. Isso evita que o computador fique completamente parado esperando uma única operação de entrada e saída terminar.

3.2.9 Gerenciamento de rede

Os sistemas operacionais modernos incluem recursos para comunicação em rede. Eles implementam protocolos, controlam interfaces de rede, gerenciam conexões e permitem que aplicações enviem e recebam dados.

Entre as funções relacionadas à rede, estão:

- configurar endereços IP;
- enviar e receber pacotes;
- resolver nomes de domínio;
- controlar portas de comunicação;
- permitir conexões remotas;
- fornecer serviços de firewall;
- apoiar protocolos como TCP, UDP, HTTP, DNS e SSH.

Exemplo didático

Quando um usuário acessa um site, o navegador faz uma solicitação pela rede. O sistema operacional participa desse processo ao enviar pacotes, controlar a interface de rede, resolver o endereço do servidor e entregar a resposta recebida ao navegador.

3.2.10 Interface com o usuário

O sistema operacional oferece formas de interação com o usuário. Essas interfaces podem ser gráficas, textuais ou programáticas.

Entre as principais formas de interface, estão:

- interface gráfica, com janelas, ícones, menus e botões;
- terminal ou linha de comando;
- APIs usadas por programas;
- painéis de configuração;
- notificações e serviços do sistema.

Exemplo didático

Ao copiar um arquivo usando o gerenciador de arquivos gráfico, o usuário interage por meio de janelas e menus. Ao copiar o mesmo arquivo com o comando `cp`, ele usa a interface de linha de comando. Nos dois casos, o sistema operacional executa operações semelhantes nos bastidores.

3.2.11 Segurança e proteção do sistema

O sistema operacional também atua na proteção dos dados, dos programas e dos usuários. Essa proteção envolve autenticação, autorização, isolamento de processos, atualizações, permissões, criptografia e controle de acesso a recursos.

Entre as funções de segurança, podemos citar:

- autenticar usuários;
- controlar permissões de arquivos;
- isolar processos;
- impedir acesso indevido à memória;
- registrar eventos em logs;
- aplicar atualizações de segurança;
- controlar conexões de rede;
- proteger áreas críticas do sistema.

Exemplo didático

Quando um programa tenta modificar um arquivo do sistema sem permissão administrativa, o sistema operacional pode bloquear a operação. Esse bloqueio protege o sistema contra alterações acidentais ou maliciosas.

3.2.12 Execução de serviços em segundo plano

Muitas tarefas do sistema operacional ocorrem sem que o usuário perceba. Essas tarefas são executadas por serviços em segundo plano, também chamados de daemons em sistemas Unix e Linux.

Exemplos de serviços em segundo plano:

- gerenciamento de rede;
- atualização automática;
- indexação de arquivos;
- sincronização de horário;
- impressão;
- monitoramento de energia;

Exemplo didático

Em um servidor web, o serviço responsável por responder requisições HTTP pode permanecer em execução mesmo sem uma interface gráfica aberta. Ele fica aguardando conexões e respondendo aos clientes conforme as solicitações chegam.

3.2.13 Abstração dos recursos do hardware

Uma das contribuições mais importantes do sistema operacional é criar abstrações. Uma abstração esconde detalhes complexos e oferece uma forma mais simples de utilizar determinado recurso.

Por exemplo:

- em vez de lidar diretamente com setores físicos do disco, usamos arquivos;
- em vez de controlar manualmente cada região da memória, usamos variáveis e estruturas de dados;
- em vez de enviar sinais elétricos diretamente à placa de rede, usamos sockets, protocolos e bibliotecas;
- em vez de controlar diretamente o processador, executamos processos e threads.

Observação

A abstração é fundamental para o desenvolvimento de software. Ela permite que o programador se concentre no problema que deseja resolver, sem precisar controlar todos os detalhes físicos do computador.

3.2.14 Resumo das principais funções

Função	Descrição
Gerenciamento de processos	Cria, encerra, alterna e controla programas em execução.
Escalonamento do processador	Divide o tempo de CPU entre os processos.
Gerenciamento de memória	Reserva, protege e libera memória para os programas.
Gerenciamento de arquivos	Organiza dados em arquivos e diretórios.
Controle de permissões	Define quem pode ler, modificar ou executar recursos.

Gerenciamento de dispositivos	Permite o uso de hardware por meio de drivers.
Entrada e saída	Coordena comunicação com discos, teclado, mouse, tela, rede e outros dispositivos.
Gerenciamento de rede	Controla comunicação entre computadores e serviços.
Interface com o usuário	Oferece interface gráfica, terminal e APIs.
Segurança	Protege arquivos, processos, memória, usuários e conexões.
Serviços em segundo plano	Executa tarefas contínuas sem interação direta do usuário.
Abstração de hardware	Simplifica o uso de recursos físicos por programas e usuários.

3.2.15 Atividade prática

Atividade prática

Observe seu computador durante o uso normal e identifique exemplos de funções realizadas pelo sistema operacional.
Responda:

1. Quais processos de usuário estão em execução?
2. Quais serviços do sistema parecem estar funcionando em segundo plano?
3. Quais dispositivos estão sendo usados?
4. Há acesso à rede no momento?
5. Quais arquivos foram abertos, criados ou modificados?
6. Que permissões ou restrições aparecem durante o uso?

Se estiver usando Linux, experimente comandos como:

```
ps aux
top
df -h
free -h
ip addr
ls -l
```

Se estiver usando Windows, experimente:

```
tasklist
systeminfo
ipconfig
dir
netstat -ano
```

3.3 Processos

3.3.1 O que é um processo

Um processo é um programa em execução. Enquanto um arquivo executável armazenado em disco é apenas um conjunto de instruções, um processo representa esse programa carregado na memória e sendo gerenciado pelo sistema operacional.

3.3.2 Programa versus processo

Exemplo didático

O navegador instalado no computador é um programa. Quando o usuário abre esse navegador, o sistema operacional cria um ou mais processos relacionados a ele.

3.3.3 Processos de usuário

Processos de usuário são processos iniciados direta ou indiretamente por um usuário. Eles correspondem, em geral, aos programas, aplicativos e comandos utilizados durante uma sessão de trabalho.

Quando uma pessoa abre um navegador, executa um editor de texto, inicia um terminal, roda um programa em Python ou abre uma IDE, o sistema operacional cria processos de usuário para representar essas aplicações em execução.

Exemplo didático

Ao executar o comando:

```
python3 app.py
```

o sistema operacional cria um processo associado ao interpretador `python3`, que passa a executar as instruções contidas no arquivo `app.py`. Esse processo pertence ao usuário que executou o comando.

Processos de usuário normalmente possuem permissões limitadas. Isso significa que eles podem acessar apenas os arquivos, dispositivos e recursos autorizados para aquele usuário.

Essa limitação é importante porque evita que um programa comum altere partes críticas do sistema operacional ou acesse dados de outros usuários sem permissão.

Observação

Quando um programa solicita uma ação que exige privilégios elevados, como instalar pacotes, alterar arquivos do sistema ou modificar configurações de rede, o sistema operacional pode exigir autenticação administrativa. Em sistemas Linux, isso costuma ocorrer por meio do comando `sudo`. Em sistemas Windows, pode aparecer uma solicitação de permissão do administrador.

Exemplos de processos de usuário

Alguns exemplos comuns de processos de usuário são:

- navegador web;
- editor de texto;
- ambiente de desenvolvimento, como VS Code;
- terminal;
- tocador de música;
- cliente de e-mail;
- programa em Python, Java, C ou JavaScript;
- servidor local iniciado pelo desenvolvedor;
- ferramenta Git executada pelo terminal;
- aplicação em contêiner iniciada pelo usuário.

Exemplo didático

Ao iniciar uma aplicação Flask durante o desenvolvimento, o estudante pode executar:

```
flask run
```

ou:

```
python app.py
```

Nesse caso, o servidor local é um processo de usuário. Ele permanece em execução aguardando requisições até ser encerrado pelo próprio usuário ou pelo sistema.

Identificação de processos de usuário

Em sistemas Linux, é possível listar processos com comandos como:

```
ps aux
top
htop
```

No resultado desses comandos, normalmente aparece uma coluna indicando o usuário responsável por cada processo.

Exemplo didático

Um resultado simplificado poderia ser:

```
USER        PID  %CPU  %MEM  COMMAND
duca       2145   3.1   4.2  firefox
duca       2301   0.5   1.1  code
duca       2450   0.1   0.3  bash
duca       2512   1.7   2.0  python3
```

Nesse exemplo, os processos `firefox`, `code`, `bash` e `python3` estão associados ao usuário `duca`.

Relação com desenvolvimento de software

Para quem desenvolve sistemas, compreender processos de usuário é fundamental. Durante o desenvolvimento, é comum iniciar servidores locais, executar scripts, rodar testes automatizados, acessar bancos de dados, subir contêineres e monitorar aplicações.

Em muitos casos, erros de desenvolvimento estão relacionados a processos:

- uma porta de rede já está sendo usada por outro processo;
- um servidor local continua em execução sem o estudante perceber;
- um processo consome muita memória;
- um programa entra em loop infinito;
- uma aplicação trava aguardando entrada ou resposta da rede;
- um processo não possui permissão para acessar determinado arquivo.

Atividade prática

Abra um terminal e execute uma aplicação simples, como:

```
python3 -m http.server 8000
```

Em outro terminal, liste os processos em execução e tente identificar o processo correspondente ao servidor Python. Depois, encerre o processo de forma controlada usando `Ctrl+C` no terminal em que ele foi iniciado.

3.3.4 Processos do sistema operacional

Processos do sistema operacional são processos responsáveis pelo funcionamento interno do sistema. Eles executam tarefas essenciais para que o computador permaneça operacional, seguro, conectado e pronto para atender às solicitações dos usuários e dos programas.

Esses processos geralmente são iniciados automaticamente durante a inicialização do sistema e continuam em execução em segundo plano. Em sistemas Unix e Linux, muitos desses processos são chamados de *daemons*. No Windows, é comum o uso do termo serviços.

> **Observação**
>
> Processos do sistema operacional nem sempre possuem janela visível ou interação direta com o usuário. Muitos deles trabalham silenciosamente em segundo plano, realizando tarefas como gerenciar rede, autenticação, energia, impressão, logs, atualizações e dispositivos.

Exemplos de processos do sistema

Entre os processos e serviços relacionados ao sistema operacional, podemos encontrar:

- processo de inicialização do sistema;
- gerenciador de login;
- serviços de rede;
- serviços de áudio;
- gerenciador de energia;
- serviços de impressão;
- serviços de atualização;
- gerenciador gráfico;
- serviços de registro de eventos;
- serviços de sincronização de horário;
- gerenciadores de dispositivos;
- processos relacionados ao sistema de arquivos.

Exemplo didático

Em sistemas Linux modernos, o processo `systemd` costuma ser um dos primeiros processos iniciados. Ele é responsável por iniciar e controlar diversos serviços do sistema.
Em uma visão simplificada, muitos outros processos passam a existir a partir dele, formando uma árvore de processos.

Diferença entre processos de usuário e processos do sistema

A diferença entre processos de usuário e processos do sistema não está apenas no nome do programa, mas principalmente em sua função e no nível de permissão com que são executados.

Aspecto	Processos de usuário	Processos do sistema
Origem	Iniciados pelo usuário ou por aplicações do usuário	Iniciados pelo sistema operacional ou por serviços essenciais
Finalidade	Executar tarefas do usuário	Manter o sistema funcionando
Exemplos	Navegador, editor, terminal, servidor local	Serviços de rede, login, impressão, atualização, energia
Permissões	Normalmente limitadas ao usuário	Podem ter privilégios elevados
Visibilidade	Frequentemente possuem interface ou terminal associado	Muitas vezes executam em segundo plano
Encerramento	Pode ser encerrado pelo usuário	Encerramento indevido pode afetar o sistema

Observação

Nem todo processo em segundo plano é necessariamente um processo do sistema. Uma aplicação de usuário também pode executar em segundo plano, como um sincronizador de arquivos, um aplicativo de mensagens ou um servidor local iniciado pelo desenvolvedor.

Permissões e responsabilidade

Processos do sistema costumam ter permissões mais amplas que processos de usuário, pois precisam acessar recursos críticos, como interfaces de rede, dispositivos, arquivos de configuração, logs e áreas protegidas do sistema.

Por esse motivo, problemas nesses processos podem afetar a estabilidade, a segurança ou o desempenho do computador.

Exemplo didático

Se um serviço de rede falha, o computador pode perder conectividade. Se um serviço de impressão para de funcionar, documentos deixam de ser enviados à impressora. Se um gerenciador gráfico trava, a interface visual pode deixar de responder.

Observando processos do sistema

Em Linux, é possível observar processos do sistema com:

```
ps aux
systemctl status
top
htop
```

Também é possível visualizar uma árvore de processos com:

```
pstree
```

Em Windows, podem ser utilizadas ferramentas como o Gerenciador de Tarefas, o Monitor de Recursos, o Visualizador de Eventos e comandos como:

```
tasklist
services.msc
```

Atividade prática

Liste os processos em execução no seu sistema operacional e tente separar alguns exemplos em duas categorias:

- processos iniciados por você;
- processos ligados ao funcionamento do sistema.

Depois, escolha um processo do sistema e pesquise, de forma cuidadosa, qual função ele desempenha.

3.3.5 Estados de um processo

Durante sua existência, um processo não permanece sempre na mesma situação. Ele passa por diferentes estados, dependendo do que está fazendo, dos recursos disponíveis e das decisões tomadas pelo sistema operacional.

Esses estados ajudam o sistema operacional a organizar a execução dos programas e a decidir quais processos devem usar o processador em cada momento.

Observação

Um processo não fica necessariamente executando o tempo todo. Muitas vezes, ele está aguardando entrada do usuário, resposta da rede, leitura de arquivo, liberação de recurso ou sua vez de usar a CPU.

Estado novo

O estado novo ocorre quando um processo está sendo criado. Nesse momento, o sistema operacional prepara as estruturas necessárias para sua execução, como identificador do processo, espaço de memória, informações de controle e recursos iniciais.

Exemplo didático

Quando o usuário executa:

```
python3 programa.py
```

o sistema operacional inicia a criação de um novo processo para executar o interpretador Python com o arquivo indicado.

Estado pronto

Um processo está no estado pronto quando já possui as condições necessárias para executar, mas ainda não está usando o processador. Ele aguarda sua vez na fila de processos prontos.

O processo está preparado, mas depende do escalonador para receber tempo de CPU.

Exemplo didático

Imagine uma sala de atendimento. Várias pessoas já chegaram, pegaram senha e estão prontas para serem atendidas, mas apenas uma pode ser chamada por vez. De modo semelhante, vários processos podem estar prontos, mas apenas alguns estarão efetivamente usando a CPU em determinado instante.

Estado em execução

Um processo está em execução quando suas instruções estão sendo processadas pela CPU.

Em computadores com múltiplos núcleos, mais de um processo pode estar em execução ao mesmo tempo, desde que haja núcleos disponíveis. Ainda assim, normalmente há mais

processos no sistema do que núcleos físicos, o que exige alternância constante.

Exemplo didático

Quando uma aplicação está calculando algo, renderizando uma página, compactando arquivos ou processando dados, ela pode estar em estado de execução enquanto utiliza o processador.

Estado bloqueado ou em espera

Um processo está bloqueado, ou em espera, quando não pode continuar sua execução até que algum evento aconteça. Ele pode estar aguardando uma entrada do usuário, a leitura de um arquivo, uma resposta da rede, a liberação de um dispositivo ou o término de outra operação.

Exemplo didático

Um navegador pode ficar aguardando a resposta de um servidor web. Durante esse tempo, o processo não precisa usar intensamente a CPU. Ele está esperando a chegada dos dados pela rede.

Outro exemplo ocorre quando um programa solicita ao usuário que digite algo:

```
nome = input("Digite seu nome: ")
```

Enquanto o usuário não digita a resposta, o processo pode ficar em estado de espera.

Estado finalizado

O estado finalizado ocorre quando o processo terminou sua execução ou foi encerrado. Isso pode acontecer porque o programa concluiu sua tarefa, porque o usuário fechou a aplicação ou porque o sistema operacional interrompeu sua execução.

Exemplo didático

Quando um script Python termina de executar todas as suas instruções, o processo associado a ele é finalizado. O sistema operacional então libera os recursos usados, como memória e arquivos abertos.

Transições entre estados

Um processo pode mudar de estado várias vezes durante sua vida. Uma sequência simplificada pode ser descrita assim:

1. o processo é criado e entra no estado novo;

2. após ser preparado, passa para o estado pronto;
3. quando o escalonador o escolhe, passa para execução;
4. se precisar aguardar algum evento, passa para bloqueado;
5. quando o evento ocorre, volta para pronto;
6. quando recebe novamente a CPU, volta para execução;
7. ao terminar, passa para finalizado.

Estados e desempenho do sistema

Compreender os estados de um processo ajuda a interpretar o desempenho de um computador. Nem todo programa aberto está consumindo CPU o tempo todo. Alguns processos passam boa parte do tempo aguardando eventos.

Um sistema pode ficar lento por diferentes razões:

- muitos processos prontos disputando CPU;
- processos consumindo muita memória;
- processos bloqueados aguardando disco lento;
- aplicações esperando resposta da rede;
- processos travados ou em loop infinito;
- excesso de operações de entrada e saída.

Exemplo didático

Se uma aplicação web parece lenta, o problema pode não estar no processador. Ela pode estar bloqueada aguardando uma consulta ao banco de dados, uma resposta de API externa ou a leitura de arquivos no disco.

Atividade prática

Atividade prática

Execute um programa que permaneça aberto por algum tempo, como um servidor local em Python:

```
python3 -m http.server 8000
```

Depois, observe os processos do sistema usando `top`, `htop`, Gerenciador de Tarefas ou ferramenta equivalente.
Responda:

1. O processo aparece na lista de processos?
2. Ele consome CPU continuamente ou apenas em alguns momentos?
3. O que acontece quando você acessa `http://localhost:8000` no navegador?
4. O que acontece quando você encerra o servidor com `Ctrl+C`?
5. Que estados esse processo pode ter assumido durante a atividade?

3.4 Divisão de tempo do processador

Um dos papéis mais importantes do sistema operacional é organizar o uso do processador entre os diversos processos em execução. Em um computador moderno, é comum que dezenas ou centenas de processos estejam ativos ao mesmo tempo: navegador, editor de texto, terminal, serviços de rede, atualizações, antivírus, aplicativos em segundo plano e processos do próprio sistema.

No entanto, o processador é um recurso limitado. Mesmo em computadores com vários núcleos, normalmente há mais processos querendo executar do que unidades de processamento disponíveis. Por isso, o sistema operacional precisa decidir qual processo usará a CPU em cada momento.

Essa organização é chamada de **divisão de tempo do processador**.

Observação

Quando vários programas parecem funcionar ao mesmo tempo, nem sempre significa que todos estão usando a CPU exatamente no mesmo instante. Em muitos casos, o sistema operacional alterna rapidamente entre eles, criando a sensação de simultaneidade.

3.4.1 A ilusão de simultaneidade

Para o usuário, parece natural ouvir música, navegar na internet, baixar arquivos, editar um documento e receber notificações ao mesmo tempo. Essa experiência só é possível porque o sistema operacional divide o tempo do processador entre diferentes processos.

Essa divisão ocorre em intervalos muito pequenos de tempo. Um processo usa a CPU por alguns milissegundos, depois outro processo é escolhido, depois outro, e assim sucessivamente. Como essa troca é muito rápida, o usuário percebe o sistema como se tudo estivesse acontecendo ao mesmo tempo.

Exemplo didático

Imagine um professor atendendo vários estudantes em uma sala. Ele escuta uma dúvida, responde uma parte, passa para outro estudante, depois volta ao primeiro. Se ele alternar rapidamente e de forma organizada, todos têm a impressão de que estão sendo atendidos. O processador funciona de modo semelhante quando precisa lidar com vários processos.

3.4.2 Tempo de CPU

O tempo de CPU representa o período durante o qual um processo realmente usa o processador para executar instruções. Nem todo processo aberto está usando CPU o tempo todo. Muitos processos passam boa parte do tempo aguardando eventos, como entrada do usuário, leitura de disco ou resposta da rede.

Por exemplo, um editor de texto pode ficar aberto durante horas, mas só usar CPU em momentos específicos: ao abrir um arquivo, ao salvar, ao corrigir texto, ao renderizar a interface ou ao responder a uma ação do usuário.

Exemplo didático

Um navegador pode ter várias abas abertas. Algumas estão apenas aguardando interação. Outras podem estar reproduzindo vídeo, executando JavaScript ou fazendo requisições pela rede. O sistema operacional precisa avaliar quais processos ou threads precisam de CPU em cada instante.

3.4.3 Escalonamento de processos

O componente do sistema operacional responsável por decidir qual processo usará a CPU é chamado de **escalonador**. O escalonador organiza a fila de processos prontos e escolhe qual deles será executado.

Essa escolha pode considerar diferentes critérios, como:

- prioridade do processo;

- tempo que o processo já esperou;
- tipo de tarefa;
- necessidade de resposta rápida;
- uso anterior da CPU;
- estado do processo;
- quantidade de núcleos disponíveis.

Observação

O escalonador não escolhe processos de forma aleatória. Ele segue regras definidas pelo sistema operacional, buscando equilibrar desempenho, responsividade, justiça entre processos e uso eficiente do hardware.

3.4.4 Quantum de tempo

Em muitos sistemas, cada processo recebe uma pequena fatia de tempo para usar o processador. Essa fatia é chamada de **quantum** ou **time slice**.

Quando o quantum termina, o sistema operacional pode interromper o processo em execução e colocar outro processo para usar a CPU. O processo interrompido volta para a fila de prontos, aguardando uma nova oportunidade.

Exemplo didático

Suponha que três processos estejam prontos para executar:

- Processo A: navegador;
- Processo B: editor de código;
- Processo C: tocador de música.

O sistema operacional pode dar uma pequena fatia de tempo ao Processo A, depois ao Processo B, depois ao Processo C, e então repetir o ciclo. Essa alternância rápida permite que os programas continuem respondendo ao usuário.

3.4.5 Troca de contexto

Quando o sistema operacional interrompe um processo e passa a executar outro, ocorre uma **troca de contexto**.

O contexto de um processo inclui informações necessárias para que ele possa continuar sua execução posteriormente, como:

- valores dos registradores;
- posição atual da execução;
- informações de memória;
- estado do processo;
- dados de controle mantidos pelo sistema operacional.

Antes de trocar de processo, o sistema operacional salva o contexto do processo atual. Depois, carrega o contexto do próximo processo. Assim, quando um processo volta a executar, ele continua aproximadamente do ponto onde havia parado.

Exemplo didático

Imagine que você está lendo três livros ao mesmo tempo. Para alternar entre eles, você coloca um marcador na página atual de um livro, fecha-o, abre outro livro no ponto marcado e continua a leitura. A troca de contexto funciona de forma parecida: o sistema salva onde o processo estava e depois retoma sua execução a partir daquele ponto.

Observação

A troca de contexto é necessária, mas tem custo. Enquanto o sistema está salvando e restaurando contextos, ele não está executando diretamente as instruções úteis dos programas. Por isso, muitas trocas de contexto podem prejudicar o desempenho.

3.4.6 Processos interativos e processos em lote

Nem todos os processos têm as mesmas necessidades. Alguns precisam responder rapidamente ao usuário. Outros podem executar por mais tempo sem interação direta.

Um **processo interativo** é aquele que depende de respostas rápidas para manter uma boa experiência de uso. Exemplos:

- editor de texto;
- navegador;
- terminal;
- interface gráfica;
- aplicação de mensagens;
- jogo digital.

Um **processo em lote** ou de processamento intensivo pode executar por mais tempo sem exigir resposta imediata do usuário. Exemplos:

- compactação de arquivos;
- renderização de vídeo;
- treinamento de modelo de machine learning;
- processamento de grande volume de dados;
- geração de relatórios;
- backup automático.

Exemplo didático

Se um sistema está renderizando um vídeo, mas o usuário move o mouse ou digita em um editor, o sistema operacional pode priorizar temporariamente os processos interativos para manter a sensação de fluidez.

3.4.7 Prioridades de processos

Processos podem ter diferentes prioridades. Um processo com prioridade mais alta tende a receber mais atenção do escalonador. Isso não significa necessariamente que ele usará a CPU o tempo todo, mas que terá maior chance de ser escolhido quando estiver pronto para executar.

Em sistemas operacionais, prioridades ajudam a garantir que tarefas importantes sejam atendidas rapidamente.

Exemplo didático

Um processo responsável pela reprodução de áudio pode precisar de prioridade adequada. Se ele ficar muito tempo sem CPU, o som pode falhar, mesmo que o computador ainda tenha capacidade de processamento.

Em sistemas Linux, o comando `nice` pode ser usado para iniciar um processo com determinada prioridade. O comando `renice` pode alterar a prioridade de um processo já em execução.

```
nice -n 10 python3 tarefa_pesada.py
renice 5 -p 1234
```

Observação

Alterar prioridades deve ser feito com cuidado. Aumentar a prioridade de um processo pode melhorar seu desempenho, mas também pode prejudicar a responsividade de outros processos.

3.4.8 Concorrência e paralelismo

Dois conceitos importantes relacionados à divisão de tempo do processador são **concorrência** e **paralelismo**.

A **concorrência** ocorre quando múltiplas tarefas estão em andamento no mesmo período de tempo, mesmo que não estejam executando exatamente ao mesmo tempo. O sistema alterna entre elas.

O **paralelismo** ocorre quando múltiplas tarefas são executadas literalmente ao mesmo tempo, em diferentes núcleos ou processadores.

Exemplo didático

Em um computador com apenas um núcleo, dois processos podem ser concorrentes: o sistema alterna rapidamente entre eles.
Em um computador com quatro núcleos, quatro processos podem executar em paralelo, cada um em um núcleo, enquanto outros processos aguardam sua vez.

Conceito	**Descrição**
Concorrência	Várias tarefas progridem no mesmo intervalo de tempo, com alternância entre elas.
Paralelismo	Várias tarefas executam literalmente ao mesmo tempo, em núcleos diferentes.
Troca de contexto	Salvamento do estado de um processo e carregamento do estado de outro.
Quantum	Pequena fatia de tempo concedida a um processo para usar a CPU.
Escalonador	Parte do sistema operacional que escolhe qual processo será executado.

3.4.9 Processos limitados por CPU e por entrada/saída

Os processos também podem ser classificados de acordo com o tipo de recurso que mais utilizam.

Um processo **limitado por CPU** é aquele que passa a maior parte do tempo realizando cálculos. Ele depende fortemente do processador.

Exemplos:

- compactação de arquivos;
- criptografia;
- renderização;
- simulações;
- treinamento de modelos;
- processamento de imagens.

Um processo **limitado por entrada/saída** é aquele que passa boa parte do tempo aguardando disco, rede, teclado, banco de dados ou outro dispositivo.

Exemplos:

- servidor web aguardando requisições;
- aplicação lendo arquivos grandes;
- cliente de banco de dados esperando resposta;
- programa aguardando entrada do usuário;
- download de arquivos pela internet.

Exemplo didático

Uma aplicação web pode parecer lenta não porque o processador está sobrecarregado, mas porque ela está aguardando uma consulta ao banco de dados. Nesse caso, o gargalo está mais próximo da entrada e saída do que da CPU.

3.4.10 Divisão de tempo em sistemas com múltiplos núcleos

Computadores modernos frequentemente possuem processadores com múltiplos núcleos. Cada núcleo pode executar instruções de forma independente. Isso aumenta a capacidade de paralelismo do sistema.

Mesmo assim, a divisão de tempo continua necessária. Um computador com 8 núcleos pode ter centenas de processos e milhares de threads. O sistema operacional ainda precisa decidir quais tarefas serão atribuídas a quais núcleos e por quanto tempo.

Observação

Ter mais núcleos não significa que qualquer programa ficará automaticamente mais rápido. Para aproveitar bem múltiplos núcleos, uma aplicação precisa ser projetada para dividir seu trabalho em tarefas que possam executar em paralelo.

3.4.11 Threads e divisão de tempo

Além de processos, muitos sistemas operacionais trabalham com **threads**. Uma thread é uma linha de execução dentro de um processo. Um processo pode ter uma ou várias threads.

Threads permitem que um mesmo programa realize várias tarefas aparentemente ao mesmo tempo. Por exemplo, um navegador pode usar threads diferentes para interface, rede, renderização de páginas e execução de scripts.

Exemplo didático

Em um editor de texto moderno, uma thread pode cuidar da interface gráfica, enquanto outra verifica a ortografia e outra salva automaticamente o arquivo em segundo plano.

O escalonador do sistema operacional pode alternar entre threads, assim como alterna entre processos.

3.4.12 Responsividade do sistema

Um bom escalonamento busca manter o sistema responsivo. Responsividade é a capacidade do sistema de reagir rapidamente às ações do usuário.

Mesmo que uma tarefa pesada esteja em execução, o usuário espera que o mouse continue se movendo, que o teclado responda, que a interface não congele e que os sons não travem. Para isso, o sistema operacional deve evitar que uma única aplicação monopolize completamente a CPU.

Exemplo didático

Se um programa entra em loop infinito e começa a consumir 100% da CPU, o sistema pode ficar lento. O escalonador ainda tentará dividir o tempo entre os demais processos, mas a experiência do usuário pode ser afetada.

3.4.13 Observando o uso da CPU

Podemos observar a divisão de tempo do processador usando ferramentas do sistema operacional.

Em Linux:

```
top
htop
ps aux
uptime
```

Em Windows:

```
taskmgr
tasklist
wmic cpu get loadpercentage
```

No Linux, o comando `top` mostra processos em execução e o consumo de CPU. O `htop`, quando instalado, apresenta uma visualização mais interativa.

Exemplo didático

Um resultado simplificado do comando `top` poderia mostrar:

```
PID   USER   %CPU   %MEM   COMMAND
2134  aluno  85.0    2.1   python3
1450  aluno  12.5    8.4   firefox
980   root    1.2    0.5   systemd
```

Nesse exemplo, o processo `python3` está consumindo grande parte do tempo de CPU.

3.4.14 Exemplo prático com Python

Podemos criar um programa simples que consome CPU continuamente. Esse tipo de programa ajuda a observar como o sistema operacional distribui tempo de processamento.

Exemplo didático

Arquivo `cpu.py`:

```
while True:
pass
```

Ao executar esse programa, ele entra em um loop infinito. Em outro terminal, é possível observar o consumo de CPU com:

```
top
```

ou:

```
htop
```

Para interromper o programa, use `Ctrl+C` no terminal em que ele foi iniciado.

Observação

Esse exemplo deve ser executado com cuidado e apenas em ambiente de aprendizagem. Ele não danifica o computador, mas pode deixar o sistema mais lento enquanto estiver em execução, pois consome tempo de CPU continuamente.

3.4.15 Exemplo prático: processo limitado por entrada

O exemplo anterior consome CPU intensamente. Agora considere um programa que aguarda entrada do usuário:

Exemplo didático

Arquivo `entrada.py`:

```
nome = input("Digite seu nome: ")
print("Ol  ,", nome)
```

Enquanto o programa aguarda a digitação, ele não precisa consumir CPU de forma intensa. Ele fica bloqueado esperando um evento de entrada.

Esse contraste mostra que nem todo processo aberto representa uso contínuo do processador.

3.4.16 Atividade prática

Atividade prática

Execute os dois programas abaixo em momentos diferentes e observe o uso da CPU com `top`, `htop` ou Gerenciador de Tarefas.

Programa 1: uso intensivo de CPU

```
while True:
pass
```

Programa 2: espera por entrada

```
nome = input("Digite seu nome: ")
print("Ol  ,", nome)
```

Depois, responda:

1. Qual programa consumiu mais CPU?
2. Qual programa ficou mais tempo aguardando?
3. O que isso revela sobre processos limitados por CPU e por entrada/saída?
4. Como o sistema operacional lidou com os demais processos enquanto esses programas estavam abertos?

3.5 Arquivos e diretórios

Uma das funções mais visíveis de um sistema operacional é organizar informações em arquivos e diretórios. Sempre que salvamos um documento, baixamos uma imagem, instalamos um programa, criamos um projeto de software ou registramos configurações do sistema, estamos utilizando o sistema de arquivos.

Arquivos e diretórios formam a base da organização dos dados em um computador. Para o usuário, eles aparecem como pastas, documentos, imagens, vídeos, programas e atalhos. Para o sistema operacional, eles são estruturas organizadas em dispositivos de armazenamento, como HDs, SSDs, pendrives, cartões de memória ou sistemas remotos acessados pela rede.

Observação

O sistema de arquivos é uma abstração. Ele permite que usuários e programas trabalhem com nomes, pastas e caminhos, sem precisar lidar diretamente com setores físicos do disco ou detalhes eletrônicos do dispositivo de armazenamento.

3.5.1 O que é um arquivo?

Um arquivo é uma unidade de armazenamento de dados. Ele pode conter texto, imagem, áudio, vídeo, código-fonte, dados estruturados, configurações, programas executáveis ou qualquer outro tipo de informação.

Do ponto de vista do sistema operacional, um arquivo é uma sequência de bytes armazenada em algum dispositivo. O significado desses bytes depende do formato do arquivo e do programa utilizado para interpretá-lo.

Exemplo didático

Um arquivo chamado `relatorio.txt` pode conter texto simples. Um arquivo chamado `foto.jpg` contém dados codificados no formato JPEG. Um arquivo chamado `app.py` contém código-fonte em Python. Um arquivo chamado `programa.exe` pode conter instruções executáveis para o sistema operacional.

Um arquivo geralmente possui algumas informações associadas, chamadas de metadados. Entre elas, podemos citar:

- nome do arquivo;
- tamanho;
- tipo ou extensão;
- data de criação;
- data de modificação;
- permissões de acesso;
- proprietário;
- localização no sistema de arquivos.

3.5.2 Nome e extensão de arquivos

O nome de um arquivo permite identificá-lo dentro de um diretório. Muitas vezes, o nome é acompanhado por uma extensão, que aparece após um ponto e indica, de forma convencional, o tipo ou formato do arquivo.

Exemplo didático

No arquivo:

```
relatorio.pdf
```

`relatorio` é o nome principal e `.pdf` é a extensão.

No arquivo:

```
app.py
```

`app` é o nome principal e `.py` indica que se trata, provavelmente, de um arquivo de código Python.

Algumas extensões comuns são:

Extensão	**Uso comum**
`.txt`	Arquivo de texto simples.
`.pdf`	Documento em formato PDF.
`.docx`	Documento de texto do Microsoft Word ou compatível.
`.odt`	Documento de texto do LibreOffice ou OpenDocument.
`.jpg` ou `.jpeg`	Imagem compactada em formato JPEG.
`.png`	Imagem em formato PNG.
`.mp3`	Arquivo de áudio compactado.
`.mp4`	Arquivo de vídeo.
`.csv`	Arquivo de dados tabulares separados por vírgula ou outro separador.
`.json`	Arquivo de dados estruturados em JavaScript Object Notation.
`.html`	Página web em HTML.
`.css`	Folha de estilos para páginas web.
`.js`	Código JavaScript.
`.py`	Código-fonte Python.
`.java`	Código-fonte Java.
`.c`	Código-fonte em linguagem C.
`.sh`	Script de shell em sistemas Unix/Linux.
`.exe`	Arquivo executável em Windows.

Observação

A extensão ajuda o sistema operacional e os programas a identificar como abrir um arquivo, mas ela não garante, sozinha, o conteúdo real. Um arquivo pode ter a extensão alterada manualmente sem que seu conteúdo seja convertido de fato.

3.5.3 Tipo de arquivo e formato

É importante distinguir extensão, tipo e formato de arquivo.

A **extensão** é parte do nome do arquivo. O **formato** define como os dados estão organizados internamente. O **tipo** indica a categoria geral do arquivo, como texto, imagem, áudio, vídeo, código ou executável.

Exemplo didático

Um arquivo `foto.jpg` tem extensão `.jpg` e provavelmente usa o formato JPEG. Se o usuário renomear esse arquivo para `foto.txt`, ele não se tornará um arquivo de texto. O conteúdo interno continuará sendo uma imagem codificada em JPEG.

Em sistemas Linux, podemos usar o comando `file` para tentar identificar o tipo real de um arquivo:

```
file foto.jpg
file app.py
file documento.pdf
```

3.5.4 Tamanho de arquivos

Todo arquivo ocupa espaço em armazenamento. Esse tamanho é medido em bytes e em seus múltiplos.

Unidade	Significado aproximado
bit	Menor unidade de informação, podendo representar 0 ou 1.
byte	Conjunto de 8 bits.
KB	Aproximadamente mil bytes.
MB	Aproximadamente um milhão de bytes.
GB	Aproximadamente um bilhão de bytes.
TB	Aproximadamente um trilhão de bytes.

Observação

Em alguns contextos, há diferença entre unidades decimais e binárias. Por exemplo, 1 KB pode significar 1.000 bytes, enquanto 1 KiB significa 1.024 bytes. Essa diferença é importante em discussões mais técnicas sobre armazenamento e memória.

Exemplo didático

Um arquivo de texto simples com poucas linhas pode ter apenas alguns bytes ou kilobytes. Uma foto em alta resolução pode ter vários megabytes. Um vídeo longo em alta qualidade pode ocupar gigabytes.

Em Linux, podemos verificar o tamanho de arquivos com comandos como:

3.5.5 O que é um diretório?

Um diretório, também chamado de pasta, é uma estrutura usada para organizar arquivos e outros diretórios. Ele permite agrupar informações relacionadas e criar uma organização hierárquica.

Exemplo didático

Um estudante pode organizar os arquivos da disciplina assim:

```
sistemas-computacionais/
 aulas/
 exercicios/
 projetos/
 leituras/
```

Cada diretório ajuda a separar os materiais conforme sua finalidade.

Diretórios são fundamentais para evitar desorganização. Sem eles, todos os arquivos ficariam no mesmo nível, dificultando a localização e a manutenção dos dados.

3.5.6 Organização hierárquica

Os sistemas de arquivos normalmente organizam diretórios em forma de árvore. Existe um ponto inicial, chamado de raiz, a partir do qual outros diretórios e arquivos são organizados.

Em sistemas Linux e macOS, a raiz é representada por:

```
/
```

Em sistemas Windows, é comum encontrar unidades representadas por letras, como:

```
C:
D:
```

Exemplo didático

Em Linux, um caminho completo pode ser:

```
/home/aluno/documentos/aula01/anotacoes.txt
```

Em Windows, um caminho semelhante poderia ser:

```
C:\Users\aluno\Documentos\aula01\anotacoes.txt
```

3.5.7 Caminhos absolutos e relativos

Um caminho indica a localização de um arquivo ou diretório dentro do sistema de arquivos.

Um **caminho absoluto** começa a partir da raiz do sistema de arquivos. Ele indica a localização completa.

Exemplo didático

Exemplo de caminho absoluto em Linux:

```
/home/aluno/projetos/app.py
```

Exemplo de caminho absoluto em Windows:

```
C:\Users\aluno\projetos\app.py
```

Um **caminho relativo** é interpretado a partir do diretório atual. Ele depende de onde o usuário ou processo está no momento.

Exemplo didático

Se o usuário já está dentro do diretório:

```
/home/aluno/projetos/
```

então pode acessar o arquivo usando apenas:

```
app.py
```

ou:

```
./app.py
```

Alguns símbolos são muito usados em caminhos relativos:

Símbolo	**Significado**
.	Diretório atual.
..	Diretório acima, também chamado de diretório pai.
~	Diretório pessoal do usuário em sistemas Linux.
/	Separador de diretórios em Linux e macOS.
\	Separador de diretórios em Windows.

Observação

Compreender caminhos absolutos e relativos é essencial para programação. Muitos erros em projetos ocorrem porque um arquivo é referenciado a partir do diretório errado.

3.5.8 Recursão de diretórios

A organização em diretórios permite que uma pasta contenha outras pastas, que por sua vez podem conter outras pastas. Essa estrutura é chamada de organização recursiva ou em cascata.

A ideia de recursão aparece porque um diretório pode conter elementos do mesmo tipo que ele: outros diretórios.

Exemplo didático

Considere a estrutura:

```
projeto/
 src/
    controllers/
    models/
    views/
 tests/
    unitarios/
    integracao/
 docs/
 imagens/
```

O diretório `projeto` contém `src`, `tests` e `docs`. Cada um deles pode conter outros diretórios. Essa é uma estrutura recursiva.

Comandos que percorrem diretórios de forma recursiva analisam não apenas a pasta atual, mas também todas as subpastas.

```
ls -R
find . -name "*.py"
du -sh *
rm -r pasta/
```

Observação

Comandos recursivos devem ser usados com cuidado, especialmente comandos de remoção, como `rm -r`. Eles podem apagar uma pasta inteira e todos os seus conteúdos.

3.5.9 Arquivos ocultos

Em muitos sistemas operacionais, existem arquivos e diretórios ocultos. Eles geralmente armazenam configurações, preferências de aplicativos ou informações internas do sistema.

Em Linux e macOS, arquivos ocultos normalmente começam com ponto:

```
.bashrc
.gitignore
.config/
.ssh/
```

Para listar arquivos ocultos em Linux, usamos:

```
ls -a
```

ou:

```
ls -la
```

Exemplo didático

Em um projeto Git, o diretório `.git` guarda o histórico e as informações internas do repositório. Embora ele fique oculto em muitos gerenciadores de arquivos, é essencial para o funcionamento do controle de versão.

3.5.10 Permissões de arquivos e diretórios

O sistema operacional controla quem pode ler, modificar ou executar arquivos e diretórios. Esse controle é essencial em ambientes multiusuário, servidores, laboratórios e sistemas compartilhados.

Em sistemas Unix/Linux, as permissões básicas são:

Permissão	**Símbolo**	**Significado**
Leitura	`r`	Permite ler o conteúdo de um arquivo ou listar um diretório.
Escrita	`w`	Permite modificar um arquivo ou alterar o conteúdo de um diretório.
Execução	`x`	Permite executar um arquivo ou acessar um diretório.

Ao executar:

```
ls -l
```

podemos ver permissões como:

```
-rwxr-xr--
```

Essa representação indica permissões para três grupos:

- dono do arquivo;
- grupo;
- demais usuários.

Exemplo didático

No exemplo:

```
-rwxr-xr--
```

podemos interpretar:

- `rwx`: o dono pode ler, escrever e executar;
- `r-x`: o grupo pode ler e executar;
- `r--`: os demais usuários podem apenas ler.

3.5.11 Permissão de execução

Nem todo arquivo pode ser executado como programa. Em sistemas Linux, um arquivo precisa ter permissão de execução para ser executado diretamente.

Exemplo didático

Considere um script chamado `backup.sh`. Para torná-lo executável, podemos usar:

```
chmod +x backup.sh
```

Depois, podemos executá-lo com:

```
./backup.sh
```

Observação

Dar permissão de execução a um arquivo significa permitir que ele seja tratado como um programa ou script. Isso deve ser feito apenas quando se conhece a origem e a finalidade do arquivo.

3.5.12 Arquivos em projetos de software

Em desenvolvimento de software, a organização de arquivos e diretórios é fundamental. Um projeto bem organizado facilita leitura, manutenção, testes, colaboração e implantação.

Uma estrutura simples de projeto Python com Flask poderia ser:

```
meu_app/
 app.py
 requirements.txt
 README.md
 static/
    style.css
 templates/
 index.html
```

Cada arquivo tem uma função:

- `app.py`: código principal da aplicação;
- `requirements.txt`: lista de dependências;
- `README.md`: documentação inicial do projeto;
- `static/`: arquivos estáticos, como CSS, imagens e JavaScript;
- `templates/`: páginas HTML usadas pela aplicação.

Exemplo didático

Quando uma aplicação Flask procura um arquivo HTML, ela normalmente busca esse arquivo dentro do diretório `templates`. Se o arquivo estiver em outro local, a aplicação pode gerar erro, mesmo que o arquivo exista em algum lugar do projeto.

3.5.13 Arquivos temporários e arquivos de configuração

Além dos arquivos criados diretamente pelo usuário, sistemas operacionais e aplicações utilizam arquivos temporários e arquivos de configuração.

Arquivos temporários armazenam dados provisórios. Eles podem ser usados durante instalação de programas, edição de documentos, downloads, compactação ou processamento intermediário.

Arquivos de configuração guardam preferências e parâmetros de programas, como tema visual, idioma, caminhos, credenciais locais, portas de rede e opções de execução.

Exemplo didático

Em projetos de software, é comum usar arquivos como:

```
.env
config.json
settings.py
docker-compose.yml
.gitignore
```

Cada um desses arquivos pode influenciar diretamente o comportamento da aplicação ou das ferramentas de desenvolvimento.

Observação

Arquivos que contêm senhas, chaves de API ou tokens não devem ser publicados em repositórios públicos. Em projetos com Git, o arquivo `.gitignore` pode ser usado para evitar que determinados arquivos sejam versionados.

3.5.14 Operações comuns com arquivos e diretórios

Comando	Função
`pwd`	Mostra o diretório atual.
`ls`	Lista arquivos e diretórios.
`ls -l`	Lista com detalhes.
`ls -a`	Lista também arquivos ocultos.
`cd pasta`	Entra em um diretório.
`cd ..`	Volta para o diretório pai.
`mkdir pasta`	Cria um diretório.
`touch arquivo.txt`	Cria um arquivo vazio ou atualiza sua data de modificação.
`cp origem destino`	Copia arquivos ou diretórios.
`mv origem destino`	Move ou renomeia arquivos e diretórios.
`rm arquivo`	Remove um arquivo.
`rm -r pasta`	Remove uma pasta e seu conteúdo de forma recursiva.
`cat arquivo`	Exibe o conteúdo de um arquivo de texto.
`less arquivo`	Permite visualizar um arquivo de texto página por página.
`find`	Procura arquivos e diretórios.

3.5.15 Exemplo prático: criando uma estrutura de projeto

Atividade prática

Abra o terminal e execute os comandos abaixo para criar uma estrutura simples de projeto:

```
mkdir sistema-arquivos
cd sistema-arquivos

mkdir src docs tests
touch README.md
touch src/app.py
touch docs/anotacoes.md
touch tests/test_app.py

ls -R
```

Depois, responda:

1. Qual comando criou o diretório principal?
2. Qual comando criou os subdiretórios?
3. Qual comando criou arquivos vazios?
4. O que o comando `ls -R` mostrou?
5. Como essa organização poderia ajudar em um projeto real?

3.5.16 Exemplo prático: caminhos relativos

Atividade prática

Considerando a estrutura criada na atividade anterior, entre no diretório `src`:

```
cd src
pwd
```

Agora tente listar o conteúdo do diretório `docs` usando um caminho relativo:

```
ls ../docs
```

Depois, responda:

1. O que significa `..` nesse comando?
2. Por que o caminho `../docs` funciona a partir de `src`?
3. Como esse conceito aparece em projetos de software?

3.5.17 Relação com o desenvolvimento de software

Para o desenvolvedor, arquivos e diretórios não são apenas uma forma de organizar documentos. Eles fazem parte da estrutura lógica e operacional de um sistema.

Um erro de caminho pode impedir uma aplicação de encontrar um template. Uma permissão incorreta pode impedir a execução de um script. Um arquivo de configuração ausente pode impedir a conexão com um banco de dados. Um arquivo sensível publicado por engano pode comprometer a segurança do sistema.

Por isso, compreender arquivos e diretórios ajuda o estudante a lidar com problemas comuns no desenvolvimento, como:

- erro de arquivo não encontrado;
- importações incorretas;
- permissões insuficientes;
- scripts sem permissão de execução;
- dependências mal organizadas;
- configurações expostas indevidamente;
- dificuldade de colaboração em projetos desorganizados.

Exemplo didático

Uma aplicação pode tentar abrir um arquivo chamado:

```
dados/usuarios.csv
```

Se o programa for executado a partir de outro diretório, esse caminho relativo pode deixar de funcionar. Nesse caso, compreender o diretório atual do processo é essencial para corrigir o problema.

3.6 Comandos úteis para observar o sistema

3.6.1 Linux

```
ps aux
top
htop
df -h
free -h
lsblk
uname -a
```

3.6.2 Windows

```
tasklist
systeminfo
ipconfig
dir
tree
```

Atividade prática

Execute comandos para observar os processos em execução no seu sistema. Identifique:

- um processo relacionado ao navegador;
- um processo relacionado ao sistema operacional;
- o consumo de memória de algum processo;
- o identificador do processo, também conhecido como PID.

Exercícios

1. Um programa armazenado no disco não consome CPU enquanto não está em execução. Já um processo pode consumir CPU, memória, arquivos e recursos de rede. A partir dessa diferença, explique por que não basta dizer que "um processo é apenas um programa aberto".

2. Imagine que você abriu um navegador, um editor de texto, um terminal e um tocador de música ao mesmo tempo. Como o sistema operacional organiza a execução desses programas para que o usuário tenha a impressão de que todos funcionam simultaneamente?

3. O escalonador precisa decidir qual processo usará a CPU em cada momento. Que problemas poderiam acontecer se um único processo pudesse ocupar o processador por tempo indeterminado?

4. A memória RAM é limitada e compartilhada por vários processos. O que poderia acontecer se cada processo pudesse acessar livremente a memória dos demais?

5. Ao tentar executar um script no Linux, um usuário recebe a mensagem `Permission denied`. Que hipóteses poderiam explicar esse erro? Como o conceito de permissão de execução ajuda a entender o problema?

6. Compare processos de usuário e processos do sistema operacional. Por que encerrar um processo de usuário geralmente é menos arriscado do que encerrar um processo essencial do sistema?

7. Observe um computador em uso cotidiano. Quais processos provavelmente foram iniciados diretamente pelo usuário? Quais podem estar funcionando em segundo plano para manter o sistema operacional ativo?

8. Imagine que uma aplicação web local não inicia porque a porta desejada já está em uso. Como o conhecimento sobre processos pode ajudar o desenvolvedor a investigar e resolver esse problema?

9. Explique como os conceitos de processo, memória, permissões e arquivos aparecem juntos quando um desenvolvedor executa uma aplicação pelo terminal.

10. Por que compreender processos do sistema operacional é importante mesmo para quem pretende trabalhar principalmente com desenvolvimento de aplicações?

Conteúdos extras recomendados

Filmes, séries e documentários

- *Halt and Catch Fire* — série sobre a evolução da computação pessoal e da internet.
- *Revolution OS* — documentário sobre software livre e sistemas operacionais.

Capítulo 4

Máquinas Virtuais e Contêineres

Objetivos de aprendizagem

Ao final deste capítulo, você deverá ser capaz de:

- compreender o conceito de virtualização;
- diferenciar máquina física, máquina virtual e contêiner;
- identificar ferramentas de virtualização;
- compreender o papel dos contêineres no desenvolvimento de software;
- executar comandos básicos com Docker.

4.1 O que é virtualização

Virtualização é a técnica que permite simular recursos computacionais, como processadores, memória, discos e redes, criando ambientes isolados dentro de uma máquina física.

4.2 Hipervisores

Para compreender o funcionamento das máquinas virtuais, é necessário entender o papel dos **hipervisores**. Um hipervisor é uma camada de software, firmware ou sistema especializada em criar, executar e gerenciar máquinas virtuais.

Uma máquina virtual simula um computador completo. Ela pode possuir processador virtual, memória, disco, placa de rede e outros dispositivos virtuais. No entanto, esses recursos não existem de forma independente: eles são fornecidos a partir dos recursos reais da máquina física.

O hipervisor é o responsável por fazer essa mediação entre o hardware real e as máquinas virtuais.

Observação

Uma máquina virtual não "possui" fisicamente um processador, uma memória RAM ou um disco próprio. Ela utiliza recursos virtuais que são mapeados, controlados e isolados pelo hipervisor.

4.2.1 O que o hipervisor faz?

O hipervisor permite que vários sistemas operacionais sejam executados sobre uma mesma máquina física. Cada sistema operacional convidado (*guest*) acredita estar utilizando um computador próprio, mas, na realidade, está compartilhando os recursos da máquina hospedeira (*host*).

Entre as principais funções de um hipervisor, estão:

- criar e executar máquinas virtuais;
- distribuir CPU, memória e armazenamento entre as VMs;
- isolar uma máquina virtual das demais;
- controlar o acesso das VMs ao hardware real;
- virtualizar dispositivos, como disco, rede, vídeo e USB;
- permitir snapshots, clonagem e migração de máquinas virtuais;
- monitorar o consumo de recursos.

Exemplo didático

Imagine um servidor físico com bastante memória, processador e armazenamento. Com um hipervisor, esse servidor pode executar várias máquinas virtuais: uma com Linux para servidor web, outra com Windows Server para uma aplicação administrativa, outra com banco de dados e outra para testes de desenvolvimento.

4.2.2 Sistema hospedeiro e sistema convidado

Na virtualização, é comum usar os termos **hospedeiro** e **convidado**.

O **sistema hospedeiro**, ou *host*, é o sistema físico ou principal que fornece os recursos. O **sistema convidado**, ou *guest*, é o sistema operacional executado dentro da máquina virtual.

Exemplo didático

Se um estudante usa um notebook com Debian instalado e, dentro dele, executa uma máquina virtual com Windows, o Debian é o sistema hospedeiro e o Windows é o sistema convidado.
Se, em outro caso, um servidor físico executa diretamente um hipervisor como VMware ESXi, Proxmox VE ou Hyper-V Server, esse hipervisor atua como a camada principal sobre o hardware.

4.2.3 Isolamento entre máquinas virtuais

Uma característica importante da virtualização é o isolamento. Cada máquina virtual funciona como se fosse um computador separado. Em condições normais, uma VM não deve acessar diretamente a memória, os arquivos ou os processos de outra VM.

Esse isolamento permite executar ambientes diferentes em uma mesma máquina física, reduzindo riscos de interferência.

Exemplo didático

Um professor pode criar uma máquina virtual para testes de segurança, outra para desenvolvimento web e outra para banco de dados. Se uma delas for configurada incorretamente ou sofrer algum problema, as demais podem continuar funcionando.

Observação

O isolamento oferecido por máquinas virtuais é uma das razões pelas quais elas são utilizadas em servidores, laboratórios, computação em nuvem e ambientes de teste. Ainda assim, a segurança depende também da correta configuração do hipervisor, da rede, das permissões e das atualizações.

4.2.4 Virtualização de recursos

O hipervisor precisa apresentar recursos virtuais ao sistema convidado. Esses recursos são abstrações dos recursos físicos.

Recurso virtual	**Relação com o recurso físico**
CPU virtual	Representa uma parcela do tempo de processamento da CPU física.
Memória virtual da VM	Corresponde a uma quantidade de RAM reservada ou gerenciada pelo hipervisor.
Disco virtual	Normalmente é armazenado como um arquivo ou volume no armazenamento real.

Placa de rede virtual	Permite que a VM se comunique com redes internas, externas ou com o host.
Dispositivos USB virtuais	Podem representar dispositivos reais repassados à VM ou dispositivos simulados.
Vídeo virtual	Permite que o sistema convidado exiba interface gráfica por meio de adaptadores simulados.

Exemplo didático

Ao criar uma VM no VirtualBox, o usuário pode definir 4 GB de RAM, 2 CPUs virtuais e um disco virtual de 30 GB. Esses valores não criam hardware novo; eles definem como o hipervisor apresentará parte dos recursos da máquina física ao sistema convidado.

4.2.5 Tipos de hipervisores

Os hipervisores são geralmente classificados em dois tipos principais:

- hipervisor tipo 1;
- hipervisor tipo 2.

A diferença principal está em onde o hipervisor é executado em relação ao hardware e ao sistema operacional hospedeiro.

4.2.6 Hipervisor tipo 1

O hipervisor tipo 1 é executado diretamente sobre o hardware físico da máquina. Por isso, ele também é chamado de hipervisor *bare-metal*, expressão que pode ser traduzida como "metal nu", indicando que não há um sistema operacional convencional entre o hipervisor e o hardware.

Nesse modelo, o hipervisor assume o papel principal de controlar os recursos físicos da máquina e disponibilizá-los para as máquinas virtuais.

Características do hipervisor tipo 1

O hipervisor tipo 1 costuma ser usado em ambientes profissionais, servidores, datacenters e infraestrutura de nuvem. Ele é projetado para oferecer alto desempenho, estabilidade e melhor controle dos recursos.

Entre suas características, podemos destacar:

- execução direta sobre o hardware;

- maior desempenho em comparação com muitos hipervisores tipo 2;
- melhor isolamento entre máquinas virtuais;
- uso comum em servidores e datacenters;
- gerenciamento avançado de recursos;
- suporte a alta disponibilidade, migração e snapshots, dependendo da plataforma;
- administração geralmente feita por interface web, console ou ferramentas especializadas.

Exemplo didático

Em um datacenter, uma empresa pode possuir um servidor físico executando um hipervisor tipo 1. Sobre esse servidor, são criadas várias máquinas virtuais para diferentes finalidades: servidor web, banco de dados, ambiente de testes, sistema interno e serviços de rede.

Exemplos de hipervisores tipo 1

Alguns exemplos de hipervisores tipo 1 são:

- VMware ESXi;
- Microsoft Hyper-V, quando executado como função de virtualização no servidor;
- KVM, integrado ao kernel Linux;
- Xen;
- Proxmox VE, baseado em KVM e ferramentas de gerenciamento;
- Oracle VM Server.

Observação

A classificação de algumas tecnologias pode variar conforme o contexto. Por exemplo, o KVM é integrado ao kernel Linux, e o próprio Linux passa a atuar como hipervisor. Por isso, em materiais didáticos, ele costuma ser apresentado como uma tecnologia de virtualização de tipo 1.

Vantagens do hipervisor tipo 1

Entre as principais vantagens do hipervisor tipo 1, estão:

- melhor desempenho, por estar mais próximo do hardware;
- maior estabilidade para ambientes de produção;
- melhor aproveitamento dos recursos físicos;
- maior isolamento entre ambientes;
- recursos avançados de administração;
- adequado para servidores e computação em nuvem.

Limitações do hipervisor tipo 1

Apesar das vantagens, o hipervisor tipo 1 também possui limitações:

- pode exigir hardware compatível;
- pode ter instalação e configuração mais complexas;
- nem sempre é ideal para uso didático em computadores pessoais;
- pode exigir conhecimento de redes, armazenamento e segurança;
- algumas soluções comerciais podem ter custos de licença.

Exemplo didático

Para um estudante que deseja apenas testar uma distribuição Linux dentro do próprio notebook, instalar um hipervisor tipo 1 diretamente no hardware pode ser desnecessário. Nesse caso, um hipervisor tipo 2, como VirtualBox ou VMware Workstation, pode ser mais simples.

4.2.7 Hipervisor tipo 2

O hipervisor tipo 2 é executado sobre um sistema operacional hospedeiro já instalado. Nesse modelo, o usuário primeiro utiliza um sistema operacional comum, como Windows, Linux ou macOS. Dentro dele, instala um software de virtualização, que permite criar e executar máquinas virtuais.

Esse tipo de hipervisor é muito comum em computadores pessoais, laboratórios didáticos, ambientes de desenvolvimento e testes locais.

Características do hipervisor tipo 2

O hipervisor tipo 2 é mais simples de instalar e utilizar em ambientes de aprendizagem. Como ele funciona como um programa dentro do sistema operacional hospedeiro, o usuário pode criar e remover máquinas virtuais sem alterar profundamente a instalação principal do computador.

Entre suas características, estão:

- execução sobre um sistema operacional hospedeiro;
- instalação semelhante à de um aplicativo comum;
- uso frequente em computadores pessoais;
- boa opção para estudo, testes e desenvolvimento;
- menor complexidade inicial;
- possível perda de desempenho em relação ao tipo 1;
- dependência do sistema operacional hospedeiro.

Exemplo didático

Um estudante que usa Windows pode instalar o VirtualBox e criar uma máquina virtual com Ubuntu para praticar comandos Linux, testar servidores locais, estudar permissões de arquivos e executar ferramentas de desenvolvimento.

Exemplos de hipervisores tipo 2

Alguns exemplos de hipervisores tipo 2 são:

- Oracle VirtualBox;
- VMware Workstation;
- VMware Fusion;
- Parallels Desktop;
- UTM, em alguns cenários de uso;
- QEMU com interface gráfica ou uso sobre sistema operacional hospedeiro.

Vantagens do hipervisor tipo 2

Entre as vantagens do hipervisor tipo 2, podemos destacar:

- instalação simples;
- uso adequado para estudantes e desenvolvedores;
- permite testar sistemas sem substituir o sistema principal;
- facilita experimentação e aprendizagem;
- permite criar laboratórios locais;
- geralmente possui interface gráfica amigável;
- pode ser usado em notebooks e computadores pessoais.

Exemplo didático

Em uma disciplina de Sistemas Computacionais, o professor pode orientar os estudantes a instalar uma máquina virtual Linux usando VirtualBox. Com isso, todos podem praticar comandos, permissões, processos, redes e servidores locais sem alterar o sistema principal de seus computadores.

Limitações do hipervisor tipo 2

O hipervisor tipo 2 também possui limitações. Como ele depende do sistema operacional hospedeiro, parte dos recursos é consumida pelo próprio host antes de ser disponibilizada às máquinas virtuais.

Entre as limitações, estão:

- desempenho geralmente inferior ao tipo 1;
- maior dependência do sistema operacional hospedeiro;
- se o host travar, as VMs podem ser afetadas;
- maior sobrecarga de recursos;
- menor adequação para ambientes de produção críticos;
- limitações no acesso direto a alguns dispositivos físicos.

Observação

Apesar dessas limitações, hipervisores tipo 2 são excelentes para fins didáticos, desenvolvimento, testes de software e simulação de ambientes.

4.2.8 Comparação entre hipervisor tipo 1 e tipo 2

Aspecto	Tipo 1	Tipo 2
Posição na arquitetura	Executa diretamente sobre o hardware	Executa sobre um sistema operacional hospedeiro
Também chamado de	*Bare-metal*	Hospedado
Uso comum	Servidores, datacenters, nuvem e produção	Estudo, testes, desenvolvimento e uso pessoal
Desempenho	Geralmente maior	Geralmente menor que o tipo 1
Complexidade	Maior	Menor
Instalação	Pode substituir o sistema principal da máquina	Instalação como aplicativo no sistema existente
Exemplos	VMware ESXi, Hyper-V, KVM, Xen, Proxmox VE	VirtualBox, VMware Workstation, VMware Fusion, Parallels Desktop
Indicado para	Infraestrutura profissional e ambientes críticos	Laboratórios, ensino, testes e experimentação

Exemplo didático

Se o objetivo é montar um servidor de virtualização para hospedar vários sistemas em produção, um hipervisor tipo 1 tende a ser mais adequado.
Se o objetivo é estudar Linux dentro de um notebook Windows ou testar uma aplicação em outro sistema operacional, um hipervisor tipo 2 tende a ser mais simples e suficiente.

4.3 Máquinas virtuais

Máquinas virtuais são ambientes computacionais criados por software que simulam o funcionamento de um computador físico. Elas permitem executar um sistema operacional completo dentro de outro ambiente, utilizando recursos como processador, memória, armazenamento, rede e dispositivos virtuais.

Para o usuário, uma máquina virtual pode parecer um computador separado: ela possui tela, disco, sistema operacional, programas instalados, configurações próprias e conexão de rede. No entanto, todos esses elementos são fornecidos e controlados por uma camada de virtualização, geralmente administrada por um hipervisor.

Observação

Uma máquina virtual não é apenas uma janela com outro sistema operacional. Ela representa um computador simulado, com recursos próprios, regras de isolamento e uma relação controlada com o hardware real.

4.3.1 Conceito

Uma máquina virtual, frequentemente abreviada como VM (*Virtual Machine*), é uma instância de computador criada em software. Ela executa um sistema operacional convidado sobre recursos virtuais fornecidos por um hipervisor.

Em uma máquina física tradicional, o sistema operacional acessa diretamente o hardware, como CPU, memória, disco e placa de rede. Em uma máquina virtual, o sistema operacional convidado acessa versões virtuais desses recursos.

Exemplo didático

Um estudante pode usar um notebook com Windows e, dentro dele, executar uma máquina virtual com Ubuntu Linux. Nesse caso, o Windows é o sistema hospedeiro, o Ubuntu é o sistema convidado, e a ferramenta de virtualização, como VirtualBox ou VMware Workstation, gerencia os recursos da VM.

Componentes de uma máquina virtual

Uma máquina virtual normalmente possui:

- **CPU virtual**: representa uma ou mais unidades de processamento disponibilizadas para a VM;
- **memória virtual**: quantidade de memória RAM reservada ou gerenciada para o sistema convidado;
- **disco virtual**: arquivo ou volume que funciona como o armazenamento da VM;
- **placa de rede virtual**: interface que permite comunicação com o host, com outras VMs ou com a internet;
- **dispositivos virtuais**: adaptadores de vídeo, som, USB e outros recursos simulados ou repassados pelo hipervisor;
- **sistema operacional convidado**: sistema instalado dentro da VM, como Linux, Windows, BSD ou outro.

Exemplo didático

Ao criar uma VM, o usuário pode configurar:

```
Sistema convidado: Ubuntu Server
Memria RAM: 4096 MB
CPUs virtuais: 2
Disco virtual: 40 GB
Rede: NAT
```

Essas configurações definem como o hipervisor apresentará os recursos para o sistema operacional convidado.

4.3.2 Vantagens

Máquinas virtuais oferecem várias vantagens para estudo, desenvolvimento, testes, infraestrutura e administração de sistemas. Elas permitem criar ambientes isolados, reproduzíveis e flexíveis sem exigir um computador físico separado para cada sistema.

Isolamento

Uma das principais vantagens das máquinas virtuais é o isolamento. Cada VM possui seu próprio sistema operacional, seus próprios processos, seus arquivos e suas configurações.

Exemplo didático

Um estudante pode testar comandos administrativos em uma VM Linux sem alterar o sistema principal do notebook. Se uma configuração for feita de forma incorreta, o impacto fica restrito à máquina virtual.

Esse isolamento é especialmente útil para:

- testar sistemas operacionais;
- experimentar configurações;
- executar softwares desconhecidos;
- criar laboratórios de redes;
- separar ambientes de desenvolvimento;
- reduzir riscos ao sistema principal.

Execução de múltiplos sistemas operacionais

Com máquinas virtuais, é possível executar diferentes sistemas operacionais em uma mesma máquina física.

Exemplo didático

Um computador com Linux pode executar uma VM com Windows para testar uma aplicação nesse ambiente. Da mesma forma, um computador com Windows pode executar uma VM com Linux para estudo de terminal, servidores e ferramentas de desenvolvimento.

Isso é útil para desenvolvedores que precisam verificar se um sistema funciona corretamente em diferentes plataformas.

Ambientes reproduzíveis

Máquinas virtuais permitem criar ambientes padronizados. Em uma turma, por exemplo, o professor pode disponibilizar uma VM com sistema operacional, ferramentas e configurações já preparadas.

Exemplo didático

Em uma disciplina de Sistemas Computacionais, todos os estudantes podem usar uma VM com Linux, terminal configurado, editor instalado, Docker, Git e exemplos de aula. Isso reduz diferenças entre computadores pessoais e facilita o acompanhamento das atividades.

Facilidade para testes

Máquinas virtuais facilitam testes que poderiam ser arriscados em um sistema principal. É possível instalar pacotes, alterar configurações, testar scripts, criar usuários, modificar permissões e simular servidores sem comprometer diretamente o host.

Exemplo didático

Antes de aplicar uma atualização em um servidor real, uma equipe pode testar a atualização em uma VM semelhante. Se o teste for bem-sucedido, a alteração pode ser aplicada com mais segurança no ambiente de produção.

Snapshots e reversão de estado

Como visto anteriormente, snapshots permitem retornar a VM a um estado anterior. Isso torna as máquinas virtuais muito úteis em laboratórios.

Exemplo didático

Em uma aula sobre permissões de arquivos, os estudantes podem alterar permissões, criar usuários e modificar configurações. Ao final, o professor pode restaurar a VM para o estado inicial, preparando o ambiente para uma nova turma.

Melhor aproveitamento de hardware

Em servidores, máquinas virtuais permitem aproveitar melhor o hardware. Em vez de manter vários servidores físicos subutilizados, uma organização pode consolidar serviços em um número menor de máquinas físicas, executando várias VMs isoladas.

Exemplo didático

Uma instituição pode executar, em um único servidor físico, uma VM para sistema acadêmico, outra para banco de dados, outra para servidor web e outra para testes internos. Cada VM funciona como um servidor separado.

Apoio à computação em nuvem

Grande parte da computação em nuvem é baseada em virtualização. Quando um usuário cria uma instância em um provedor de nuvem, muitas vezes está criando uma máquina virtual hospedada em datacenters.

Exemplo didático

Ao contratar uma instância Linux em um provedor de nuvem, o usuário recebe acesso a um servidor virtual com CPU, memória, disco e rede definidos. Esse servidor pode ser usado para hospedar aplicações, bancos de dados, APIs ou ambientes de desenvolvimento.

4.3.3 Limitações

Apesar das vantagens, máquinas virtuais também possuem limitações. Elas não substituem todos os tipos de ambiente e podem apresentar custos de desempenho, armazenamento e complexidade.

Consumo de recursos

Uma VM precisa de memória, CPU e armazenamento. Como ela executa um sistema operacional completo, seu consumo costuma ser maior que o de um contêiner ou de um simples processo local.

Exemplo didático

Se uma VM recebe 4 GB de RAM, essa memória fica reservada ou comprometida para seu funcionamento enquanto ela está ativa. Em um computador com apenas 8 GB de RAM, executar várias VMs ao mesmo tempo pode deixar o sistema lento.

Desempenho inferior ao hardware direto

Embora a virtualização moderna seja eficiente, uma VM pode ter desempenho inferior ao de um sistema executando diretamente no hardware físico. Isso ocorre porque há uma camada adicional de virtualização entre o sistema convidado e os recursos reais.

Observação

A diferença de desempenho depende do tipo de tarefa, do hipervisor, do hardware, das configurações da VM e do suporte do processador à virtualização.

Tarefas que exigem muito desempenho gráfico, acesso intenso a disco, baixa latência ou uso especializado de hardware podem ser mais difíceis de executar eficientemente em uma VM.

Uso de armazenamento

Discos virtuais podem ocupar bastante espaço. Uma única VM com sistema operacional, atualizações e ferramentas de desenvolvimento pode consumir dezenas de gigabytes.

Exemplo didático

Uma VM criada para desenvolvimento web pode incluir Linux, bibliotecas, banco de dados, Docker, editores, logs e arquivos temporários. Com o tempo, o disco virtual pode crescer significativamente.

Complexidade de configuração

Algumas configurações de VM podem ser desafiadoras para iniciantes, especialmente em temas como rede, compartilhamento de pastas, resolução de vídeo, integração com dispositivos USB e instalação de drivers adicionais.

Exemplo didático

Um estudante pode conseguir instalar uma VM, mas ter dificuldade para configurar a rede em modo bridge, acessar um servidor dentro da VM a partir do host ou compartilhar uma pasta entre os dois sistemas.

Dependência do host

Em hipervisores tipo 2, a VM depende do sistema operacional hospedeiro. Se o host travar, for desligado ou ficar sem recursos, as máquinas virtuais também serão afetadas.

Observação

Em ambientes de produção, essa dependência é uma das razões para o uso de hipervisores tipo 1, servidores dedicados, armazenamento redundante e estratégias de alta disponibilidade.

Limitações com dispositivos físicos

Embora seja possível repassar alguns dispositivos físicos para uma VM, nem sempre isso é simples ou eficiente. Dispositivos como placas de vídeo, adaptadores USB específicos, câmeras, impressoras e interfaces especiais podem exigir configuração adicional.

Exemplo didático

Executar uma aplicação que depende fortemente de GPU dentro de uma VM pode exigir configurações avançadas, como passagem direta de dispositivo (*passthrough*), recurso que nem sempre está disponível ou é fácil de configurar.

Não substituem contêineres em todos os cenários

Máquinas virtuais oferecem isolamento forte, mas podem ser mais pesadas do que contêineres. Para empacotar e distribuir aplicações modernas, contêineres podem ser mais rápidos, leves e práticos.

Observação

A escolha entre VM e contêiner depende do objetivo. Se é necessário executar um sistema operacional completo, uma VM pode ser adequada. Se o objetivo é empacotar uma aplicação e suas dependências, um contêiner pode ser mais eficiente.

4.3.4 Exemplos de uso

Máquinas virtuais aparecem em muitos contextos da tecnologia. Elas são usadas tanto em ambientes educacionais quanto em servidores, empresas, nuvem, testes de software, segurança e desenvolvimento.

Estudo de sistemas operacionais

Uma das formas mais comuns de uso de VMs é o estudo de sistemas operacionais. O estudante pode instalar Linux, Windows Server, FreeBSD ou outras plataformas sem

alterar seu computador principal.

Exemplo didático

Em uma disciplina de Sistemas Computacionais, uma VM com Linux pode ser usada para praticar comandos de terminal, permissões de arquivos, gerenciamento de processos, instalação de pacotes e configuração de rede.

Laboratórios de redes

Máquinas virtuais podem ser usadas para simular ambientes de rede. É possível criar várias VMs representando servidores, clientes, roteadores ou máquinas de teste.

Exemplo didático

Um laboratório pode conter:

- uma VM como servidor web;
- uma VM como cliente;
- uma VM como servidor de banco de dados;
- uma VM como firewall ou roteador.

Com isso, os estudantes podem estudar endereçamento IP, portas, requisições, serviços e comunicação entre máquinas.

Servidores e infraestrutura

Em empresas e instituições, máquinas virtuais são amplamente utilizadas para hospedar serviços. Cada serviço pode ficar em uma VM separada, facilitando isolamento, manutenção e escalabilidade.

Exemplo didático

Uma instituição pode organizar sua infraestrutura assim:

```
VM 1: servidor web
VM 2: banco de dados
VM 3: sistema acadmico
VM 4: ambiente de homologao
VM 5: servidor de arquivos
```

Essa separação ajuda a administrar recursos, permissões, atualizações e riscos.

Computação em nuvem

Quando usamos provedores de nuvem, muitas vezes estamos utilizando máquinas virtuais. O usuário escolhe a quantidade de CPU, memória, disco, sistema operacional e região geográfica, e o provedor cria uma instância virtual.

Exemplo didático

Um desenvolvedor pode criar uma VM em nuvem para hospedar uma API, instalar um banco de dados, configurar um servidor web ou executar um modelo de machine learning.

Segurança e análise de software

Máquinas virtuais são úteis para testar softwares em ambientes controlados. Pesquisadores e profissionais de segurança podem analisar comportamentos sem comprometer diretamente o sistema principal.

Observação

Atividades de segurança devem ser realizadas apenas em ambientes autorizados, controlados e com finalidade educativa ou profissional legítima.

Preservação de sistemas antigos

Algumas organizações dependem de softwares antigos que só funcionam em sistemas operacionais desatualizados. Máquinas virtuais podem ajudar a manter esses ambientes acessíveis por mais tempo.

Exemplo didático

Um sistema legado desenvolvido para uma versão antiga de Windows pode ser executado dentro de uma VM, evitando a necessidade de manter um computador físico antigo apenas para essa finalidade.

4.4 Principais ferramentas de virtualização

- VirtualBox;
- VMware Workstation;
- VMware ESXi;
- Hyper-V;

- KVM;
- QEMU;
- Proxmox VE.

4.5 Contêineres

Contêineres são uma forma moderna de empacotar, distribuir e executar aplicações de software em ambientes isolados. Eles permitem reunir, em uma única unidade, o código da aplicação, suas dependências, bibliotecas, configurações e demais elementos necessários para sua execução.

A ideia central é simples: se uma aplicação funciona dentro de um contêiner, ela tende a funcionar de maneira semelhante em diferentes ambientes, como o computador do desenvolvedor, um servidor de testes, um ambiente de produção ou uma infraestrutura em nuvem.

Observação

Contêineres não são máquinas virtuais completas. Eles não executam, necessariamente, um sistema operacional inteiro separado. Em geral, os contêineres compartilham o kernel do sistema hospedeiro e isolam apenas o ambiente de execução da aplicação.

4.5.1 O que são contêineres

Um contêiner pode ser entendido como um ambiente isolado de execução para uma aplicação. Ele contém tudo o que a aplicação precisa para funcionar, mas de forma mais leve do que uma máquina virtual tradicional.

Em vez de instalar manualmente dependências no computador, configurar versões de bibliotecas, ajustar variáveis de ambiente e torcer para que tudo funcione da mesma forma em outro lugar, o desenvolvedor descreve o ambiente da aplicação em arquivos de configuração. A partir dessa descrição, uma imagem de contêiner pode ser criada e executada em diferentes máquinas.

Exemplo didático

Uma aplicação web feita em Python com Flask pode depender de uma versão específica do Python, da biblioteca Flask e de outras bibliotecas auxiliares. Em vez de instalar tudo manualmente em cada computador, é possível criar uma imagem contendo essas dependências. Quando a imagem é executada, ela gera um contêiner com o ambiente pronto para rodar a aplicação.

Imagem e contêiner

Para compreender contêineres, é importante diferenciar dois conceitos: **imagem** e **contêiner**.

Uma **imagem** é um modelo pronto, imutável, que contém a aplicação e suas dependências. Ela funciona como uma receita ou molde.

Um **contêiner** é uma instância em execução dessa imagem. Ele é o ambiente efetivamente iniciado a partir da imagem.

Exemplo didático

Podemos comparar com uma receita de bolo. A imagem seria a receita: define os ingredientes e o modo de preparo. O contêiner seria o bolo preparado a partir daquela receita.
Da mesma forma, uma única imagem pode ser usada para criar vários contêineres.

Contêineres e isolamento

Cada contêiner executa em um ambiente isolado. Isso significa que ele pode ter seus próprios processos, sistema de arquivos, variáveis de ambiente, configurações de rede e dependências.

Esse isolamento ajuda a evitar conflitos entre aplicações.

Exemplo didático

Imagine que um projeto precise do Python 3.10 e outro precise do Python 3.12. Se ambos forem instalados diretamente no mesmo sistema operacional, podem surgir conflitos de versões. Com contêineres, cada aplicação pode executar em seu próprio ambiente, com a versão adequada de Python.

Apesar desse isolamento, contêineres são mais leves do que máquinas virtuais porque compartilham o kernel do sistema operacional hospedeiro.

Contêineres e o kernel do sistema

Em uma máquina virtual, cada VM normalmente executa seu próprio sistema operacional completo, incluindo seu próprio kernel. Já em contêineres, o kernel é compartilhado com o sistema hospedeiro.

O contêiner isola o ambiente da aplicação, mas não cria um sistema operacional completo do zero. Por isso, sua inicialização costuma ser mais rápida e seu consumo de recursos tende a ser menor.

Aspecto	Máquina virtual	Contêiner
Sistema operacional	Possui sistema operacional convidado completo	Compartilha o kernel do hospedeiro
Inicialização	Geralmente mais lenta	Geralmente mais rápida
Consumo de recursos	Maior	Menor
Isolamento	Mais completo	Isolamento no nível do sistema operacional
Uso comum	Sistemas completos e ambientes heterogêneos	Aplicações e serviços empacotados

Observação

Dizer que contêineres são mais leves não significa que sejam sempre a melhor escolha. Máquinas virtuais e contêineres resolvem problemas diferentes e muitas vezes são usados em conjunto.

Por que contêineres se tornaram populares?

Contêineres se tornaram populares porque ajudam a resolver um problema muito comum no desenvolvimento de software: a diferença entre ambientes.

É comum ouvir frases como:

> "Na minha máquina funciona."

Essa frase aparece quando uma aplicação funciona no computador do desenvolvedor, mas falha no computador de outra pessoa, no servidor ou no ambiente de produção. As causas podem incluir diferenças de versões, dependências ausentes, configurações incorretas, variáveis de ambiente ou sistemas operacionais diferentes.

Os contêineres reduzem esse problema ao permitir que o ambiente da aplicação seja descrito, versionado e reproduzido.

Exemplo didático

Uma equipe pode definir que a aplicação será executada em um contêiner com Python 3.12, Flask, Gunicorn e determinadas variáveis de ambiente. Todos os desenvolvedores executam a mesma imagem, reduzindo diferenças entre os ambientes individuais.

Contêineres no desenvolvimento de software

No desenvolvimento de software, contêineres são úteis porque permitem padronizar ambientes e reduzir conflitos de configuração.

Eles podem ser usados para:

- executar aplicações web;
- executar bancos de dados locais;
- testar diferentes versões de linguagens;
- criar ambientes de desenvolvimento;
- empacotar aplicações para implantação;
- executar testes automatizados;
- simular ambientes de produção;
- facilitar integração contínua e entrega contínua.

Contêineres em produção

Em ambientes de produção, contêineres facilitam a implantação de aplicações. A mesma imagem testada em desenvolvimento pode ser enviada para um servidor, registro de imagens ou ambiente de nuvem.

Isso ajuda a reduzir diferenças entre desenvolvimento, teste e produção.

Observação

Em produção, o uso de contêineres exige cuidados adicionais com segurança, redes, volumes, variáveis de ambiente, logs, monitoramento, atualização de imagens e orquestração.

Ferramentas como Kubernetes são usadas para gerenciar muitos contêineres em ambientes distribuídos. Elas permitem controlar escalabilidade, recuperação automática, balanceamento de carga e implantação gradual.

Limites do isolamento em contêineres

Embora contêineres ofereçam isolamento, eles compartilham o kernel do sistema hospedeiro. Isso significa que o isolamento é diferente daquele oferecido por máquinas virtuais.

Exemplo didático

Uma máquina virtual pode executar um kernel completamente diferente do host. Um contêiner, por sua vez, depende do kernel do hospedeiro. Por isso, contêineres Linux normalmente dependem de um kernel Linux, mesmo quando executados em sistemas como Windows ou macOS por meio de camadas adicionais de virtualização.

Observação

Contêineres são muito úteis, mas não devem ser vistos como uma solução mágica para todos os problemas. Eles precisam ser configurados com boas práticas de segurança e manutenção.

4.5.2 Diferenças entre máquinas virtuais e contêineres

Aspecto	Máquina virtual	Contêiner
Sistema operacional	Cada VM possui seu próprio sistema operacional	Compartilha o kernel do sistema hospedeiro
Isolamento	Alto	Alto, mas mais leve
Consumo de recursos	Maior	Menor
Tempo de inicialização	Mais lento	Mais rápido
Uso comum	Simular servidores completos	Empacotar e executar aplicações

4.6 Docker

Docker é uma plataforma para criação, distribuição e execução de contêineres. Ele permite empacotar uma aplicação junto com suas dependências, bibliotecas, configurações e comandos de inicialização, facilitando a execução do mesmo sistema em diferentes ambientes.

Na prática, Docker ajuda a resolver um problema comum no desenvolvimento de software: uma aplicação funciona no computador de uma pessoa, mas não funciona corretamente no computador de outra ou no servidor de produção. Isso pode acontecer por diferenças de versão do sistema operacional, da linguagem, das bibliotecas, das variáveis de ambiente ou dos serviços necessários.

Com Docker, o ambiente de execução passa a ser descrito em arquivos, podendo ser versionado, compartilhado e reproduzido.

Observação

Docker não elimina a necessidade de compreender sistemas operacionais, redes, arquivos e processos. Pelo contrário: ele torna esses conceitos ainda mais importantes, pois contêineres usam recursos do sistema operacional, expõem portas, montam volumes, executam processos e acessam arquivos.

4.6.1 Por que usar Docker?

Entre os principais motivos para usar Docker, estão:

- padronizar ambientes de desenvolvimento;
- reduzir conflitos de dependências;
- facilitar a execução de aplicações em diferentes computadores;
- simplificar testes e implantação;
- executar serviços sem instalá-los diretamente no sistema;
- criar ambientes descartáveis;
- aproximar desenvolvimento, teste e produção;
- facilitar o uso de bancos de dados, servidores web, filas e outras ferramentas auxiliares.

Exemplo didático

Um estudante pode executar um banco PostgreSQL para uma atividade sem instalar o PostgreSQL diretamente no computador. Basta executar um contêiner com a imagem oficial do banco:

```
docker run --name banco-aula \
-e POSTGRES_PASSWORD=senha \
-p 5432:5432 \
-d postgres:16
```

Quando não precisar mais do banco, o contêiner pode ser parado e removido.

4.6.2 Arquitetura básica do Docker

De forma simplificada, o Docker envolve alguns elementos principais:

- **Docker Engine**: mecanismo responsável por construir e executar contêineres;
- **Docker CLI**: interface de linha de comando usada para enviar instruções ao Docker;
- **Imagem**: modelo usado para criar contêineres;
- **Contêiner**: instância em execução de uma imagem;
- **Dockerfile**: arquivo que descreve como construir uma imagem;
- **Registry**: repositório de imagens, como Docker Hub ou registros privados;

- **Volume**: mecanismo para persistência de dados;
- **Rede**: mecanismo para comunicação entre contêineres, host e outros serviços.

4.6.3 Imagem

Uma imagem Docker é um modelo usado para criar contêineres. Ela contém todos os elementos necessários para executar uma aplicação: sistema base, bibliotecas, dependências, arquivos, configurações e comandos.

A imagem é como uma fotografia ou molde do ambiente da aplicação. Ela não está necessariamente em execução. Ela serve como base para iniciar um ou mais contêineres.

Exemplo didático

A imagem `python:3.12-slim` contém uma versão reduzida de um ambiente Linux com Python 3.12 instalado. A partir dela, podemos construir uma imagem própria para executar uma aplicação Python.

Imagem como modelo imutável

Uma característica importante das imagens é que elas são tratadas como modelos imutáveis. Quando executamos uma imagem, criamos um contêiner. Se alteramos arquivos dentro do contêiner, a imagem original não é modificada automaticamente.

Exemplo didático

Se executarmos um contêiner a partir da imagem `ubuntu` e criarmos um arquivo dentro dele, esse arquivo pertence ao contêiner em execução. A imagem `ubuntu` original continua a mesma.

Camadas de uma imagem

Imagens Docker são compostas por camadas. Cada instrução importante em um `Dockerfile` pode gerar uma nova camada.

Essa organização em camadas permite reaproveitamento e cache. Se uma camada já existe localmente, o Docker pode reutilizá-la, tornando a construção de imagens mais rápida.

Exemplo didático

Considere o trecho:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
```

A instalação das dependências pode ficar em uma camada separada. Se o arquivo `requirements.txt` não mudar, o Docker pode reutilizar essa etapa em construções futuras.

Listando imagens

Para listar imagens disponíveis localmente, usamos:

```
docker images
```

ou:

```
docker image ls
```

Um resultado simplificado poderia ser:

```
REPOSITORY      TAG         IMAGE ID       SIZE
python          3.12-slim   abc123         130MB
ubuntu          latest      def456          80MB
flask-aula      latest      ghi789         180MB
```

Nesse exemplo:

- `REPOSITORY` indica o nome da imagem;
- `TAG` indica a versão ou variação da imagem;
- `IMAGE ID` é um identificador interno;
- `SIZE` mostra o tamanho ocupado.



drawn by adit.io

Tags de imagens

Uma tag identifica uma versão ou variante de uma imagem. Por exemplo:

```
python:3.12
python:3.12-slim
python:3.11
python:latest
```

A parte antes dos dois pontos é o nome da imagem. A parte depois dos dois pontos é a tag.

Observação

Evite depender automaticamente da tag `latest` em ambientes de produção ou em materiais didáticos que precisam ser reproduzíveis. Ela pode mudar com o tempo. Sempre que possível, indique uma versão explícita, como `python:3.12-slim`.

Baixando imagens

Para baixar uma imagem sem executá-la imediatamente, usamos:

```
docker pull python:3.12-slim
```

O Docker busca a imagem em um registro, como o Docker Hub, e salva as camadas localmente.

Construindo imagens

Podemos construir uma imagem a partir de um `Dockerfile` usando:

```
docker build -t minha-imagem .
```

Nesse comando:

- `docker build` inicia a construção;
- `-t minha-imagem` atribui um nome à imagem;
- `.` indica que o contexto de construção é o diretório atual.

Removendo imagens

Para remover uma imagem local, usamos:

```
docker image rm nome_da_imagem
```

Observação

Uma imagem não pode ser removida facilmente se ainda houver contêineres baseados nela. Nesse caso, é necessário remover os contêineres primeiro ou usar opções específicas com cuidado.

4.6.4 Contêiner

Um contêiner é uma instância em execução de uma imagem. Se a imagem é o modelo, o contêiner é o ambiente efetivamente iniciado a partir desse modelo.

Um mesmo modelo pode gerar vários contêineres independentes.

Exemplo didático

A imagem `nginx` pode ser usada para criar vários contêineres de servidor web. Cada contêiner pode ter nome, portas, volumes e configurações próprias.

Executando um contêiner

Para executar um contêiner simples, usamos:

```
docker run hello-world
```

Esse comando baixa a imagem, se ela ainda não estiver disponível, cria um contêiner e executa o programa definido pela imagem.

Exemplo didático

Para executar um terminal Ubuntu dentro de um contêiner:

```
docker run -it ubuntu bash
```

Nesse comando:

- `-i` mantém a entrada interativa;
- `-t` cria um terminal;
- `ubuntu` é a imagem usada;
- `bash` é o comando executado dentro do contêiner.

Listando contêineres

Para listar contêineres em execução:

```
docker ps
```

Para listar todos os contêineres, inclusive os parados:

```
docker ps -a
```

Um resultado simplificado poderia ser:

```
CONTAINER ID   IMAGE     STATUS       PORTS                  NAMES
abc123         nginx     Up 2 min     0.0.0.0:8080->80/tcp   web-aula
def456         ubuntu    Exited       -                      teste-ubuntu
```

Nomeando contêineres

Podemos atribuir um nome a um contêiner com a opção `-name`:

```
docker run --name web-aula -d -p 8080:80 nginx
```

Nesse exemplo:

- `-name web-aula` define o nome do contêiner;
- `-d` executa em segundo plano;
- `-p 8080:80` mapeia a porta 8080 do host para a porta 80 do contêiner;
- `nginx` é a imagem usada.

Parando e removendo contêineres

Para parar um contêiner:

```
docker stop web-aula
```

Para remover um contêiner parado:

```
docker rm web-aula
```

Para remover vários contêineres parados, pode-se usar:

```
docker container prune
```

Observação

Remover um contêiner pode apagar alterações feitas dentro dele, caso os dados não estejam armazenados em volumes ou em diretórios montados do host.

Portas

Aplicações web dentro de contêineres normalmente escutam em uma porta interna. Para acessá-las pelo navegador no computador hospedeiro, é necessário mapear uma porta do host para uma porta do contêiner.

Exemplo didático

O comando abaixo executa o Nginx dentro do contêiner na porta 80, mas expõe o serviço no host pela porta 8080:

```
docker run --name nginx-aula -d -p 8080:80 nginx
```

Depois, podemos acessar:

```
http://localhost:8080
```

Volumes

Por padrão, dados criados dentro de um contêiner podem ser perdidos quando o contêiner é removido. Para persistir dados, usamos volumes ou montagens de diretório.

Exemplo didático

Criando um volume:

```
docker volume create dados_postgres
```

Executando PostgreSQL com volume:

```
docker run --name postgres-aula \
-e POSTGRES_PASSWORD=senha \
-v dados_postgres:/var/lib/postgresql/data \
-p 5432:5432 \
-d postgres:16
```

Nesse caso, os dados do banco são armazenados no volume `dados_postgres`, e não apenas no sistema de arquivos temporário do contêiner.

Logs

Para visualizar a saída gerada por um contêiner:

```
docker logs nome_do_container
```

Para acompanhar os logs em tempo real:

```
docker logs -f nome_do_container
```

Logs são importantes para entender erros de inicialização, falhas de conexão, mensagens de aplicação e comportamento do serviço.

Entrando em um contêiner em execução

Podemos executar comandos dentro de um contêiner em execução usando `docker exec`.

Exemplo didático

Entrando em um contêiner com shell:

```
docker exec -it web-aula bash
```

Se a imagem não possuir `bash`, pode ser necessário usar `sh`:

```
docker exec -it web-aula sh
```

Observação

Entrar em um contêiner pode ser útil para diagnóstico, mas não deve ser a principal forma de configurar aplicações em produção. O ideal é que a configuração do ambiente esteja descrita em arquivos versionados, como `Dockerfile` e `docker-compose.yml`.

4.6.5 Dockerfile

Um `Dockerfile` é um arquivo de texto que contém instruções para construir uma imagem Docker. Ele descreve o ambiente da aplicação de forma declarativa e reproduzível.

Em vez de instalar manualmente dependências em cada computador, o desenvolvedor registra no `Dockerfile` os passos necessários para preparar o ambiente.

Exemplo didático

Um `Dockerfile` simples para uma aplicação Python pode conter:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

CMD ["python", "app.py"]
```

Principais instruções

As instruções mais comuns em um `Dockerfile` incluem:

Instrução	Função
`FROM`	Define a imagem base.
`WORKDIR`	Define o diretório de trabalho dentro da imagem.
`COPY`	Copia arquivos do host para a imagem.
`ADD`	Copia arquivos e possui recursos adicionais, como extrair arquivos compactados.
`RUN`	Executa comandos durante a construção da imagem.
`CMD`	Define o comando padrão executado ao iniciar o contêiner.
`ENTRYPOINT`	Define o executável principal do contêiner.
`ENV`	Define variáveis de ambiente.
`EXPOSE`	Documenta a porta usada pela aplicação dentro do contêiner.
`VOLUME`	Indica pontos de montagem para persistência de dados.
`USER`	Define o usuário usado para executar comandos.

Imagem base

A instrução `FROM` define a imagem inicial. Ela é a base sobre a qual a imagem da aplicação será construída.

Exemplo didático

```
FROM python:3.12-slim
```

Esse comando indica que a imagem será construída a partir de uma imagem que já possui Python 3.12 em uma versão reduzida.

Diretório de trabalho

A instrução `WORKDIR` define o diretório dentro da imagem onde os comandos seguintes serão executados.

```
WORKDIR /app
```

Isso evita a necessidade de escrever caminhos completos em todas as instruções seguintes.

Copiando arquivos

A instrução `COPY` copia arquivos do diretório do projeto no host para dentro da imagem.

```
COPY requirements.txt .
COPY app.py .
```

O ponto final representa o diretório de trabalho definido por `WORKDIR`.

Instalando dependências

A instrução `RUN` executa comandos durante a construção da imagem.

```
RUN pip install -r requirements.txt
```

Esse comando instala as dependências listadas no arquivo `requirements.txt`.

Observação

É comum copiar primeiro o arquivo de dependências e instalar as bibliotecas antes de copiar todo o restante do projeto. Isso permite que o Docker aproveite o cache quando o código muda, mas as dependências continuam iguais.

Comando padrão

A instrução `CMD` define o comando padrão que será executado quando um contêiner for iniciado a partir da imagem.

```
CMD ["python", "app.py"]
```

Esse formato em lista é comum e evita alguns problemas de interpretação de comandos.

Arquivo `.dockerignore`

O arquivo `.dockerignore` informa ao Docker quais arquivos ou diretórios não devem ser enviados para o contexto de construção da imagem.

Exemplo didático

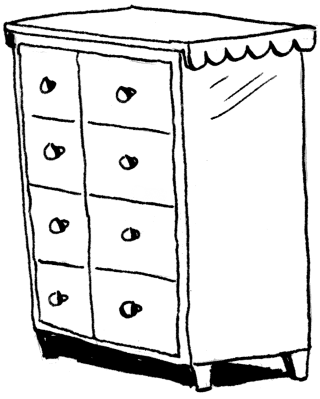
Exemplo de `.dockerignore`:

```
venv/
__pycache__/
.git/
.env
*.pyc
```

Isso ajuda a reduzir o tamanho da imagem, proteger arquivos sensíveis e evitar cópias desnecessárias.

Observação

Arquivos com senhas, tokens e chaves privadas não devem ser copiados para imagens Docker. Uma imagem pode ser compartilhada com outras pessoas ou enviada para um registro remoto.



drawn by adit.io

Exemplo completo com Flask

Considere a seguinte aplicação Flask.

Arquivo `app.py`:

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/")
def home():
return jsonify({
        "mensagem": "Ol  , Docker!",
        "disciplina": "Sistemas Computacionais"
})

if __name__ == "__main__":
app.run(host="0.0.0.0", port=5000)
```

Arquivo `requirements.txt`:

```
flask
```

Arquivo `Dockerfile`:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Construção da imagem:

```
docker build -t flask-docker-aula .
```

Execução:

```
docker run --name flask-aula -p 5000:5000 flask-docker-aula
```

Teste:

```
curl http://localhost:5000
```

4.6.6 Docker Compose

Docker Compose é uma ferramenta que permite definir e executar aplicações compostas por múltiplos contêineres. Em vez de executar vários comandos `docker run` manualmente, descrevemos os serviços em um arquivo `docker-compose.yml`.

Esse arquivo pode definir aplicações web, bancos de dados, volumes, redes, variáveis de ambiente e portas.

Exemplo didático

Uma aplicação web pode depender de um banco de dados. Em vez de iniciar manualmente um contêiner para a aplicação e outro para o banco, podemos descrever ambos no Docker Compose.

Serviços

No Docker Compose, cada contêiner principal é descrito como um serviço.

```
services:
web:
build: .
ports:
- "5000:5000"

db:
image: postgres:16
environment:
POSTGRES_PASSWORD: senha
```

Nesse exemplo, existem dois serviços:

- `web`: aplicação construída a partir do `Dockerfile` do projeto;
- `db`: banco PostgreSQL criado a partir da imagem `postgres:16`.

Executando com Compose

Para iniciar os serviços definidos no arquivo:

```
docker compose up
```

Para executar em segundo plano:

```
docker compose up -d
```

Para parar e remover os contêineres criados:

```
docker compose down
```

Para visualizar logs:

```
docker compose logs
docker compose logs -f
```

Volumes no Compose

Volumes permitem persistir dados mesmo quando contêineres são removidos.

Exemplo didático

Exemplo com volume para PostgreSQL:

```
services:
db:
image: postgres:16
environment:
POSTGRES_PASSWORD: senha
volumes:
- dados_db:/var/lib/postgresql/data

volumes:
dados_db:
```

Nesse exemplo, os dados do banco ficam armazenados no volume `dados_db`.

Redes no Compose

Por padrão, o Docker Compose cria uma rede para os serviços definidos no arquivo. Dentro dessa rede, os serviços podem se comunicar pelo nome.

Exemplo didático

Se há um serviço chamado `db`, a aplicação web pode tentar acessar o banco usando o nome `db` como endereço de host:

```
DATABASE_HOST=db
```

Isso facilita a comunicação entre contêineres sem depender de endereços IP fixos.

Exemplo completo: Flask com PostgreSQL

A estrutura do projeto poderia ser:

```
projeto-compose/
 app.py
 requirements.txt
 Dockerfile
 docker-compose.yml
```

Arquivo `app.py` simplificado:

```
from flask import Flask, jsonify
import os

app = Flask(__name__)

@app.route("/")
def home():
return jsonify({
        "mensagem": "Aplica  o com Docker Compose",
        "banco": os.getenv("DATABASE_HOST", "n o
           configurado")
})

if __name__ == "__main__":
app.run(host="0.0.0.0", port=5000)
```

Arquivo `requirements.txt`:

```
flask
```

Arquivo `Dockerfile`:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

CMD ["python", "app.py"]
```

Arquivo `docker-compose.yml`:

```
services:
web:
build: .
ports:
- "5000:5000"
environment:
DATABASE_HOST: db
depends_on:
- db

db:
image: postgres:16
environment:
POSTGRES_PASSWORD: senha
POSTGRES_DB: aula
POSTGRES_USER: aluno
volumes:
- dados_db:/var/lib/postgresql/data

volumes:
dados_db:
```

Executando:

```
docker compose up --build
```

Testando:

```
curl http://localhost:5000
```

Encerrando:

```
docker compose down
```

Observação

O campo `depends_on` indica que o serviço `web` depende do serviço `db`. Isso organiza a ordem de inicialização, mas não garante necessariamente que o banco já esteja pronto para receber conexões. Em aplicações reais, pode ser necessário implementar lógica de tentativa de reconexão.

4.6.7 Comandos Docker e Docker Compose

Comando	Função
`docker build -t app .`	Constrói uma imagem a partir de um `Dockerfile`.
`docker run app`	Executa um contêiner a partir de uma imagem.
`docker ps`	Lista contêineres em execução.
`docker stop nome`	Para um contêiner.
`docker rm nome`	Remove um contêiner.
`docker logs nome`	Exibe logs de um contêiner.
`docker compose up`	Inicia os serviços definidos no Compose.
`docker compose up -d`	Inicia os serviços em segundo plano.
`docker compose down`	Para e remove os serviços criados pelo Compose.
`docker compose logs -f`	Acompanha os logs dos serviços.

4.6.8 Boas práticas iniciais

Algumas boas práticas ao trabalhar com Docker incluem:

- usar imagens base confiáveis;
- preferir tags específicas em vez de `latest`;
- evitar copiar arquivos desnecessários para a imagem;
- usar `.dockerignore`;
- não colocar senhas e tokens diretamente na imagem;

- usar volumes para dados persistentes;
- mapear apenas as portas necessárias;
- manter imagens atualizadas;
- remover imagens e contêineres antigos quando não forem mais usados;
- documentar os comandos no `README.md`.

Observação

Docker facilita muito o desenvolvimento, mas também pode esconder complexidades. Quando algo dá errado, é importante saber investigar logs, portas, volumes, redes, permissões e processos.

4.7 Comandos básicos de Docker

```
docker --version
docker ps
docker ps -a
docker images
docker run hello-world
docker run -it ubuntu bash
docker stop nome_do_container
docker rm nome_do_container
```

Exercícios

1. Máquinas virtuais e contêineres são duas formas de criar ambientes isolados, mas fazem isso em níveis diferentes. Compare as duas tecnologias considerando consumo de recursos, tempo de inicialização, isolamento, dependência do sistema operacional e facilidade de uso no desenvolvimento de software.

2. Uma imagem Docker pode ser vista como uma receita ou modelo para criar contêineres. Explique essa comparação e discuta por que é importante que a imagem descreva não apenas o código da aplicação, mas também suas dependências, configurações e comandos de execução.

3. Ao executar o comando `docker run`, o Docker cria e inicia um contêiner a partir de uma imagem. Quais etapas estão envolvidas nesse processo? Considere aspectos como imagem, sistema de arquivos, processo principal, rede, portas e volumes.

4. Contêineres são frequentemente descritos como ambientes leves e portáteis. Em quais situações essas características são vantajosas para desenvolvimento, testes, integração contínua e implantação de sistemas?

5. Apesar das vantagens dos contêineres, máquinas virtuais ainda são amplamente utilizadas. Em quais cenários uma máquina virtual pode ser mais adequada do que um contêiner? Considere casos que exigem sistemas operacionais completos, kernels diferentes, isolamento mais forte ou simulação de infraestrutura.

6. Contêineres compartilham o kernel do sistema hospedeiro, enquanto máquinas virtuais normalmente executam um sistema operacional convidado completo. Que implicações essa diferença traz para desempenho, compatibilidade e segurança?

7. Ao baixar uma imagem pronta da internet, o desenvolvedor ganha praticidade, mas também assume riscos. Que cuidados devem ser tomados antes de usar imagens de terceiros em projetos acadêmicos, profissionais ou em produção?

8. Pense em uma aplicação que você já desenvolveu ou pretende desenvolver. Quais partes dela poderiam ser empacotadas em contêineres? Quais serviços auxiliares poderiam ser executados separadamente? Como isso mudaria a organização do projeto?

9. Explique como conceitos como processos, arquivos, permissões, portas, redes e variáveis de ambiente aparecem quando trabalhamos com contêineres.

Capítulo 5

Aplicações para Desenvolvimento de Software

Objetivos de aprendizagem

Ao final deste capítulo, você deverá ser capaz de:

- relacionar conceitos de sistemas computacionais com desenvolvimento de software;
- compreender o ambiente de execução de aplicações;
- utilizar terminal, editor de código, Git e servidores locais;
- preparar um ambiente básico de desenvolvimento.

5.1 O computador como ambiente de desenvolvimento

Para o usuário comum, o computador é uma ferramenta para acessar a internet, produzir documentos, assistir a vídeos, enviar mensagens ou usar sistemas prontos. Para o desenvolvedor de software, porém, o computador é também um ambiente de criação, execução, teste, depuração e publicação de aplicações.

Desenvolver software não significa apenas escrever código. Envolve configurar ferramentas, instalar linguagens, gerenciar dependências, organizar arquivos, executar programas, testar serviços, controlar versões, acessar servidores e compreender como o sistema operacional interage com os programas.

Observação

O ambiente de desenvolvimento é o conjunto de ferramentas, configurações e recursos usados para criar e testar software. Ele inclui sistema operacional, editor de código, terminal, linguagens, bibliotecas, servidores locais, bancos de dados, Git e outras ferramentas auxiliares.

5.1.1 Sistema operacional

O sistema operacional é a base do ambiente de desenvolvimento. Ele gerencia arquivos, processos, memória, rede, permissões e dispositivos. Todas as ferramentas utilizadas pelo desenvolvedor dependem dele.

Entre as tarefas realizadas pelo sistema operacional durante o desenvolvimento, estão:

- executar editores de código e IDEs;
- criar processos para programas e servidores locais;
- gerenciar arquivos do projeto;
- controlar permissões de scripts;
- disponibilizar terminal;
- permitir instalação de pacotes;
- gerenciar conexões de rede;
- executar máquinas virtuais e contêineres.

Exemplo didático

Ao executar uma aplicação Flask com:

```
python app.py
```

o sistema operacional cria um processo para o interpretador Python, reserva memória, permite acesso aos arquivos do projeto e abre uma porta de rede para que a aplicação responda a requisições.

5.1.2 Editor de código

O editor de código é a ferramenta usada para escrever, organizar e modificar os arquivos de um projeto. Diferentemente de um editor de texto comum, ele oferece recursos voltados à programação.

Entre os recursos comuns de editores de código, estão:

- destaque de sintaxe;
- autocompletar;
- integração com terminal;
- integração com Git;
- formatação automática;
- extensões;
- depuração;
- busca em múltiplos arquivos;
- navegação entre funções, classes e módulos.

Exemplo didático

Ferramentas como Visual Studio Code, PyCharm, Eclipse, NetBeans, Vim, Neovim e Sublime Text podem ser usadas para desenvolvimento de software. A escolha depende da linguagem, do projeto, da preferência do desenvolvedor e dos recursos necessários.

Observação

Um bom editor não substitui o entendimento do código. Ele ajuda na produtividade, mas o desenvolvedor ainda precisa compreender estruturas, erros, dependências, caminhos, execução e lógica do programa.

5.1.3 Terminal

O terminal é uma das ferramentas mais importantes no desenvolvimento de software. Ele permite executar comandos, iniciar programas, instalar pacotes, navegar por diretórios, usar Git, testar APIs e controlar processos.

Mesmo em ambientes com interface gráfica, muitas tarefas de desenvolvimento são realizadas com comandos.

Exemplo didático

Com o terminal, um desenvolvedor pode criar um projeto, iniciar um ambiente virtual, instalar dependências e executar uma aplicação:

```
mkdir meu-projeto
cd meu-projeto
python3 -m venv venv
source venv/bin/activate
pip install flask
python app.py
```

O terminal aproxima o estudante do funcionamento real do sistema operacional. Ele evidencia caminhos, arquivos, processos, permissões, portas e mensagens de erro.

5.1.4 Interpretadores, compiladores e runtimes

Linguagens de programação precisam de ferramentas para transformar ou executar o código escrito pelo desenvolvedor.

Um **interpretador** executa o código diretamente ou por etapas, traduzindo instruções durante a execução. Um **compilador** traduz o código-fonte para outro formato antes da execução, frequentemente para código de máquina ou bytecode. Um **runtime** é o ambiente necessário para executar programas de determinada tecnologia.

Exemplo didático

Em Python, normalmente usamos um interpretador:

```
python programa.py
```

Em C, normalmente compilamos o código antes de executar:

```
gcc programa.c -o programa
./programa
```

Em Java, o código é compilado para bytecode e executado pela JVM:

```
javac Programa.java
java Programa
```

5.1.5 Gerenciadores de pacotes

Gerenciadores de pacotes são ferramentas usadas para instalar, atualizar e remover bibliotecas, frameworks e dependências de software.

Eles evitam que o desenvolvedor precise baixar arquivos manualmente, copiar bibliotecas e configurar tudo de forma artesanal.

Ferramenta	Ecossistema	Uso comum
`pip`	Python	Instalar bibliotecas Python.
`npm`	JavaScript/Node.js	Instalar pacotes JavaScript.
`composer`	PHP	Gerenciar dependências PHP.
`maven`	Java	Gerenciar projetos e dependências Java.
`cargo`	Rust	Compilar e gerenciar pacotes Rust.
`apt`	Debian/Ubuntu	Instalar pacotes do sistema operacional.

Exemplo didático

Em Python, podemos instalar o Flask com:

```
pip install flask
```

Em um projeto Node.js, podemos instalar o Express com:

```
npm install express
```

Observação

Gerenciadores de pacotes facilitam o desenvolvimento, mas também exigem cuidado. Dependências desatualizadas, vulneráveis ou incompatíveis podem causar problemas de segurança e manutenção.

5.2 Interpretadores e compiladores

Linguagens de programação precisam ser executadas por computadores, mas o processador não entende diretamente linguagens como Python, Java, C ou JavaScript. O código escrito pelo programador precisa ser traduzido, interpretado ou executado em um

ambiente apropriado.

Essa etapa é fundamental para entender por que diferentes linguagens têm formas distintas de execução, desempenho, portabilidade e distribuição.

5.2.1 Linguagens compiladas

Em linguagens compiladas, o código-fonte é traduzido antes da execução para um formato mais próximo da máquina. Esse processo é feito por um compilador.

Depois da compilação, geralmente é gerado um arquivo executável ou algum formato intermediário.

Exemplo didático

Em C, um arquivo `programa.c` pode ser compilado assim:

```
gcc programa.c -o programa
```

Depois, o programa pode ser executado:

```
./programa
```

Entre as características comuns de linguagens compiladas, estão:

- necessidade de etapa de compilação;
- detecção de muitos erros antes da execução;
- geração de executáveis;
- maior controle sobre recursos do sistema;
- desempenho geralmente elevado;
- dependência da arquitetura ou plataforma de destino, em muitos casos.

Observação

Compilar um programa não significa apenas "transformar em executável". O compilador analisa o código, verifica erros, otimiza instruções e gera um resultado que pode ser executado pela máquina ou por outro ambiente.

5.2.2 Linguagens interpretadas

Em linguagens interpretadas, o código é executado por um interpretador. Em vez de gerar previamente um executável final, o interpretador lê e executa as instruções.

Python, JavaScript e Ruby são frequentemente apresentados como exemplos de linguagens interpretadas, embora suas implementações modernas possam envolver etapas internas de compilação para bytecode ou otimizações em tempo de execução.

Exemplo didático

Em Python, podemos executar diretamente:

```
python programa.py
```

O arquivo `programa.py` contém o código-fonte, e o interpretador Python é responsável por executar suas instruções.

Entre as características comuns de linguagens interpretadas, estão:

- execução direta a partir do código-fonte ou de formato intermediário;
- facilidade para testes rápidos;
- maior flexibilidade durante o desenvolvimento;
- dependência de um interpretador instalado;
- desempenho geralmente menor que linguagens compiladas de baixo nível, dependendo do contexto;
- forte uso em automação, scripts, web, ciência de dados e prototipação.

5.2.3 Máquinas virtuais de linguagem

Algumas linguagens usam uma estratégia intermediária: o código-fonte é compilado para um formato chamado bytecode, que depois é executado por uma máquina virtual de linguagem.

A máquina virtual de linguagem não é o mesmo que uma máquina virtual de sistema, como VirtualBox ou VMware. Ela é um ambiente de execução para programas de uma linguagem ou plataforma específica.

Exemplo didático

Em Java, o código-fonte é compilado para bytecode:

```
javac Programa.java
```

Esse processo gera um arquivo `Programa.class`. Depois, o programa é executado pela JVM:

```
java Programa
```

A JVM, ou Java Virtual Machine, permite que o mesmo bytecode seja executado em diferentes sistemas operacionais, desde que exista uma JVM compatível instalada.

Observação

A frase "escreva uma vez, execute em qualquer lugar", frequentemente associada ao Java, está relacionada à ideia de compilar para bytecode e executar esse bytecode em uma máquina virtual de linguagem.

Modelo	**Como funciona**	**Exemplos**
Compilado	Código é traduzido antes da execução.	C, C++, Rust, Go.
Interpretado	Código é executado por um interpretador.	Python, Ruby, JavaScript.
Bytecode + VM	Código é compilado para bytecode e executado por uma máquina virtual.	Java, Kotlin, C#.

5.3 Ambientes locais de desenvolvimento

O ambiente local de desenvolvimento é o conjunto de ferramentas instalado no computador do desenvolvedor para criar, executar e testar sistemas antes de publicá-los em outros ambientes.

Ele pode incluir linguagem de programação, editor, terminal, banco de dados, servidor local, dependências, variáveis de ambiente, Git, contêineres e arquivos de configuração.

5.3.1 Instalação de linguagens

Para desenvolver em uma linguagem, normalmente é necessário instalar seu interpretador, compilador, SDK ou runtime.

Exemplo didático

Para desenvolver em Python, é necessário ter o Python instalado:

```
python3 --version
```

Para desenvolver em Node.js, verificamos:

```
node --version
npm --version
```

Para desenvolver em Java, podemos verificar:

```
java --version
javac --version
```

Observação

Instalar uma linguagem não significa apenas copiar um programa para o computador. Muitas vezes, a instalação configura caminhos, comandos, bibliotecas padrão, ferramentas auxiliares e integração com o terminal.

5.3.2 Variáveis de ambiente

Variáveis de ambiente são valores configurados no sistema operacional que podem ser lidos por programas em execução. Elas são usadas para armazenar configurações do ambiente sem colocá-las diretamente no código-fonte.

Exemplos de uso:

- definir modo de execução da aplicação;
- informar endereço de banco de dados;
- armazenar nome de usuário;
- indicar caminho de arquivos;
- guardar chaves e tokens em ambiente controlado;

- configurar portas de servidores locais.

Exemplo didático

Em Linux, podemos definir uma variável temporária no terminal:

```
export FLASK_ENV=development
export PORT=5000
```

Em Python, podemos ler uma variável de ambiente:

```
import os

porta = os.getenv("PORT", "5000")
print(porta)
```

Observação

Variáveis de ambiente são úteis para separar código e configuração. Isso permite que a mesma aplicação seja executada com configurações diferentes em desenvolvimento, teste e produção.

5.3.3 PATH

A variável de ambiente `PATH` informa ao sistema operacional em quais diretórios ele deve procurar programas executáveis quando um comando é digitado no terminal.

Quando executamos:

```
python
```

ou:

```
git
```

o sistema procura esses executáveis nos diretórios listados em `PATH`.

Exemplo didático

Em Linux, podemos visualizar o conteúdo do `PATH` com:

```
echo $PATH
```

Em Windows, no PowerShell, podemos usar:

```
$env:Path
```

Observação

Muitos erros do tipo “comando não encontrado” indicam que o programa não está instalado ou que seu diretório não está configurado corretamente no `PATH`.

5.3.4 Ambientes virtuais

Ambientes virtuais permitem isolar dependências de um projeto. Em vez de instalar todas as bibliotecas globalmente no sistema, cada projeto pode ter seu próprio conjunto de pacotes.

Isso evita conflitos entre projetos que usam versões diferentes das mesmas bibliotecas.

Exemplo didático

Em Python, podemos criar um ambiente virtual com:

```
python3 -m venv venv
```

Ativar no Linux ou macOS:

```
source venv/bin/activate
```

Ativar no Windows:

```
venv\Scripts\activate
```

Depois, instalamos as dependências dentro desse ambiente:

```
pip install flask
```

É comum registrar as dependências em um arquivo:

```
pip freeze > requirements.txt
```

E depois reinstalá-las em outro ambiente:

```
pip install -r requirements.txt
```

Observação

Ambientes virtuais e contêineres resolvem problemas relacionados, mas em níveis diferentes. O ambiente virtual isola dependências de uma linguagem, como Python. O contêiner isola um ambiente mais amplo, incluindo sistema base, arquivos, processos, rede e dependências do sistema.

5.4 Controle de versão com Git

Controle de versão é uma prática essencial no desenvolvimento de software. Ele permite registrar o histórico de alterações de um projeto, comparar versões, recuperar estados anteriores, trabalhar em equipe e organizar mudanças de forma controlada.

Git é um dos sistemas de controle de versão mais utilizados no mundo. Ele é distribuído, rápido e amplamente integrado a plataformas como GitHub, GitLab e Bitbucket.

5.4.1 Repositórios

Um repositório Git é uma pasta de projeto acompanhada de um histórico de versões. Esse histórico fica armazenado em um diretório oculto chamado `.git`.

Para criar um repositório em uma pasta, usamos:

```
git init
```

Depois, podemos verificar o estado do projeto com:

```
git status
```

Observação

O diretório `.git` é essencial para o funcionamento do Git. Se ele for removido, a pasta deixa de ser um repositório Git, embora os arquivos do projeto possam continuar existindo.

5.4.2 Commits

Um commit é um registro de uma alteração no projeto. Ele funciona como um ponto no histórico, contendo informações sobre quais arquivos mudaram, quando mudaram e qual mensagem descreve a mudança.

O fluxo básico é:

```
git add .
git commit -m "mensagem do commit"
```

O comando `git add` seleciona arquivos para o próximo commit. O comando `git commit` registra a mudança no histórico.

Exemplo didático

Depois de criar a primeira versão de uma aplicação, podemos registrar:

```
git add .
git commit -m "cria estrutura inicial da aplicao"
```

Observação

Mensagens de commit devem ser claras. Elas ajudam o próprio desenvolvedor e a equipe a entenderem o motivo das mudanças no futuro.

5.4.3 Branches

Branches são linhas alternativas de desenvolvimento. Elas permitem trabalhar em novas funcionalidades, correções ou experimentos sem alterar diretamente a linha principal do projeto.

Para criar uma nova branch:

```
git branch nova-funcionalidade
```

Para mudar para ela:

```
git checkout nova-funcionalidade
```

Ou, em versões recentes:

```
git switch -c nova-funcionalidade
```

Exemplo didático

Uma equipe pode manter a branch `main` com a versão estável do sistema e criar uma branch `login-google` para desenvolver uma nova funcionalidade de autenticação.

Depois que o trabalho é concluído, as mudanças podem ser integradas à branch principal por meio de merge ou pull request, dependendo do fluxo da equipe.

5.4.4 Repositórios remotos

Um repositório remoto é uma cópia do projeto hospedada em outro local, normalmente em uma plataforma como GitHub, GitLab ou Bitbucket.

Ele permite colaboração entre desenvolvedores, backup do código e integração com ferramentas de revisão, automação e implantação.

Para associar um repositório local a um remoto:

```
git remote add origin URL_DO_REPOSITORIO
```

Para enviar commits:

```
git push origin main
```

Para baixar mudanças:

```
git pull
```

Observação

Git não é apenas uma ferramenta para "salvar código na nuvem". Ele registra decisões, permite colaboração, organiza versões e ajuda a manter a história técnica do projeto.

5.5 Servidores locais

Durante o desenvolvimento, é comum executar aplicações no próprio computador antes de publicá-las em um servidor real. Esse processo usa servidores locais.

Um servidor local é um programa que escuta requisições em uma porta da própria máquina, geralmente acessível por endereços como:

```
http://localhost:8000
http://127.0.0.1:5000
```

Observação

`localhost` representa a própria máquina. O endereço `127.0.0.1` é conhecido como endereço de loopback e também aponta para o próprio computador.

5.5.1 Servidor HTTP simples

Um servidor HTTP simples pode ser usado para disponibilizar arquivos de um diretório local pelo navegador.

Em Python, podemos executar:

```
python3 -m http.server 8000
```

Esse comando inicia um servidor HTTP na porta 8000. Depois, basta acessar no navegador:

```
http://localhost:8000
```

Exemplo didático

Se o diretório atual contém um arquivo `index.html`, o servidor pode exibi-lo no navegador. Isso é útil para testar páginas HTML, CSS, JavaScript e arquivos estáticos.

Observação

Esse servidor é útil para desenvolvimento e testes locais, mas não deve ser usado como servidor de produção. Ele não possui todos os recursos, otimizações e controles de segurança necessários para ambientes públicos.

5.6 Da máquina local para a produção

O caminho entre escrever código no computador pessoal e disponibilizar uma aplicação para usuários reais envolve vários ambientes. Cada ambiente possui uma finalidade e um nível de controle.

Em projetos profissionais, é importante separar desenvolvimento local, testes, homologação e produção.

5.6.1 Desenvolvimento local

O desenvolvimento local ocorre no computador do desenvolvedor. É o ambiente usado para escrever código, executar testes iniciais, depurar erros e experimentar soluções.

Exemplo didático

Um desenvolvedor executa uma aplicação Flask em:

```
http://localhost:5000
```

Esse ambiente é local. Ele serve para desenvolvimento, não para acesso público por usuários finais.

5.6.2 Ambiente de testes

O ambiente de testes é usado para verificar se o sistema funciona de forma correta. Ele pode executar testes automatizados, validar regras de negócio, verificar integração entre componentes e identificar erros antes da publicação.

Exemplo didático

Uma equipe pode configurar um ambiente de testes que executa automaticamente:

```
pytest
npm test
```

sempre que uma nova alteração é enviada ao repositório.

5.6.3 Homologação

Homologação é um ambiente intermediário, geralmente parecido com a produção, usado para validação final antes da publicação.

Nele, usuários-chave, clientes, professores, gestores ou equipe técnica podem testar a aplicação em condições mais próximas do uso real.

Observação

O ambiente de homologação não deve usar dados sensíveis reais sem cuidados adequados. Sempre que possível, devem ser usados dados fictícios, anonimizados ou controlados.

5.6.4 Produção

Produção é o ambiente real, acessado pelos usuários finais. Nesse ambiente, estabilidade, segurança, desempenho, backup, monitoramento e controle de alterações são fundamentais.

Exemplo didático

Uma aplicação acadêmica em produção pode ser acessada por estudantes, professores e técnicos. Uma falha nesse ambiente pode afetar usuários reais, dados reais e processos institucionais.

5.6.5 Deploy

Deploy é o processo de publicar uma aplicação em um ambiente onde ela será executada. Pode envolver copiar arquivos, instalar dependências, configurar variáveis de ambiente, reiniciar serviços, aplicar migrações de banco de dados e atualizar contêineres.

Exemplo didático

Um processo simples de deploy pode incluir:

```
git pull
pip install -r requirements.txt
flask db upgrade
sudo systemctl restart minha-aplicacao
```

Em ambientes com Docker, o deploy pode envolver:

```
docker compose pull
docker compose up -d --build
```

Observação

Deploy não deve ser tratado como uma cópia improvisada de arquivos. Ele é uma etapa crítica do ciclo de vida do software e deve ser documentado, testado e, sempre que possível, automatizado.

5.7 Relação com DevOps

DevOps é uma abordagem que busca aproximar desenvolvimento de software e operações de infraestrutura. Seu objetivo é tornar a entrega de sistemas mais rápida, confiável, automatizada e monitorável.

Embora DevOps envolva cultura, processos e colaboração, também depende de ferramentas técnicas como Git, contêineres, integração contínua, entrega contínua, monitoramento, automação e infraestrutura como código.

5.7.1 Automação

Automação consiste em transformar tarefas repetitivas em processos executáveis por scripts, ferramentas ou pipelines.

Exemplos de tarefas que podem ser automatizadas:

- instalação de dependências;
- execução de testes;
- formatação de código;
- construção de imagens Docker;

- deploy;
- backup;
- verificação de segurança;
- geração de documentação.

Exemplo didático

Um script simples pode preparar um ambiente Python:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

5.7.2 Integração contínua

Integração contínua, ou CI (*Continuous Integration*), é a prática de integrar mudanças de código com frequência e verificar automaticamente se elas não quebram o projeto.

Normalmente, a cada envio de código ao repositório, uma ferramenta executa etapas como:

- instalar dependências;
- rodar testes;
- verificar estilo de código;
- analisar vulnerabilidades;
- construir a aplicação.

Exemplo didático

Quando um estudante envia código para o GitLab, um pipeline pode executar automaticamente os testes do projeto. Se algum teste falhar, a equipe é avisada antes que o erro chegue ao ambiente de produção.

5.7.3 Entrega contínua

Entrega contínua, ou CD (*Continuous Delivery*), é a prática de manter o software sempre em condição de ser publicado. Isso não significa necessariamente publicar automaticamente toda alteração, mas garantir que o processo de entrega seja confiável e repetível.

Em alguns contextos, fala-se também em implantação contínua (*Continuous Deployment*), quando as mudanças aprovadas são publicadas automaticamente em produção.

Observação

A entrega contínua depende de testes confiáveis, automação, controle de versão, ambientes bem configurados e capacidade de reverter mudanças em caso de falha.

5.7.4 Monitoramento

Monitoramento é o acompanhamento contínuo do funcionamento de uma aplicação ou infraestrutura. Ele permite identificar falhas, lentidão, uso excessivo de recursos e comportamentos inesperados.

Podem ser monitorados:

- uso de CPU;
- uso de memória;
- espaço em disco;
- tempo de resposta;
- erros da aplicação;
- logs;
- disponibilidade do serviço;
- quantidade de requisições;
- falhas de banco de dados;
- consumo de rede.

Exemplo didático

Se uma API começa a responder lentamente, o monitoramento pode indicar se o problema está no uso de CPU, na memória, no banco de dados, na rede ou em uma mudança recente no código.

Observação

Colocar um sistema em produção sem monitoramento é como dirigir à noite com os faróis apagados. O sistema pode até funcionar, mas a equipe terá dificuldade para perceber e corrigir problemas rapidamente.

5.7.5 Atividade prática

Atividade prática

Escolha um pequeno projeto de software, real ou fictício, e descreva como ele passaria da máquina local para a produção.
Responda:

1. Quais ferramentas seriam usadas no ambiente local?
2. Como as dependências seriam instaladas?
3. O projeto usaria Git? Como?
4. Haveria ambiente de testes?
5. Haveria ambiente de homologação?
6. Como seria feito o deploy?
7. Que aspectos deveriam ser monitorados em produção?
8. Quais etapas poderiam ser automatizadas?

Apêndice A

Lista de Comandos Úteis

A.1 Comandos de arquivos e diretórios

```
pwd
ls
ls -l
cd
mkdir
touch
cp
mv
rm
tree
```

A.2 Comandos de processos

```
ps
ps aux
top
htop
kill
kill -9
```

A.3 Comandos de rede

```
ping
ip addr
```

```
ifconfig
curl
wget
ssh
scp
netstat
ss
```

A.4 Comandos de Git

```
git init
git status
git add .
git commit -m "mensagem"
git log
git branch
git checkout
git push
git pull
```

A.5 Comandos de Docker

```
docker ps
docker ps -a
docker images
docker run
docker build
docker stop
docker rm
docker rmi
docker compose up
docker compose down
```

Apêndice B

Glossário

Arquivo Unidade de armazenamento de dados em um sistema computacional.

Diretório Estrutura usada para organizar arquivos.

Processo Programa em execução.

Kernel Núcleo do sistema operacional.

Driver Software que permite ao sistema operacional controlar um dispositivo.

Virtualização Técnica que permite simular recursos computacionais.

Contêiner Ambiente isolado e leve para execução de aplicações.

Endereço IP Identificador de um dispositivo em uma rede.

SSH Protocolo usado para acesso remoto seguro via terminal.

Docker Plataforma para criação e execução de contêineres.